SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

DATABASE MANAGEMENT SYSTEM — PROJECT REPORT

Security Incident Attack-Path Database

TEAM MEMBERS

Member	Name	PRN	Module
Member 1	Shreya Panda	25070122181	Incident + Attack Path
Member 2	Sneha Sah	25070122189	Assets + Vulnerabilities
Member 3	Saurav Paudel	25070122174	Evidence & Investigation
Member 4	Shreeya Alwar	25070122180	Response & Transaction Management

Branch	Computer Science & Engineering (CSE)
Semester	III
Academic Year	2026–27

Table of Contents

Member 1: Incident and Attack Path 3
1.1 Problem statement and requirements 3
1.2 My tables and the shared ones 3
1.3 Relationships and relational schema 4
1.4 Anomalies, functional dependencies and normalization 5
1.5 Creating the tables and loading the data 6
1.6 Basic queries 8
1.7 Rebuilding the attack path with JOIN 10
1.8 GROUP BY and HAVING 12
1.9 Nested queries 15
1.10 Aggregate functions 19
1.11 Function, procedure, trigger and view 20
1.12 My contribution and conclusion 24

Member 2: Assets and Vulnerabilities 26
2.1 What this part covers 26
2.2 The ASSET table 26
2.3 Joins, GROUP BY and nested queries 27
2.4 Aggregate functions 29
2.5 Function, procedure, trigger and view 30
2.6 Dependencies, normalization and relationships 34
2.7 Database setup screenshots 34
2.8 Conclusion 39

Member 3: Evidence and Investigation 40
3.1 What this part covers 40
3.2 Tables in the database 40
3.3 SQL commands 1 to 16 41
3.4 ER diagram 51
3.5 EER diagram 52
3.6 Functional dependencies and normalization 53
3.7 ER relationships and cardinalities 54
3.8 Conclusion 54

Member 4: Response and Transaction Management 55
4.1 Member 4 Responsibility 55
4.2 Overview of Response & Transaction Management 56
4.3 SQL Implementation and Results — Commands 1–16 57
4.4 Response Workflow 70
4.5 ER Diagram — Security Incident Attack-Path Database 72
4.6 Enhanced Entity–Relationship (EER) Diagram 73
4.7 Functional Dependencies 74
4.8 Normalization 74
4.9 ER Relationships and Cardinalities 75
4.10 Transactions, Concurrency and Serializability 76
4.11 Conclusion 78
4.12 Appendix: Additional Implementation Notes and Screenshots 79

Member 1: Incident and Attack Path

This part covers the incidents, the attackers and the individual attack steps. Together they answer one question: how did the attack actually happen? I own three tables here (SECURITY_INCIDENT, ACTOR and ATTACK_EVENT) and bring in the asset, technique and software tables through joins.

1.1 Problem statement and requirements

Companies deal with phishing, ransomware and supply-chain attacks all the time, but the details usually end up scattered: what happened, who did it, which system was hit, which technique and tool were used, and in what order. For this project we built a Security Incident Attack-Path Database that keeps all of this in connected tables, so the sequence of an attack can be rebuilt and analysed with SQL.

The design borrows from two sources: VERIS for recording incidents, and MITRE ATT&CK for groups, techniques and software. All data in the tables is sample data made up for the project, not real incidents.

#	What the module must do	Where it is handled
FR1	Store every incident with its type, severity, date and summary	SECURITY_INCIDENT
FR2	Store attackers and link them to a MITRE group when known	ACTOR, MITRE_GROUP
FR3	Store each attack step with its incident, actor, asset, technique and software	ATTACK_EVENT
FR4	Keep the steps of an incident in order	Event_Order column
FR5	Rebuild the full attack path of an incident	JOIN query, ShowAttackPath, AttackPathView
FR6	Answer questions such as events per incident or per actor	GROUP BY, HAVING, nested queries, aggregates
FR7	Return the number of steps of a given incident	Function GetAttackStepCount
FR8	Reject invalid event order values automatically	Trigger CheckAttackOrder

1.2 My tables and the shared ones

Table	Owner	What it holds
SECURITY_INCIDENT	Member 1	One row per incident: year, month, type, severity, summary and how it was found
ACTOR	Member 1	One row per attacker, with the MITRE group if there is one
ATTACK_EVENT	Member 1	One row per attack step; links the incident, actor, asset, technique and software
ASSET	Member 2	The system that was attacked
MITRE_TECHNIQUE, MITRE_SOFTWARE, MITRE_GROUP	Shared	Reference tables for technique names, tools and threat groups

1.3 Relationships and relational schema

ATTACK_EVENT sits at the centre of the design. Each row points to one incident, one actor, one asset, one technique and, optionally, one piece of software. ACTOR can optionally point to a MITRE group. All six relationships are one-to-many.

Parent	Child	Foreign key	Note
SECURITY_INCIDENT	ATTACK_EVENT	Incident_ID	I006 has 4 events
ACTOR	ATTACK_EVENT	Actor_ID	APT29 performed 4 events
ASSET	ATTACK_EVENT	Asset_ID	One asset can be hit many times
MITRE_TECHNIQUE	ATTACK_EVENT	Technique_ID	T1078 appears in two incidents
MITRE_SOFTWARE	ATTACK_EVENT	Software_ID	Optional: 7 of 14 events have none
MITRE_GROUP	ACTOR	Group_ID	Optional

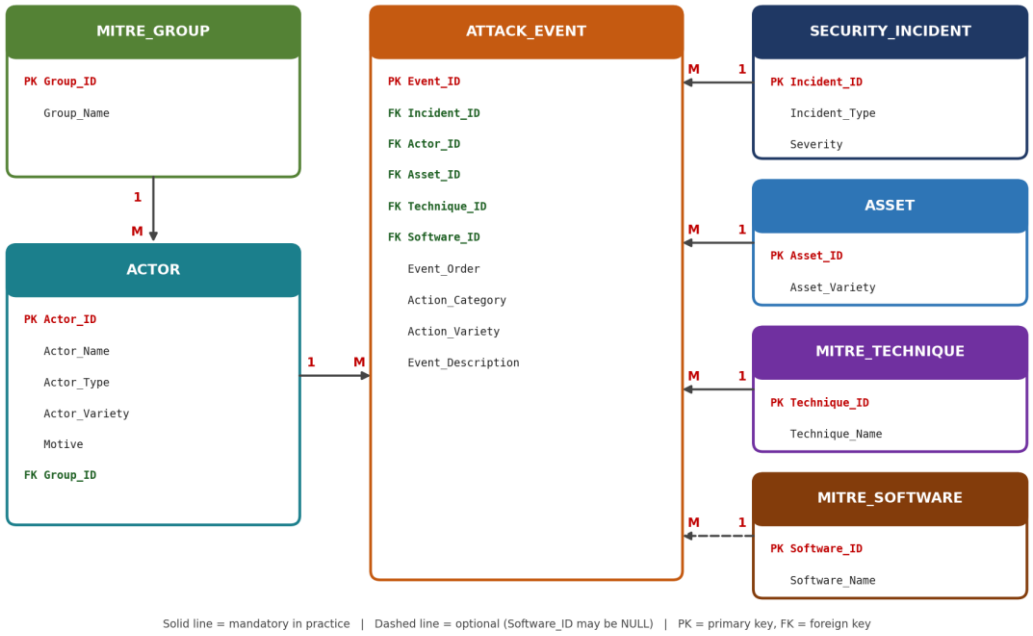


Figure 1.1: Foreign keys of the Incident + Attack Path module

The full ER and EER diagrams for the whole database are in Member 3's part.

There is no separate Attack_Path table, on purpose. The path is worked out when you query: every ATTACK_EVENT row with the same Incident_ID belongs to one incident, and sorting by Event_Order gives the steps in sequence. Joins then add the actor, asset, technique and software names. Storing the order once and deriving the path avoids keeping a second copy of the same information.

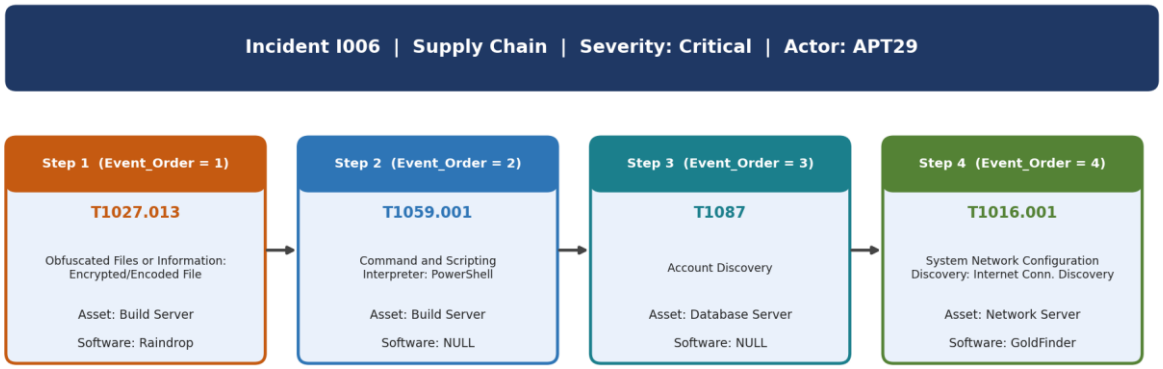


Figure 1.2: Attack path of incident I006, rebuilt from ATTACK_EVENT using Event_Order

1.4 Anomalies, functional dependencies and normalization

Imagine everything in one flat table, with the incident, actor, technique and asset details repeated on every attack step. For I006 that would mean four rows all repeating the same incident and actor information. Three problems follow:

Anomaly	What would go wrong
Update	If the details of APT29 or incident I006 change, every repeated row has to be edited, and missing one leaves the data inconsistent.
Insertion	A new MITRE technique could not be stored until some attack event uses it.
Deletion	Deleting the last event that uses a technique would also wipe out the only record of that technique.

Splitting the data into SECURITY_INCIDENT, ACTOR, ASSET, MITRE_TECHNIQUE, MITRE_SOFTWARE and ATTACK_EVENT, joined by primary and foreign keys, removes all three.

In each table the primary key decides every other column:

Table	Functional dependency
SECURITY_INCIDENT	Incident_ID → Incident_Year, Incident_Month, Security_Incident, Incident_Type, Severity, Summary, Discovery_Method, Confidence, Data_Source
ACTOR	Actor_ID → Actor_Name, Actor_Type, Actor_Variety, Motive, Group_ID
ATTACK_EVENT	Event_ID → Incident_ID, Actor_ID, Asset_ID, Technique_ID, Software_ID, Event_Order, Action_Category, Action_Variety, Event_Description
MITRE_TECHNIQUE	Technique_ID → Technique_Name, Tactic, Platform
MITRE_SOFTWARE	Software_ID → Software_Name, Software_Type, Platform

A couple of things look like dependencies but are not. Event_Order decides nothing on its own, because the value 1 appears in all six incidents. Action_Variety does not decide Action_Category either: Valid Accounts is stored under Credential Access in E006 and under Initial Access in E007.

Normal form	Why the tables satisfy it
1NF	Every cell holds a single value. Each attack step is its own ATTACK_EVENT row instead of “Step1, Step2, Step3” packed into the incident row.
2NF	All three tables have single-column primary keys, so a partial dependency cannot happen.

Normal form	Why the tables satisfy it
3NF	No non-key column depends on another non-key column. ATTACK_EVENT keeps only Actor_ID and Technique_ID; the names live in ACTOR and MITRE_TECHNIQUE.

The Platform column in the MITRE tables can hold comma-separated values, but that belongs to the shared tables rather than mine.

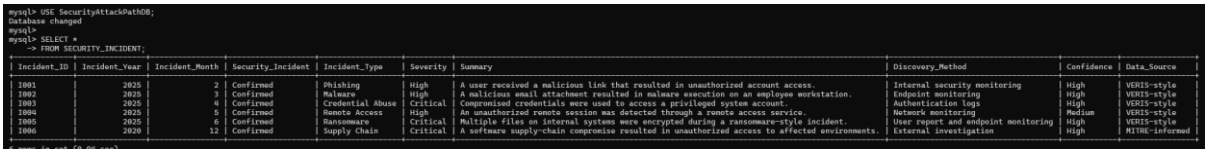
1.5 Creating the tables and loading the data

SECURITY_INCIDENT

The parent table of the module, with one row for each of the six incidents.

```
CREATE TABLE SECURITY_INCIDENT (
    Incident_ID       VARCHAR(10) PRIMARY KEY,
    Incident_Year     INT,
    Incident_Month    INT,
    Security_Incident VARCHAR(30),
    Incident_Type     VARCHAR(50),
    Severity          VARCHAR(20),
    Summary           VARCHAR(600),
    Discovery_Method  VARCHAR(100),
    Confidence        VARCHAR(20),
    Data_Source       VARCHAR(50)
);
```

ID	Year	Month	Type	Severity	Discovered through
I001	2025	2	Phishing	High	Internal security monitoring
I002	2025	3	Malware	High	Endpoint monitoring
I003	2025	4	Credential Abuse	Critical	Authentication logs
I004	2025	5	Remote Access	High	Network monitoring
I005	2025	6	Ransomware	Critical	User report and endpoint monitoring
I006	2020	12	Supply Chain	Critical	External investigation



ACTOR

One row per attacker. Group_ID optionally links an actor to a MITRE group.

```
CREATE TABLE ACTOR (
    Actor_ID      VARCHAR(10) PRIMARY KEY,
    Actor_Name    VARCHAR(100),
    Actor_Type    VARCHAR(50),
    Actor_Variety VARCHAR(100),
    Motive        VARCHAR(100),
    Group_ID      VARCHAR(10),
    FOREIGN KEY (Group_ID) REFERENCES MITRE_GROUP(Group_ID)
);
```

The actors that appear in attack events are below (all are external):

Actor	Attack events	Incidents
APT29	4	I006
APT3	2	I003
OilRig	2	I002
Wizard Spider	4	I001, I005
Unknown External Actor	2	I004

ATTACK_EVENT

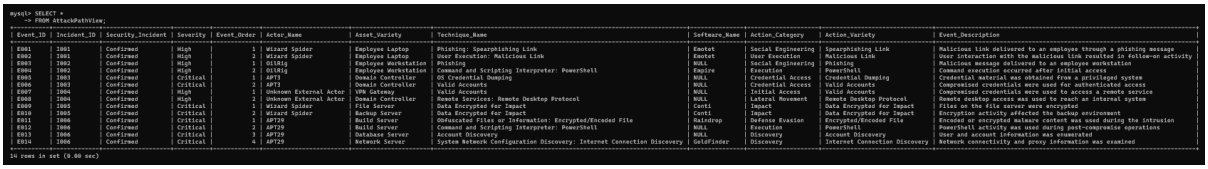
This is the most important table of my part. One row is one attack step, and it connects five other tables.

```
CREATE TABLE ATTACK_EVENT (
    Event_ID          VARCHAR(10) PRIMARY KEY,
    Incident_ID       VARCHAR(10),
    Actor_ID          VARCHAR(10),
    Asset_ID          VARCHAR(10),
    Technique_ID      VARCHAR(15),
    Software_ID       VARCHAR(10),
    Event_Order       INT,
    Action_Category   VARCHAR(100),
    Action_Variety    VARCHAR(100),
    Event_Description VARCHAR(500),
    FOREIGN KEY (Incident_ID)  REFERENCES SECURITY_INCIDENT(Incident_ID),
    FOREIGN KEY (Actor_ID)     REFERENCES ACTOR(Actor_ID),
    FOREIGN KEY (Asset_ID)     REFERENCES ASSET(Asset_ID),
    FOREIGN KEY (Technique_ID) REFERENCES MITRE_TECHNIQUE(Technique_ID),
    FOREIGN KEY (Software_ID)  REFERENCES MITRE_SOFTWARE(Software_ID)
);
```

Event_Order is the step number inside an incident, starting at 1. I001 to I005 have two steps each and I006 has four, the longest path in the database. The number is only unique within one incident (every incident has a step 1), so it always has to be used together with Incident_ID. The trigger in section 1.11 rejects zero and negative values.

Event	Incident	Order	Actor	Asset	Technique	Software
E001	I001	1	Wizard Spider	Employee Laptop	T1566.002	Emotet
E002	I001	2	Wizard Spider	Employee Laptop	T1204.001	Emotet
E003	I002	1	OilRig	Employee Workstation	T1566	NULL
E004	I002	2	OilRig	Employee Workstation	T1059.001	Empire
E005	I003	1	APT3	Domain Controller	T1003	NULL
E006	I003	2	APT3	Domain Controller	T1078	NULL
E007	I004	1	Unknown External Actor	VPN Gateway	T1078	NULL

Event	Incident	Order	Actor	Asset	Technique	Software
E008	I004	2	Unknown External Actor	Domain Controller	T1021.001	NULL
E009	I005	1	Wizard Spider	File Server	T1486	Conti
E010	I005	2	Wizard Spider	Backup Server	T1486	Conti
E011	I006	1	APT29	Build Server	T1027.013	Raindrop
E012	I006	2	APT29	Build Server	T1059.001	NULL
E013	I006	3	APT29	Database Server	T1087	NULL
E014	I006	4	APT29	Network Server	T1016.001	GoldFinder



1.6 Basic queries

High and Critical incidents

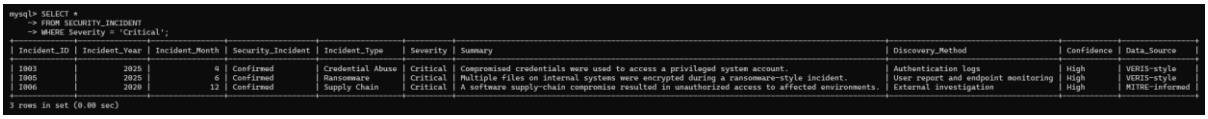
```
SELECT Incident_ID, Security_Incident, Incident_Type, Severity
FROM SECURITY_INCIDENT
WHERE Severity IN ('High', 'Critical');
```



IN lets the filter match either value. All six incidents come back, since every stored incident is High or Critical.

Critical incidents, all columns

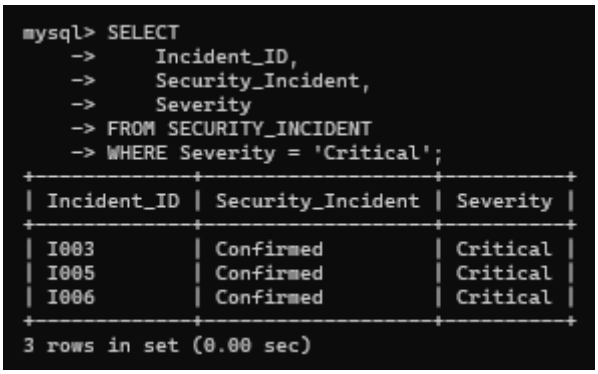
```
SELECT *
FROM SECURITY_INCIDENT
WHERE Severity = 'Critical';
```



Three rows: I003 (Credential Abuse), I005 (Ransomware) and I006 (Supply Chain).

Critical incidents, selected columns

```
SELECT Incident_ID, Security_Incident, Severity
FROM SECURITY_INCIDENT
WHERE Severity = 'Critical';
```



```
mysql> SELECT
    ->     Incident_ID,
    ->     Security_Incident,
    ->     Severity
    -> FROM SECURITY_INCIDENT
    -> WHERE Severity = 'Critical';
+-------------+-------------------+----------+
| Incident_ID | Security_Incident | Severity |
+-------------+-------------------+----------+
| I003        | Confirmed         | Critical |
| I005        | Confirmed         | Critical |
| I006        | Confirmed         | Critical |
+-------------+-------------------+----------+
3 rows in set (0.00 sec)
```

Same filter, but only three columns are shown. The same three incidents come back.

Attack events by external actors

```
SELECT A.Actor_Name, AE.Incident_ID, AE.Event_ID
FROM ACTOR A
JOIN ATTACK_EVENT AE ON A.Actor_ID = AE.Actor_ID
WHERE A.Actor_Type = 'External';
```



```
mysql> SELECT
    ->     A.Actor_Name,
    ->     AE.Incident_ID,
    ->     AE.Event_ID
    -> FROM ACTOR A
    ->
    -> JOIN ATTACK_EVENT AE
    ->     ON A.Actor_ID = AE.Actor_ID
    ->
    -> WHERE A.Actor_Type = 'External';
+-----------------------+-------------+----------+
| Actor_Name            | Incident_ID | Event_ID |
+-----------------------+-------------+----------+
| APT29                 | I006        | E011     |
| APT29                 | I006        | E012     |
| APT29                 | I006        | E013     |
| APT29                 | I006        | E014     |
| APT3                  | I003        | E005     |
| APT3                  | I003        | E006     |
| OilRig                | I002        | E003     |
| OilRig                | I002        | E004     |
| Wizard Spider         | I001        | E001     |
| Wizard Spider         | I001        | E002     |
| Wizard Spider         | I005        | E009     |
| Wizard Spider         | I005        | E010     |
| Unknown External Actor | I004        | E007     |
| Unknown External Actor | I004        | E008     |
+-----------------------+-------------+----------+
14 rows in set (0.00 sec)
```

All 14 events are returned, because every actor in the data is external.

Events that have software

```
SELECT AE.Event_ID, AE.Incident_ID, MS.Software_Name, MT.Technique_Name
FROM ATTACK_EVENT AE
JOIN MITRE_SOFTWARE MS  ON AE.Software_ID  = MS.Software_ID
JOIN MITRE_TECHNIQUE MT ON AE.Technique_ID = MT.Technique_ID;
```

```
mysql> SELECT
    ->     AE.Event_ID,
    ->     AE.Incident_ID,
    ->     MS.Software_Name,
    ->     MT.Technique_Name
    -> FROM ATTACK_EVENT AE
    ->
    -> JOIN MITRE_SOFTWARE MS
    ->     ON AE.Software_ID = MS.Software_ID
    ->
    -> JOIN MITRE_TECHNIQUE MT
    ->     ON AE.Technique_ID = MT.Technique_ID;
+----------+-------------+---------------+-----------------------------------------------------------------+
| Event_ID | Incident_ID | Software_Name | Technique_Name                                                  |
+----------+-------------+---------------+-----------------------------------------------------------------+
| E001     | I001        | Emotet        | Phishing: Spearphishing Link                                    |
| E002     | I001        | Emotet        | User Execution: Malicious Link                                  |
| E004     | I002        | Empire        | Command and Scripting Interpreter: PowerShell                   |
| E009     | I005        | Conti         | Data Encrypted for Impact                                       |
| E010     | I005        | Conti         | Data Encrypted for Impact                                       |
| E011     | I006        | Raindrop      | Obfuscated Files or Information: Encrypted/Encoded File         |
| E014     | I006        | GoldFinder    | System Network Configuration Discovery: Internet Connection Discovery |
+----------+-------------+---------------+-----------------------------------------------------------------+
7 rows in set (0.00 sec)
```

The inner join drops events whose Software_ID is NULL, leaving seven: E001 and E002 (Emotet), E004 (Empire), E009 and E010 (Conti), E011 (Raindrop) and E014 (GoldFinder).

Events without software

```
SELECT Event_ID, Incident_ID, Technique_ID
FROM ATTACK_EVENT
WHERE Software_ID IS NULL;
```

```
mysql> SELECT
    ->     Event_ID,
    ->     Incident_ID,
    ->     Technique_ID
    -> FROM ATTACK_EVENT
    -> WHERE Software_ID IS NULL;
+----------+-------------+--------------+
| Event_ID | Incident_ID | Technique_ID |
+----------+-------------+--------------+
| E003     | I002        | T1566        |
| E005     | I003        | T1003        |
| E006     | I003        | T1078        |
| E007     | I004        | T1078        |
| E008     | I004        | T1021.001    |
| E012     | I006        | T1059.001    |
| E013     | I006        | T1087        |
+----------+-------------+--------------+
7 rows in set (0.00 sec)
```

NULL cannot be compared with =, so IS NULL is used. The other seven events come back (E003, E005, E006, E007, E008, E012, E013), which together with the previous query makes all 14.

1.7 Rebuilding the attack path with JOIN

This is the main query of my part. ATTACK_EVENT is in the middle and the other five tables are joined to it. The software join is a LEFT JOIN so steps with no software are not lost.

Attack path of every incident

```
SELECT AE.Incident_ID, SI.Security_Incident, SI.Severity, AE.Event_Order,
       A.Actor_Name, ASSET.Asset_Variety, MT.Technique_ID, MT.Technique_Name,
       MS.Software_Name, AE.Action_Category, AE.Action_Variety, AE.Event_Description
FROM ATTACK_EVENT AE
JOIN SECURITY_INCIDENT SI ON AE.Incident_ID  = SI.Incident_ID
JOIN ACTOR A              ON AE.Actor_ID     = A.Actor_ID
JOIN ASSET                ON AE.Asset_ID     = ASSET.Asset_ID
JOIN MITRE_TECHNIQUE MT   ON AE.Technique_ID = MT.Technique_ID
LEFT JOIN MITRE_SOFTWARE MS ON AE.Software_ID = MS.Software_ID
ORDER BY AE.Incident_ID, AE.Event_Order;
```



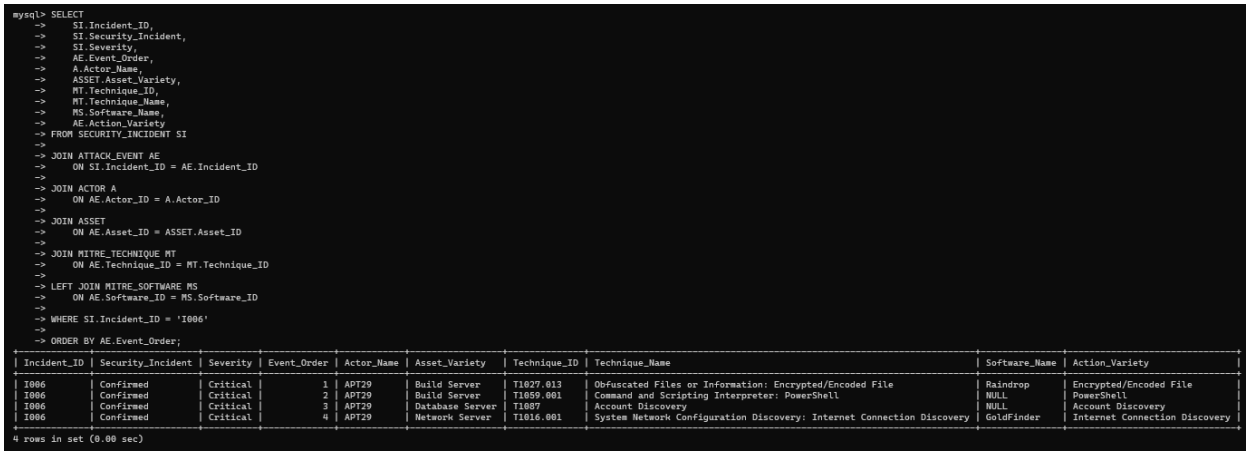
Incident_ID	Security_Incident	Severity	Event_Order	Actor_Name	Asset_Variety	Technique_ID	Technique_Name	Software_Name	Action_Category	Action_Variety	Event_Description
I001	Confirmed	High	1	Wizard Spider	Employee Laptop	T1566.002	Phishing: Spearphishing Link	Emotet	Social Engineering	Spearphishing Link	Malicious link delivered to an employee through a phishing message
I001	Confirmed	High	2	Wizard Spider	Employee Laptop	T1204.001	User Execution: Malicious Link	Emotet	User Execution	Malicious Link	User interaction with the malicious link resulted in follow-on activity
I002	Confirmed	High	1	OilRig	Employee Workstation	T1566	Phishing	NULL	Social Engineering	Phishing	Malicious message delivered to an employee workstation
I002	Confirmed	High	2	OilRig	Employee Workstation	T1059.001	Command and Scripting Interpreter: PowerShell	Empire	Execution	PowerShell	Command execution occurred after initial access
I003	Confirmed	Critical	1	APT3	Domain Controller	T1003	OS Credential Dumping	NULL	Credential Access	Credential Dumping	Credential material was obtained from a privileged system
I003	Confirmed	Critical	2	APT3	Domain Controller	T1078	Valid Accounts	NULL	Credential Access	Valid Accounts	Compromised credentials were used for authenticated access
I004	Confirmed	High	1	Unknown External Actor	VPN Gateway	T1078	Valid Accounts	NULL	Initial Access	Valid Accounts	Compromised credentials were used to access a remote service
I004	Confirmed	High	2	Unknown External Actor	Domain Controller	T1021.001	Remote Services: Remote Desktop Protocol	NULL	Lateral Movement	Remote Desktop Protocol	Remote desktop access was used to reach an internal system
I005	Confirmed	Critical	1	Wizard Spider	File Server	T1486	Data Encrypted for Impact	Conti	Impact	Data Encrypted for Impact	Files on the file server were encrypted
I005	Confirmed	Critical	2	Wizard Spider	Backup Server	T1486	Data Encrypted for Impact	Conti	Impact	Data Encrypted for Impact	Encryption activity affected the backup environment
I006	Confirmed	Critical	1	APT29	Build Server	T1027.013	Obfuscated Files or Information: Encrypted/Encoded File	Raindrop	Defense Evasion	Encrypted/Encoded File	Encoded or encrypted malware content was used during the intrusion
I006	Confirmed	Critical	2	APT29	Build Server	T1059.001	Command and Scripting Interpreter: PowerShell	NULL	Execution	PowerShell	PowerShell activity was used during post-compromise operations
I006	Confirmed	Critical	3	APT29	Database Server	T1087	Account Discovery	NULL	Discovery	Account Discovery	User and account information was enumerated
I006	Confirmed	Critical	4	APT29	Network Server	T1016.001	System Network Configuration Discovery: Internet Connection Discovery	GoldFinder	Discovery	Internet Connection Discovery	Network connectivity and proxy information was examined

14 rows in set (0.02 sec)

14 rows, one per attack event, in incident and step order. The NULLs in Software_Name are the steps with no recorded software; a normal JOIN would have removed all seven of those rows.

Attack path of incident I006

```
SELECT SI.Incident_ID, SI.Security_Incident, SI.Severity, AE.Event_Order,
       A.Actor_Name, ASSET.Asset_Variety, MT.Technique_ID, MT.Technique_Name,
       MS.Software_Name, AE.Action_Variety
FROM SECURITY_INCIDENT SI
JOIN ATTACK_EVENT AE        ON SI.Incident_ID  = AE.Incident_ID
JOIN ACTOR A                ON AE.Actor_ID     = A.Actor_ID
JOIN ASSET                  ON AE.Asset_ID     = ASSET.Asset_ID
JOIN MITRE_TECHNIQUE MT     ON AE.Technique_ID = MT.Technique_ID
LEFT JOIN MITRE_SOFTWARE MS ON AE.Software_ID  = MS.Software_ID
WHERE SI.Incident_ID = 'I006'
ORDER BY AE.Event_Order;
```

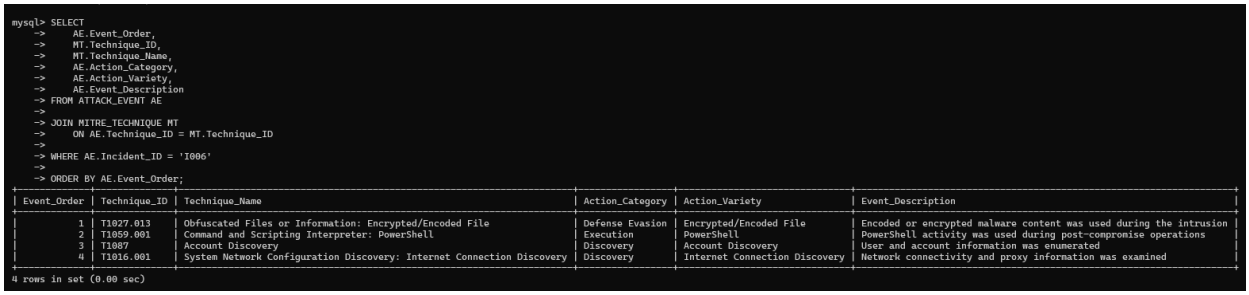


```
mysql> SELECT
    ->     SI.Incident_ID,
    ->     SI.Security_Incident,
    ->     SI.Severity,
    ->     AE.Event_Order,
    ->     A.Actor_Name,
    ->     ASSET.Asset_Variety,
    ->     MT.Technique_ID,
    ->     MT.Technique_Name,
    ->     MS.Software_Name,
    ->     AE.Action_Variety
    -> FROM SECURITY_INCIDENT SI
    ->
    -> JOIN ATTACK_EVENT AE
    ->     ON SI.Incident_ID = AE.Incident_ID
    ->
    -> JOIN ACTOR A
    ->     ON AE.Actor_ID = A.Actor_ID
    ->
    -> JOIN ASSET
    ->     ON AE.Asset_ID = ASSET.Asset_ID
    ->
    -> JOIN MITRE_TECHNIQUE MT
    ->     ON AE.Technique_ID = MT.Technique_ID
    ->
    -> LEFT JOIN MITRE_SOFTWARE MS
    ->     ON AE.Software_ID = MS.Software_ID
    ->
    -> WHERE SI.Incident_ID = 'I006'
    ->
    -> ORDER BY AE.Event_Order;
+-------------+-------------------+----------+-------------+------------+---------------+---------------+-----------------------------------------------------------------------+---------------+-----------------------------+
| Incident_ID | Security_Incident | Severity | Event_Order | Actor_Name | Asset_Variety | Technique_ID  | Technique_Name                                                        | Software_Name | Action_Variety              |
+-------------+-------------------+----------+-------------+------------+---------------+---------------+-----------------------------------------------------------------------+---------------+-----------------------------+
| I006        | Confirmed         | Critical |           1 | APT29      | Build Server  | T1027.013     | Obfuscated Files or Information: Encrypted/Encoded File               | Raindrop      | Encrypted/Encoded File      |
| I006        | Confirmed         | Critical |           2 | APT29      | Build Server  | T1059.001     | Command and Scripting Interpreter: PowerShell                         | NULL          | PowerShell                  |
| I006        | Confirmed         | Critical |           3 | APT29      | Database Server | T1087       | Account Discovery                                                     | NULL          | Account Discovery           |
| I006        | Confirmed         | Critical |           4 | APT29      | Network Server | T1016.001    | System Network Configuration Discovery: Internet Connection Discovery | GoldFinder    | Internet Connection Discovery |
+-------------+-------------------+----------+-------------+------------+---------------+---------------+-----------------------------------------------------------------------+---------------+-----------------------------+
4 rows in set (0.00 sec)
```

Four steps, all by APT29: Raindrop (obfuscated files), PowerShell, account discovery, and internet connection discovery with GoldFinder.

I006 with technique details only

```
SELECT AE.Event_Order, MT.Technique_ID, MT.Technique_Name,
       AE.Action_Category, AE.Action_Variety, AE.Event_Description
FROM ATTACK_EVENT AE
JOIN MITRE_TECHNIQUE MT ON AE.Technique_ID = MT.Technique_ID
WHERE AE.Incident_ID = 'I006'
ORDER BY AE.Event_Order;
```



```
mysql> SELECT
    ->     AE.Event_Order,
    ->     MT.Technique_ID,
    ->     MT.Technique_Name,
    ->     AE.Action_Category,
    ->     AE.Action_Variety,
    ->     AE.Event_Description
    -> FROM ATTACK_EVENT AE
    ->
    -> JOIN MITRE_TECHNIQUE MT
    ->     ON AE.Technique_ID = MT.Technique_ID
    ->
    -> WHERE AE.Incident_ID = 'I006'
    ->
    -> ORDER BY AE.Event_Order;
+-------------+--------------+-----------------------------------------------------------+-----------------+-----------------------+-----------------------------------------------------------+
| Event_Order | Technique_ID | Technique_Name                                            | Action_Category | Action_Variety        | Event_Description                                         |
+-------------+--------------+-----------------------------------------------------------+-----------------+-----------------------+-----------------------------------------------------------+
|           1 | T1027.013    | Obfuscated Files or Information: Encrypted/Encoded File   | Defense Evasion | Encrypted/Encoded File | Encoded or encrypted malware content was used during the intrusion |
|           2 | T1059.001    | Command and Scripting Interpreter: PowerShell             | Execution       | PowerShell            | PowerShell activity was used during post-compromise operations |
|           3 | T1087        | Account Discovery                                         | Discovery       | Account Discovery     | User and account information was enumerated               |
|           4 | T1016.001    | System Network Configuration Discovery: Internet Connection Discovery | Discovery | Internet Connection Discovery | Network connectivity and proxy information was examined |
+-------------+--------------+-----------------------------------------------------------+-----------------+-----------------------+-----------------------------------------------------------+
4 rows in set (0.00 sec)
```

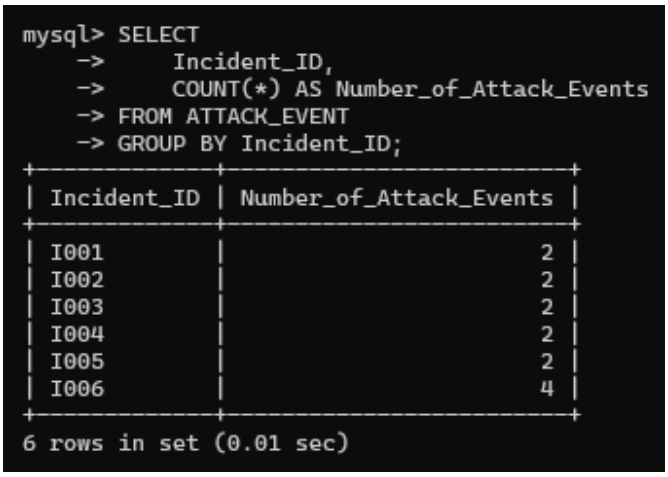
A smaller version of the same path without actor, asset or software: T1027.013, T1059.001, T1087 and T1016.001 in order.

1.8 GROUP BY and HAVING

GROUP BY collects rows with the same value into a group so that COUNT can work on each one. HAVING then filters those groups.

Attack events per incident

```
SELECT Incident_ID, COUNT(*) AS Number_of_Attack_Events
FROM ATTACK_EVENT
GROUP BY Incident_ID;
```



```
mysql> SELECT
    ->     Incident_ID,
    ->     COUNT(*) AS Number_of_Attack_Events
    -> FROM ATTACK_EVENT
    -> GROUP BY Incident_ID;
+-------------+--------------------------+
| Incident_ID | Number_of_Attack_Events  |
+-------------+--------------------------+
| I001        |                        2 |
| I002        |                        2 |
| I003        |                        2 |
| I004        |                        2 |
| I005        |                        2 |
| I006        |                        4 |
+-------------+--------------------------+
6 rows in set (0.01 sec)
```

I001 to I005 have two events each and I006 has four.

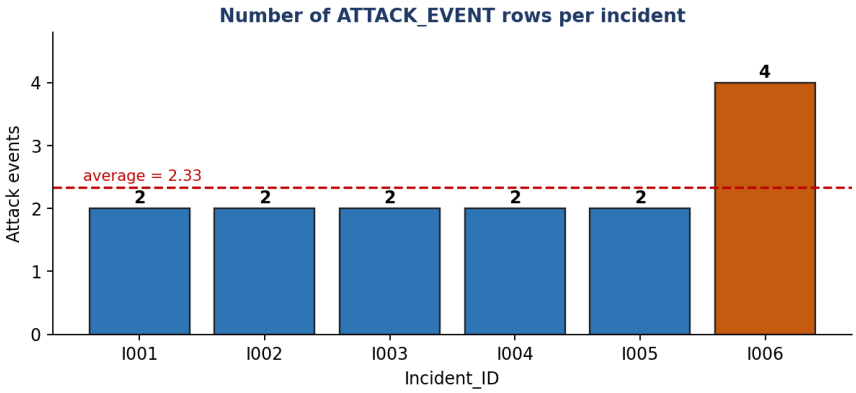


Figure 1.3: Attack events per incident (dashed line = average, 14 / 6 = 2.33)

Incidents with more than one event

```
SELECT Incident_ID, COUNT(*) AS Attack_Event_Count
FROM ATTACK_EVENT
GROUP BY Incident_ID
HAVING COUNT(*) > 1;
```

```
mysql> SELECT
    ->     Incident_ID,
    ->     COUNT(*) AS Attack_Event_Count
    -> FROM ATTACK_EVENT
    -> GROUP BY Incident_ID
    -> HAVING COUNT(*) > 1;
+-------------+---------------------+
| Incident_ID | Attack_Event_Count  |
+-------------+---------------------+
| I001        |                   2 |
| I002        |                   2 |
| I003        |                   2 |
| I004        |                   2 |
| I005        |                   2 |
| I006        |                   4 |
+-------------+---------------------+
6 rows in set (0.00 sec)
```

All six qualify, so HAVING does not remove anything here.

Events per actor

```
SELECT Actor_ID, COUNT(*) AS Total_Attacks
FROM ATTACK_EVENT
GROUP BY Actor_ID;
```

```
mysql> SELECT
    ->     Actor_ID,
    ->     COUNT(*) AS Total_Attacks
    -> FROM ATTACK_EVENT
    -> GROUP BY Actor_ID;
+----------+---------------+
| Actor_ID | Total_Attacks |
+----------+---------------+
| A002     |             4 |
| A003     |             2 |
| A004     |             2 |
| A005     |             4 |
| A006     |             2 |
+----------+---------------+
5 rows in set (0.00 sec)
```

Grouping directly on the foreign key, no join needed. Five actors: two with 4 events and three with 2.

Actor names with their number of attacks

```
SELECT A.Actor_Name, COUNT(AE.Event_ID) AS Total_Attacks
FROM ACTOR A
JOIN ATTACK_EVENT AE ON A.Actor_ID = AE.Actor_ID
GROUP BY A.Actor_ID, A.Actor_Name;
```

```
mysql> SELECT
    ->     A.Actor_Name,
    ->     COUNT(AE.Event_ID) AS Total_Attacks
    -> FROM ACTOR A
    ->
    -> JOIN ATTACK_EVENT AE
    ->     ON A.Actor_ID = AE.Actor_ID
    ->
    -> GROUP BY A.Actor_ID, A.Actor_Name;
+----------------------+---------------+
| Actor_Name           | Total_Attacks |
+----------------------+---------------+
| APT29                |             4 |
| APT3                 |             2 |
| OilRig               |             2 |
| Wizard Spider        |             4 |
| Unknown External Actor |             2 |
+----------------------+---------------+
5 rows in set (0.01 sec)
```

Same counts, with readable names: APT29 4, APT3 2, OilRig 2, Wizard Spider 4 and Unknown External Actor 2.

Actors with more than one attack

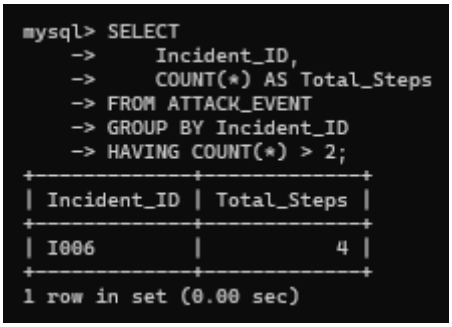
```
SELECT A.Actor_Name, COUNT(AE.Event_ID) AS Total_Attacks
FROM ACTOR A
JOIN ATTACK_EVENT AE ON A.Actor_ID = AE.Actor_ID
GROUP BY A.Actor_ID, A.Actor_Name
HAVING COUNT(AE.Event_ID) > 1;
```

```
mysql> SELECT
    ->     A.Actor_Name,
    ->     COUNT(AE.Event_ID) AS Total_Attacks
    -> FROM ACTOR A
    ->
    -> JOIN ATTACK_EVENT AE
    ->     ON A.Actor_ID = AE.Actor_ID
    ->
    -> GROUP BY A.Actor_ID, A.Actor_Name
    ->
    -> HAVING COUNT(AE.Event_ID) > 1;
+----------------------+---------------+
| Actor_Name           | Total_Attacks |
+----------------------+---------------+
| APT29                |             4 |
| APT3                 |             2 |
| OilRig               |             2 |
| Wizard Spider        |             4 |
| Unknown External Actor |             2 |
+----------------------+---------------+
5 rows in set (0.00 sec)
```

Again all five actors, since each performed at least two events.

Incidents with more than two steps

```
SELECT Incident_ID, COUNT(*) AS Total_Steps
FROM ATTACK_EVENT
GROUP BY Incident_ID
HAVING COUNT(*) > 2;
```



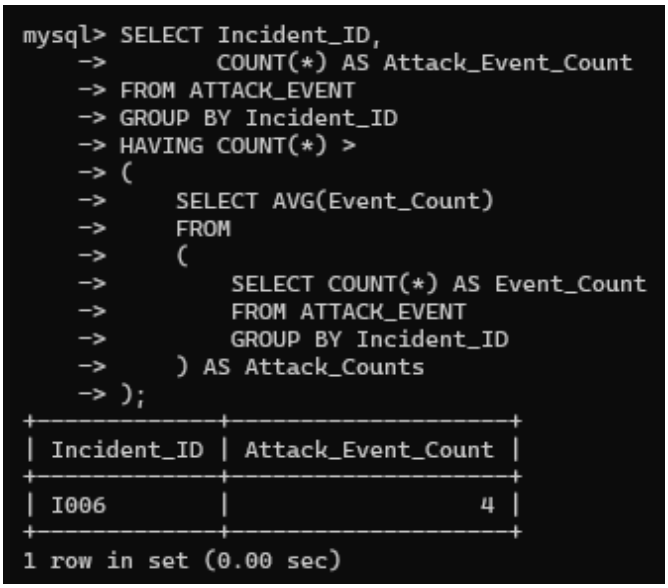
Only I006, with four steps. This is the longest attack path in the database.

1.9 Nested queries

A nested query is a SELECT inside another SELECT. The inner query runs first and the outer query uses its result.

Incidents with above-average attack events

```
SELECT Incident_ID, COUNT(*) AS Attack_Event_Count
FROM ATTACK_EVENT
GROUP BY Incident_ID
HAVING COUNT(*) > (
    SELECT AVG(Event_Count)
    FROM (
        SELECT COUNT(*) AS Event_Count
        FROM ATTACK_EVENT
        GROUP BY Incident_ID
    ) AS Attack_Counts
);
```



The average is 14 / 6 = 2.33 events per incident, so only I006 (4 events) is above it.

Attackers in critical incidents

```
SELECT Actor_Name
FROM ACTOR
WHERE Actor_ID IN (
    SELECT Actor_ID
    FROM ATTACK_EVENT
    WHERE Incident_ID IN (
        SELECT Incident_ID
        FROM SECURITY_INCIDENT
        WHERE Severity = 'Critical'
    )
);
```



Three levels of nesting, from critical incidents to their events to the actors: APT29, APT3 and Wizard Spider.

Incidents where PowerShell was used

```
SELECT Incident_ID, Security_Incident
FROM SECURITY_INCIDENT
WHERE Incident_ID IN (
    SELECT Incident_ID
    FROM ATTACK_EVENT
    WHERE Technique_ID = 'T1059.001'
);
```



T1059.001 is the MITRE technique for PowerShell. It shows up in I002 and I006.

Incidents involving T1078 (Valid Accounts)

```
SELECT SI.Incident_ID, SI.Security_Incident, SI.Severity
FROM SECURITY_INCIDENT SI
WHERE SI.Incident_ID IN (
    SELECT Incident_ID
    FROM ATTACK_EVENT
    WHERE Technique_ID = 'T1078'
);
```

```
mysql> SELECT
    ->     SI.Incident_ID,
    ->     SI.Security_Incident,
    ->     SI.Severity
    -> FROM SECURITY_INCIDENT SI
    -> WHERE SI.Incident_ID IN
    -> (
    ->     SELECT Incident_ID
    ->     FROM ATTACK_EVENT
    ->     WHERE Technique_ID = 'T1078'
    -> );
+-------------+-------------------+----------+
| Incident_ID | Security_Incident | Severity |
+-------------+-------------------+----------+
| I003        | Confirmed         | Critical |
| I004        | Confirmed         | High     |
+-------------+-------------------+----------+
2 rows in set (0.00 sec)
```

Stolen or valid accounts were used in I003 (Critical) and I004 (High).

Actor involved in incident I006

```
SELECT Actor_Name
FROM ACTOR
WHERE Actor_ID = (
    SELECT Actor_ID
    FROM ATTACK_EVENT
    WHERE Incident_ID = 'I006'
    LIMIT 1
);
```

```
mysql> SELECT Actor_Name
    -> FROM ACTOR
    -> WHERE Actor_ID =
    -> (
    ->     SELECT Actor_ID
    ->     FROM ATTACK_EVENT
    ->     WHERE Incident_ID = 'I006'
    ->     LIMIT 1
    -> );
+------------+
| Actor_Name |
+------------+
| APT29      |
+------------+
1 row in set (0.00 sec)
```

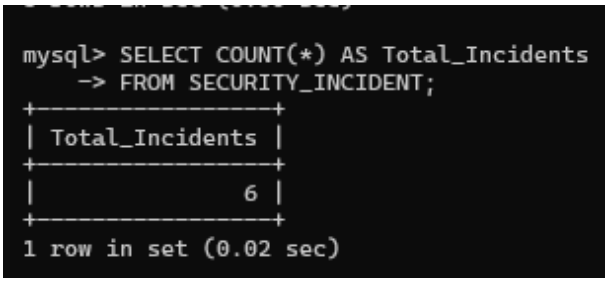
APT29. The inner query returns a single value, so = works here, and LIMIT 1 is safe only because I006 has just one actor.

1.10 Aggregate functions

Each of these returns one value computed from many rows.

COUNT: total incidents

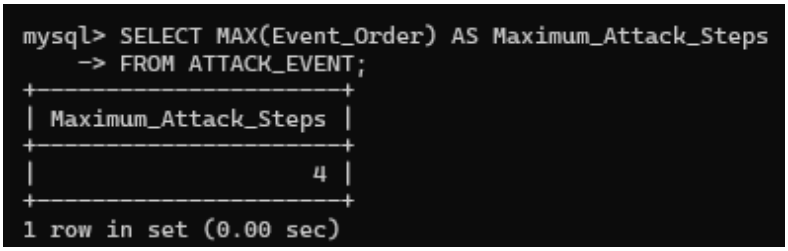
```
SELECT COUNT(*) AS Total_Incidents FROM SECURITY_INCIDENT;
```



There are 6 incidents.

MAX: highest Event_Order

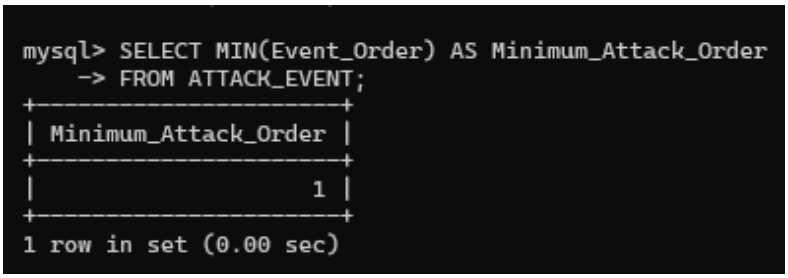
```
SELECT MAX(Event_Order) AS Maximum_Attack_Steps FROM ATTACK_EVENT;
```



4, the length of the longest path (I006).

MIN: lowest Event_Order

```
SELECT MIN(Event_Order) AS Minimum_Attack_Order FROM ATTACK_EVENT;
```



1, since every attack path starts at step 1.

AVG: average Event_Order

```
SELECT AVG(Event_Order) AS Average_Attack_Order FROM ATTACK_EVENT;
```

```
mysql> SELECT AVG(Event_Order) AS Average_Attack_Order
    -> FROM ATTACK_EVENT;
+----------------------+
| Average_Attack_Order |
+----------------------+
|               1.7857 |
+----------------------+
1 row in set (0.00 sec)
```

1.7857, which is 25 / 14. This is the average step number, not the average number of events per incident (2.33) used in the nested query above.

1.11 Function, procedure, trigger and view

Stored function: GetAttackStepCount

Takes an incident ID and returns how many attack steps that incident has.

```
DELIMITER //
CREATE FUNCTION GetAttackStepCount(p_incident_id VARCHAR(10))
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE step_count INT;
    SELECT COUNT(*)
    INTO step_count
    FROM ATTACK_EVENT
    WHERE Incident_ID = p_incident_id;
    RETURN step_count;
END //
DELIMITER ;

SELECT GetAttackStepCount('I006') AS Attack_Steps;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION GetAttackStepCount(
    ->     p_incident_id VARCHAR(10)
    -> )
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->
    ->     DECLARE step_count INT;
    ->
    ->     SELECT COUNT(*)
    ->     INTO step_count
    ->     FROM ATTACK_EVENT
    ->     WHERE Incident_ID = p_incident_id;
    ->
    ->     RETURN step_count;
    ->
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT GetAttackStepCount('I006') AS Attack_Steps;
+---------------+
| Attack_Steps |
+---------------+
|            4 |
+---------------+
1 row in set (0.01 sec)
```

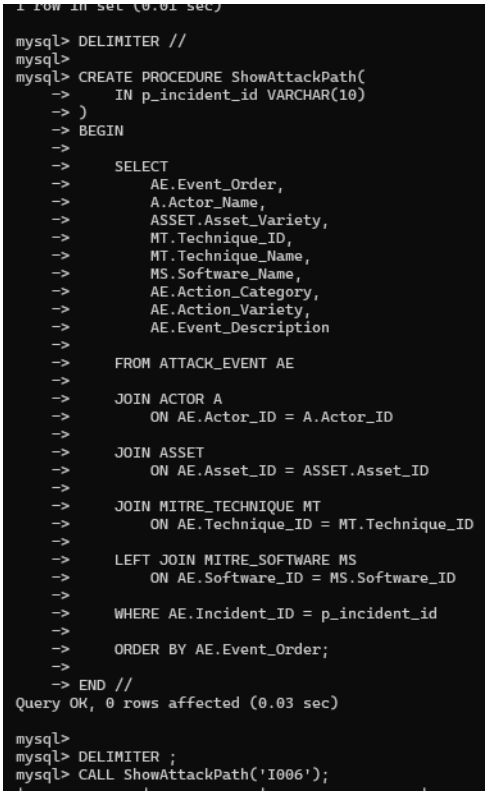
It returns 4 for I006, and the same function gives 2 for I001.

Stored procedure: ShowAttackPath

Takes an incident ID and displays its full attack path in order. It wraps the six-table join from section 1.7 into one reusable command.

```
DELIMITER //
CREATE PROCEDURE ShowAttackPath(IN p_incident_id VARCHAR(10))
BEGIN
    SELECT AE.Event_Order, A.Actor_Name, ASSET.Asset_Variety,
           MT.Technique_ID, MT.Technique_Name, MS.Software_Name,
           AE.Action_Category, AE.Action_Variety, AE.Event_Description
    FROM ATTACK_EVENT AE
    JOIN ACTOR A                ON AE.Actor_ID     = A.Actor_ID
    JOIN ASSET                  ON AE.Asset_ID     = ASSET.Asset_ID
    JOIN MITRE_TECHNIQUE MT     ON AE.Technique_ID = MT.Technique_ID
    LEFT JOIN MITRE_SOFTWARE MS ON AE.Software_ID  = MS.Software_ID
    WHERE AE.Incident_ID = p_incident_id
    ORDER BY AE.Event_Order;
END //
DELIMITER ;

CALL ShowAttackPath('I006');
```





The value in the CALL replaces p_incident_id, so the same procedure works for any incident. For I006 it lists the four steps in order. Unlike a function, a procedure can return a whole result set.

Trigger: CheckAttackOrder

Runs before every INSERT into ATTACK_EVENT and rejects an Event_Order of zero or less, because steps start at 1.

```
DELIMITER //
CREATE TRIGGER CheckAttackOrder
BEFORE INSERT ON ATTACK_EVENT
FOR EACH ROW
BEGIN
    IF NEW.Event_Order <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Event order must be greater than zero';
    END IF;
END //
DELIMITER ;

INSERT INTO ATTACK_EVENT
(Event_ID, Incident_ID, Actor_ID, Asset_ID, Technique_ID,
 Software_ID, Event_Order, Action_Category, Action_Variety, Event_Description)
VALUES
('E099', 'I001', 'A001', 'AS02', 'T1566',
 'S0367', 0, 'Initial Access', 'Phishing', 'Invalid attack order');
```



The insert fails with ERROR 1644 (45000): Event order must be greater than zero, and no row is added. The screenshot also shows ERROR 1305 from an accidental CALL CheckAttackOrder. That is expected, since a trigger fires by itself on an INSERT and cannot be called by name.

View: AttackPathView

Saves the six-table attack-path join as a virtual table, so nobody has to rewrite it every time.

```sql
CREATE VIEW AttackPathView AS
SELECT AE.Event_ID, AE.Incident_ID, SI.Security_Incident, SI.Severity,
       AE.Event_Order, A.Actor_Name, ASSET.Asset_Variety, MT.Technique_Name,
       MS.Software_Name, AE.Action_Category, AE.Action_Variety, AE.Event_Description
FROM ATTACK_EVENT AE
JOIN SECURITY_INCIDENT SI   ON AE.Incident_ID  = SI.Incident_ID
JOIN ACTOR A                ON AE.Actor_ID     = A.Actor_ID
JOIN ASSET                  ON AE.Asset_ID     = ASSET.Asset_ID
JOIN MITRE_TECHNIQUE MT     ON AE.Technique_ID = MT.Technique_ID
LEFT JOIN MITRE_SOFTWARE MS ON AE.Software_ID  = MS.Software_ID;
```

```
mysql> CREATE VIEW AttackPathView AS
    ->
    -> SELECT
    ->     AE.Event_ID,
    ->     AE.Incident_ID,
    ->     SI.Security_Incident,
    ->     SI.Severity,
    ->     AE.Event_Order,
    ->     A.Actor_Name,
    ->     ASSET.Asset_Variety,
    ->     MT.Technique_Name,
    ->     MS.Software_Name,
    ->     AE.Action_Category,
    ->     AE.Action_Variety,
    ->     AE.Event_Description
    ->
    -> FROM ATTACK_EVENT AE
    ->
    -> JOIN SECURITY_INCIDENT SI
    ->     ON AE.Incident_ID = SI.Incident_ID
    ->
    -> JOIN ACTOR A
    ->     ON AE.Actor_ID = A.Actor_ID
    ->
    -> JOIN ASSET
    ->     ON AE.Asset_ID = ASSET.Asset_ID
    ->
    -> JOIN MITRE_TECHNIQUE MT
    ->     ON AE.Technique_ID = MT.Technique_ID
    ->
    -> LEFT JOIN MITRE_SOFTWARE MS
    ->     ON AE.Software_ID = MS.Software_ID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT *
    -> FROM AttackPathView;
```

`SELECT * FROM AttackPathView;` returns the same 14 rows shown in section 1.5. The view can be filtered like a table:

```sql
SELECT *
FROM AttackPathView
WHERE Incident_ID = 'I006'
ORDER BY Event_Order;
```

```
mysql> SELECT *
    -> FROM AttackPathView
    -> WHERE Incident_ID = 'I006'
    -> ORDER BY Event_Order;
+----------+-------------+-----------------+----------+-------------+------------+----------------+-----------------------------------------------------------------------+---------------+----------------+-----------------------+-----------------------------------------------------------------------------------------+
| Event_ID | Incident_ID | Security_Incident | Severity | Event_Order | Actor_Name | Asset_Variety  | Technique_Name                                                        | Software_Name | Action_Category | Action_Variety        | Event_Description                                                                       |
+----------+-------------+-----------------+----------+-------------+------------+----------------+-----------------------------------------------------------------------+---------------+----------------+-----------------------+-----------------------------------------------------------------------------------------+
| E011     | I006        | Confirmed       | Critical |           1 | APT29      | Build Server   | Obfuscated Files or Information: Encrypted/Encoded File               | Raindrop      | Defense Evasion | Encrypted/Encoded File | Encoded or encrypted malware content was used during the intrusion                      |
| E012     | I006        | Confirmed       | Critical |           2 | APT29      | Build Server   | Command and Scripting Interpreter: PowerShell                         | NULL          | Execution       | PowerShell             | PowerShell activity was used during post-compromise operations                          |
| E013     | I006        | Confirmed       | Critical |           3 | APT29      | Database Server | Account Discovery                                                     | NULL          | Discovery       | Account Discovery      | User and account information was enumerated                                             |
| E014     | I006        | Confirmed       | Critical |           4 | APT29      | Network Server | System Network Configuration Discovery: Internet Connection Discovery | GoloFinder    | Discovery       | Internet Connection Discovery | Network connectivity and proxy information was examined                          |
+----------+-------------+-----------------+----------+-------------+------------+----------------+-----------------------------------------------------------------------+---------------+----------------+-----------------------+-----------------------------------------------------------------------------------------+
4 rows in set (0.00 sec)
```

Four rows (E011 to E014) in order 1 to 4. Because a view reads the base tables each time it is queried, new ATTACK_EVENT rows show up in it automatically.

1.12 My contribution and conclusion

I worked out the relationships and schema for incidents, actors and attack events, along with the functional dependencies and normal forms. I created SECURITY_INCIDENT, ACTOR and ATTACK_EVENT (including the Event_Order idea), wrote the queries in this part, and built the function, procedure, trigger and view.

Put together, the tables and queries answer the questions the module was built for: which incident happened, who was behind it, which asset was hit, which technique and software were used, and in what order. A six-table join rebuilds the path, a LEFT JOIN keeps steps that have no software, and GROUP BY, HAVING and nested

queries pick out patterns such as I006 having the longest path with four steps. The function, procedure, trigger and view then make the same work reusable and protect the data from bad input.

Member 2: Assets and Vulnerabilities

This part is about what got attacked: the kind of asset, who owns it, what state it is in, and how it connects to attack events and MITRE techniques.

2.1 What this part covers

The database has no separate VULNERABILITY table. Instead, a weakness shows up through the MITRE technique used in an attack event against an asset. For example, Phishing → Malicious Link → Employee Laptop, or Valid Accounts → Credential Access → Domain Controller.

My work answers three questions: what was attacked, what type of asset it was, and how the attack connects to it through attack events and techniques. In practice that meant:

- managing asset type, ownership, status and location
- connecting assets to ATTACK_EVENT records, and those records to MITRE techniques
- writing the joins, GROUP BY / HAVING, nested and aggregate queries, plus a function, procedure, trigger and view
- documenting the functional dependencies, normalization and ER relationships

2.2 The ASSET table

ASSET stores the systems and resources that incidents affected.

```
CREATE TABLE ASSET (
    Asset_ID        VARCHAR(10) PRIMARY KEY,
    Asset_Variety   VARCHAR(100),
    Asset_Ownership VARCHAR(50),
    Asset_Status    VARCHAR(50),
    Asset_Location  VARCHAR(100)
);
```

Asset_ID	Asset_Variety	Ownership	Status	Location
AS01	Server	Organization	Compromised	Data Center
AS02	Endpoint	Organization	Compromised	Finance Department
AS03	Network Device	Organization	Isolated	Main Office
AS04	Database Server	Organization	Compromised	Data Center
AS05	Employee Laptop	Organization	Compromised	HR Department
AS06	Web Server	Organization	Under Investigation	DMZ
AS07	Email Server	Organization	Compromised	Mail Server Room
AS08	Cloud Server	Organization	Active	Cloud Environment
AS09	Domain Controller	Organization	Compromised	IT Department
AS10	Workstation	Organization	Isolated	Operations Department

2.3 Joins, GROUP BY and nested queries

Asset, attack and MITRE technique

Joins ASSET, ATTACK_EVENT and MITRE_TECHNIQUE to show which asset was hit, with which technique, under what action category, and what happened.

```
SELECT A.Asset_ID, A.Asset_Variety, A.Asset_Status,
       T.Technique_ID, T.Technique_Name,
       AE.Action_Category, AE.Event_Description
FROM ASSET A
JOIN ATTACK_EVENT AE   ON A.Asset_ID = AE.Asset_ID
JOIN MITRE_TECHNIQUE T ON AE.Technique_ID = T.Technique_ID
ORDER BY A.Asset_ID;
```

```
mysql> SELECT
    ->     A.Asset_ID,
    ->     A.Asset_Variety,
    ->     A.Asset_Status,
    ->     T.Technique_ID,
    ->     T.Technique_Name,
    ->     AE.Action_Category,
    ->     AE.Event_Description
    -> FROM ASSET A
    -> JOIN ATTACK_EVENT AE
    ->     ON A.Asset_ID = AE.Asset_ID
    -> JOIN MITRE_TECHNIQUE T
    ->     ON AE.Technique_ID = T.Technique_ID
    -> ORDER BY A.Asset_ID;
+----------+------------------+-------------------+--------------+-----------------------------------+-----------------+-----------------------------------------------+
| Asset_ID | Asset_Variety    | Asset_Status      | Technique_ID | Technique_Name                    | Action_Category | Event_Description                             |
+----------+------------------+-------------------+--------------+-----------------------------------+-----------------+-----------------------------------------------+
| AS01     | Server           | Compromised       | T1486        | Data Encrypted for Impact         | Impact          | Ransomware encrypted files on the server      |
| AS03     | Network Device   | Isolated          | T1021.001    | Remote Services: RDP              | Lateral Movement| Attacker accessed a network device remotely   |
| AS03     | Network Device   | Isolated          | T1087        | Account Discovery                 | Discovery       | Attacker discovered user accounts             |
| AS04     | Database Server  | Compromised       | T1486        | Data Encrypted for Impact         | Impact          | Database files were encrypted                 |
| AS04     | Database Server  | Compromised       | T1047        | Windows Management Instrumentation| Execution       | Windows Management Instrumentation was used   |
| AS05     | Employee Laptop  | Compromised       | T1566        | Phishing                          | Initial Access  | Attacker sent a phishing email to an employee |
| AS05     | Employee Laptop  | Compromised       | T1566.002    | Spearphishing Link                | Initial Access  | Employee received a malicious phishing link   |
| AS05     | Employee Laptop  | Compromised       | T1204.001    | Malicious Link                    | Execution       | Employee clicked the malicious link           |
| AS06     | Web Server       | Under Investigation| T1016.001   | Internet Connection Discovery     | Discovery       | Attacker collected network information        |
| AS06     | Web Server       | Under Investigation| T1027.013   | Encrypted/Encoded File            | Defense Evasion | Data was encoded before transfer              |
| AS09     | Domain Controller| Compromised       | T1078        | Valid Accounts                    | Defense Evasion | Attacker used stolen credentials              |
| AS09     | Domain Controller| Compromised       | T1003        | OS Credential Dumping             | Credential Access| Credentials were dumped from the system      |
| AS10     | Workstation      | Isolated          | T1059.001    | PowerShell                        | Execution       | PowerShell was used to execute malicious commands |
| AS10     | Workstation      | Isolated          | T1053.005    | Scheduled Task/Job: Scheduled Task| Persistence     | A scheduled task was created for persistence  |
+----------+------------------+-------------------+--------------+-----------------------------------+-----------------+-----------------------------------------------+
14 rows in set (0.00 sec)
```

Number of attacks per asset

GROUP BY counts the attack events against each asset.

```
SELECT A.Asset_ID, A.Asset_Variety, COUNT(AE.Event_ID) AS Number_of_Attacks
FROM ASSET A
JOIN ATTACK_EVENT AE ON A.Asset_ID = AE.Asset_ID
GROUP BY A.Asset_ID, A.Asset_Variety;
```

```
mysql> SELECT
    ->     A.Asset_ID,
    ->     A.Asset_Variety,
    ->     COUNT(AE.Event_ID) AS Number_of_Attacks
    -> FROM ASSET A
    -> JOIN ATTACK_EVENT AE
    ->     ON A.Asset_ID = AE.Asset_ID
    -> GROUP BY
    ->     A.Asset_ID,
    ->     A.Asset_Variety;
+----------+------------------+-------------------+
| Asset_ID | Asset_Variety    | Number_of_Attacks |
+----------+------------------+-------------------+
| AS01     | Server           |                 1 |
| AS03     | Network Device   |                 2 |
| AS04     | Database Server  |                 2 |
| AS05     | Employee Laptop  |                 3 |
| AS06     | Web Server       |                 2 |
| AS09     | Domain Controller|                 2 |
| AS10     | Workstation      |                 2 |
+----------+------------------+-------------------+
7 rows in set (0.00 sec)
```

Assets attacked more than once

The same query with HAVING, so only assets with more than one attack are kept.

```sql
SELECT A.Asset_ID, A.Asset_Variety, COUNT(AE.Event_ID) AS Number_of_Attacks
FROM ASSET A
JOIN ATTACK_EVENT AE ON A.Asset_ID = AE.Asset_ID
GROUP BY A.Asset_ID, A.Asset_Variety
HAVING COUNT(AE.Event_ID) > 1;
```

```
mysql> SELECT
    ->     A.Asset_ID,
    ->     A.Asset_Variety,
    ->     COUNT(AE.Event_ID) AS Number_of_Attacks
    -> FROM ASSET A
    -> JOIN ATTACK_EVENT AE
    ->     ON A.Asset_ID = AE.Asset_ID
    -> GROUP BY
    ->     A.Asset_ID,
    ->     A.Asset_Variety
    -> HAVING COUNT(AE.Event_ID) > 1;
+----------+-------------------+-------------------+
| Asset_ID | Asset_Variety     | Number_of_Attacks |
+----------+-------------------+-------------------+
| AS03     | Network Device    |                 2 |
| AS04     | Database Server   |                 2 |
| AS05     | Employee Laptop   |                 3 |
| AS06     | Web Server        |                 2 |
| AS09     | Domain Controller |                 2 |
| AS10     | Workstation       |                 2 |
+----------+-------------------+-------------------+
6 rows in set (0.00 sec)
```

Assets in critical incidents

A three-level nested query: Critical incidents, then the attack events of those incidents, then the assets those events touched.

```sql
SELECT *
FROM ASSET
WHERE Asset_ID IN (
    SELECT Asset_ID
    FROM ATTACK_EVENT
    WHERE Incident_ID IN (
        SELECT Incident_ID
        FROM SECURITY_INCIDENT
        WHERE Severity = 'Critical'
    )
);
```

```
mysql> SELECT *
    -> FROM ASSET
    -> WHERE Asset_ID IN (
    ->     SELECT Asset_ID
    ->     FROM ATTACK_EVENT
    ->     WHERE Incident_ID IN (
    ->         SELECT Incident_ID
    ->         FROM SECURITY_INCIDENT
    ->         WHERE Severity = 'Critical'
    ->     )
    -> );
+----------+----------------+----------------+--------------------+----------------+
| Asset_ID | Asset_Variety  | Asset_Ownership | Asset_Status       | Asset_Location |
+----------+----------------+----------------+--------------------+----------------+
| AS01     | Server         | Organization   | Compromised        | Data Center    |
| AS04     | Database Server | Organization   | Compromised        | Data Center    |
| AS06     | Web Server     | Organization   | Under Investigation | DMZ            |
+----------+----------------+----------------+--------------------+----------------+
3 rows in set (0.01 sec)
```

2.4 Aggregate functions

Total number of assets

```
SELECT COUNT(*) AS Total_Assets FROM ASSET;
```

```
mysql> SELECT COUNT(*) AS Total_Assets
    -> FROM ASSET;
+--------------+
| Total_Assets |
+--------------+
|           10 |
+--------------+
1 row in set (0.01 sec)
```

Number of distinct asset types

```
SELECT COUNT(DISTINCT Asset_Variety) AS Asset_Types FROM ASSET;
```

```
mysql> SELECT COUNT(DISTINCT Asset_Variety) AS Asset_Types
    -> FROM ASSET;
+-------------+
| Asset_Types |
+-------------+
|          10 |
+-------------+
1 row in set (0.00 sec)
```

Count by asset type

```
SELECT Asset_Variety, COUNT(*) AS Total
FROM ASSET
GROUP BY Asset_Variety
ORDER BY Total DESC;
```

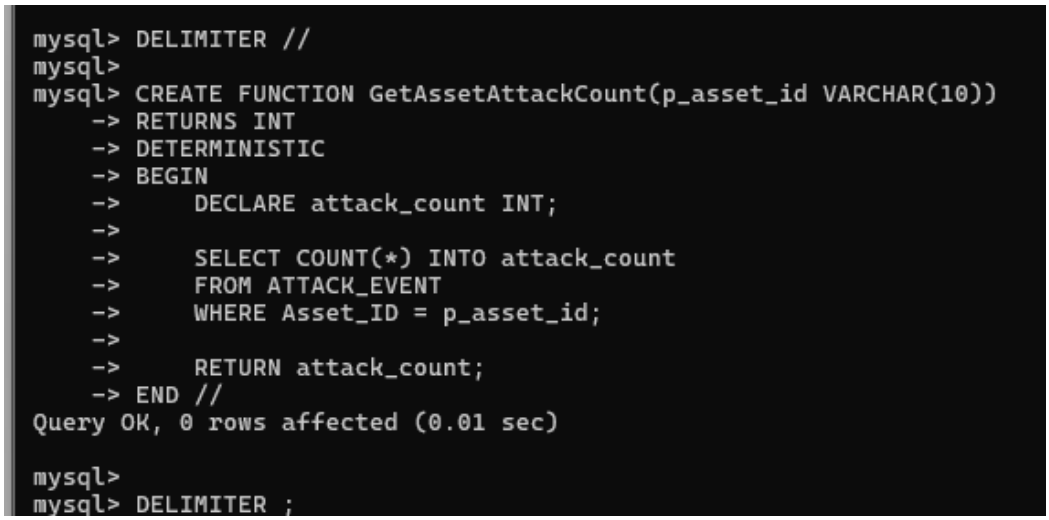
```
mysql> SELECT Asset_Variety, COUNT(*) AS Total
    -> FROM ASSET
    -> GROUP BY Asset_Variety
    -> ORDER BY Total DESC;
+-------------------+-------+
| Asset_Variety     | Total |
+-------------------+-------+
| Server            |     1 |
| Endpoint          |     1 |
| Network Device    |     1 |
| Database Server   |     1 |
| Employee Laptop   |     1 |
| Web Server        |     1 |
| Email Server      |     1 |
| Cloud Server      |     1 |
| Domain Controller |     1 |
| Workstation       |     1 |
+-------------------+-------+
10 rows in set (0.00 sec)
```

2.5 Function, procedure, trigger and view

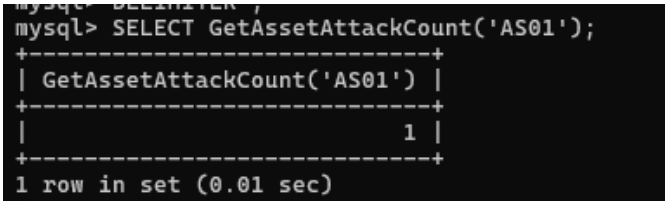
Stored function: GetAssetAttackCount

Returns how many attack events involve a given asset.

```
DELIMITER //
CREATE FUNCTION GetAssetAttackCount(p_asset_id VARCHAR(10))
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE attack_count INT;
    SELECT COUNT(*) INTO attack_count
    FROM ATTACK_EVENT
    WHERE Asset_ID = p_asset_id;
    RETURN attack_count;
END //
DELIMITER ;
```




```
SELECT GetAssetAttackCount('AS01');
```



Stored procedure: ShowAssetAttacks

Displays every attack on a chosen asset, with the MITRE technique and the event order.

```
DELIMITER //
CREATE PROCEDURE ShowAssetAttacks(IN p_asset_id VARCHAR(10))
BEGIN
    SELECT A.Asset_ID, A.Asset_Variety, T.Technique_ID, T.Technique_Name,
           AE.Event_Order, AE.Action_Category, AE.Event_Description
    FROM ASSET A
    JOIN ATTACK_EVENT AE   ON A.Asset_ID = AE.Asset_ID
    JOIN MITRE_TECHNIQUE T ON AE.Technique_ID = T.Technique_ID
    WHERE A.Asset_ID = p_asset_id
    ORDER BY AE.Event_Order;
END //
DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE ShowAssetAttacks(IN p_asset_id VARCHAR(10))
    -> BEGIN
    ->     SELECT
    ->         A.Asset_ID,
    ->         A.Asset_Variety,
    ->         T.Technique_ID,
    ->         T.Technique_Name,
    ->         AE.Event_Order,
    ->         AE.Action_Category,
    ->         AE.Event_Description
    ->     FROM ASSET A
    ->     JOIN ATTACK_EVENT AE
    ->         ON A.Asset_ID = AE.Asset_ID
    ->     JOIN MITRE_TECHNIQUE T
    ->         ON AE.Technique_ID = T.Technique_ID
    ->     WHERE A.Asset_ID = p_asset_id
    ->     ORDER BY AE.Event_Order;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
```

```
CALL ShowAssetAttacks('AS01');
```

```
mysql> CALL ShowAssetAttacks('AS01');
+----------+--------------+--------------+--------------------------+-------------+-----------------+-----------------------------------------+
| Asset_ID | Asset_Variety | Technique_ID | Technique_Name           | Event_Order | Action_Category | Event_Description                       |
+----------+--------------+--------------+--------------------------+-------------+-----------------+-----------------------------------------+
| AS01     | Server        | T1486        | Data Encrypted for Impact |           1 | Impact          | Ransomware encrypted files on the server |
+----------+--------------+--------------+--------------------------+-------------+-----------------+-----------------------------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql>
```

Trigger: CheckAssetStatus

Checks Asset_Status before a new asset is inserted. Only Active, Compromised, Isolated and Under Investigation are accepted. The test inserts an asset with the status UnknownStatus, which should be rejected with ERROR 1644 (Invalid asset status).

```
DELIMITER //
CREATE TRIGGER CheckAssetStatus
BEFORE INSERT ON ASSET
FOR EACH ROW
BEGIN
    IF NEW.Asset_Status NOT IN
       ('Active','Compromised','Isolated','Under Investigation') THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid asset status';
    END IF;
END //
DELIMITER ;

INSERT INTO ASSET
VALUES ('AS11', 'Test Server', 'Organization', 'UnknownStatus', 'Test Location');
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER CheckAssetStatus
    -> BEFORE INSERT ON ASSET
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.Asset_Status NOT IN
    ->        ('Active','Compromised','Isolated','Under Investigation') THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Invalid asset status';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO ASSET
    -> VALUES ('AS11', 'Test Server', 'Organization', 'UnknownStatus', 'Test Location');
ERROR 1644 (45000): Invalid asset status
mysql>
```

View: AssetAttackView

A reusable combined view of assets, their status, the MITRE technique, event order and action category.

```
CREATE VIEW AssetAttackView AS
SELECT A.Asset_ID, A.Asset_Variety, A.Asset_Status,
       T.Technique_ID, T.Technique_Name,
       AE.Event_Order, AE.Action_Category
FROM ASSET A
JOIN ATTACK_EVENT AE   ON A.Asset_ID = AE.Asset_ID
JOIN MITRE_TECHNIQUE T ON AE.Technique_ID = T.Technique_ID;

SELECT * FROM AssetAttackView;
```

In the MySQL session CREATE VIEW gave ERROR 1050 because AssetAttackView already existed, so the existing view was queried directly. It is not a problem with the view itself.

```
mysql> CREATE VIEW AssetAttackView AS
    -> SELECT
    ->     A.Asset_ID,
    ->     A.Asset_Variety,
    ->     A.Asset_Status,
    ->     T.Technique_ID,
    ->     T.Technique_Name,
    ->     AE.Event_Order,
    ->     AE.Action_Category
    -> FROM ASSET A
    -> JOIN ATTACK_EVENT AE
    ->     ON A.Asset_ID = AE.Asset_ID
    -> JOIN MITRE_TECHNIQUE T
    ->     ON AE.Technique_ID = T.Technique_ID;
ERROR 1050 (42S01): Table 'AssetAttackView' already exists
mysql> SELECT * FROM AssetAttackView;
+----------+------------------+--------------------+-------------+----------------------------------+-------------+-------------------+
| Asset_ID | Asset_Variety    | Asset_Status       | Technique_ID | Technique_Name                   | Event_Order | Action_Category   |
+----------+------------------+--------------------+-------------+----------------------------------+-------------+-------------------+
| AS01     | Server           | Compromised        | T1486       | Data Encrypted for Impact        |           1 | Impact            |
| AS03     | Network Device   | Isolated           | T1021.001   | Remote Services: RDP             |           1 | Lateral Movement  |
| AS03     | Network Device   | Isolated           | T1087       | Account Discovery                |           2 | Discovery         |
| AS04     | Database Server  | Compromised        | T1486       | Data Encrypted for Impact        |           2 | Impact            |
| AS04     | Database Server  | Compromised        | T1047       | Windows Management Instrumentation |         3 | Execution         |
| AS05     | Employee Laptop  | Compromised        | T1566       | Phishing                         |           1 | Initial Access    |
| AS05     | Employee Laptop  | Compromised        | T1566.002   | Spearphishing Link               |           2 | Initial Access    |
| AS05     | Employee Laptop  | Compromised        | T1204.001   | Malicious Link                   |           3 | Execution         |
| AS06     | Web Server       | Under Investigation | T1016.001  | Internet Connection Discovery    |           1 | Discovery         |
| AS06     | Web Server       | Under Investigation | T1027.013  | Encrypted/Encoded File           |           2 | Defense Evasion   |
| AS09     | Domain Controller | Compromised       | T1078       | Valid Accounts                   |           1 | Defense Evasion   |
| AS09     | Domain Controller | Compromised       | T1003       | OS Credential Dumping            |           2 | Credential Access |
| AS10     | Workstation      | Isolated           | T1059.001   | PowerShell                       |           1 | Execution         |
| AS10     | Workstation      | Isolated           | T1053.005   | Scheduled Task/Job: Scheduled Task |         2 | Persistence       |
+----------+------------------+--------------------+-------------+----------------------------------+-------------+-------------------+
14 rows in set (0.00 sec)
```

2.6 Dependencies, normalization and relationships

Each primary key decides the other columns of its table:

Table	Functional dependency
ASSET	Asset_ID → Asset_Variety, Asset_Ownership, Asset_Status, Asset_Location
MITRE_TECHNIQUE	Technique_ID → Technique_Name, Tactic, Platform
ATTACK_EVENT	Event_ID → Incident_ID, Actor_ID, Asset_ID, Technique_ID, Software_ID, Event_Order, Action_Category, Action_Variety, Event_Description

Normal form	How the tables meet it
1NF	Every attribute holds a single value and there are no repeating groups.
2NF	The tables are in 1NF and every non-key attribute depends on the whole primary key.
3NF	The tables are in 2NF and no non-key attribute depends on another non-key attribute.

The relationships follow the foreign keys actually implemented in MySQL. ATTACK_EVENT connects SECURITY_INCIDENT, ASSET and MITRE_TECHNIQUE:

Relationship	Cardinality	Meaning
SECURITY_INCIDENT → ATTACK_EVENT	1 : M	One incident can contain many attack events.
ASSET → ATTACK_EVENT	1 : M	One asset can appear in many events; each event refers to one asset.
MITRE_TECHNIQUE → ATTACK_EVENT	1 : M	One technique can be used in many events; each event uses one technique.

2.7 Database setup screenshots

These screenshots show how the database and its tables were created and filled in MySQL.

```
mysql> CREATE DATABASE SecurityAttackPathDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE SecurityAttackPathDB;
Database changed
mysql>
```

Database creation and USE command

```
mysql> CREATE TABLE ASSET (
    ->     Asset_ID VARCHAR(10) PRIMARY KEY,
    ->     Asset_Variety VARCHAR(100),
    ->     Asset_Ownership VARCHAR(50),
    ->     Asset_Status VARCHAR(50),
    ->     Asset_Location VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.11 sec)

mysql> ^C
mysql> SHOW TABLES;
+------------------------------+
| Tables_in_security_attack_db |
+------------------------------+
| asset                        |
+------------------------------+
1 row in set (0.01 sec)
```

ASSET table creation and SHOW TABLES

```
mysql> CREATE TABLE ASSET (
    ->     Asset_ID VARCHAR(10) PRIMARY KEY,
    ->     Asset_Variety VARCHAR(100),
    ->     Asset_Ownership VARCHAR(50),
    ->     Asset_Status VARCHAR(50),
    ->     Asset_Location VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO ASSET
    -> (Asset_ID, Asset_Variety, Asset_Ownership, Asset_Status, Asset_Location)
    -> VALUES
    -> ('AS01', 'Server', 'Organization', 'Compromised', 'Data Center'),
    -> ('AS02', 'Endpoint', 'Organization', 'Compromised', 'Finance Department'),
    -> ('AS03', 'Network Device', 'Organization', 'Isolated', 'Main Office'),
    -> ('AS04', 'Database Server', 'Organization', 'Compromised', 'Data Center'),
    -> ('AS05', 'Employee Laptop', 'Organization', 'Compromised', 'HR Department'),
    -> ('AS06', 'Web Server', 'Organization', 'Under Investigation', 'DMZ'),
    -> ('AS07', 'Email Server', 'Organization', 'Compromised', 'Mail Server Room'),
    -> ('AS08', 'Cloud Server', 'Organization', 'Active', 'Cloud Environment'),
    -> ('AS09', 'Domain Controller', 'Organization', 'Compromised', 'IT Department'),
    -> ('AS10', 'Workstation', 'Organization', 'Isolated', 'Operations Department');
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM ASSET;
+----------+------------------+----------------+--------------------+-----------------------+
| Asset_ID | Asset_Variety    | Asset_Ownership | Asset_Status       | Asset_Location        |
+----------+------------------+----------------+--------------------+-----------------------+
| AS01     | Server           | Organization   | Compromised        | Data Center           |
| AS02     | Endpoint         | Organization   | Compromised        | Finance Department    |
| AS03     | Network Device   | Organization   | Isolated           | Main Office           |
| AS04     | Database Server  | Organization   | Compromised        | Data Center           |
| AS05     | Employee Laptop  | Organization   | Compromised        | HR Department         |
| AS06     | Web Server       | Organization   | Under Investigation | DMZ                   |
| AS07     | Email Server     | Organization   | Compromised        | Mail Server Room      |
| AS08     | Cloud Server     | Organization   | Active             | Cloud Environment     |
| AS09     | Domain Controller | Organization   | Compromised        | IT Department         |
| AS10     | Workstation      | Organization   | Isolated           | Operations Department |
+----------+------------------+----------------+--------------------+-----------------------+
10 rows in set (0.00 sec)
```

ASSET table insertion and SELECT output (10 records)

```
mysql> CREATE TABLE MITRE_GROUP (
    ->     Group_ID VARCHAR(10) PRIMARY KEY,
    ->     Group_Name VARCHAR(100) NOT NULL,
    ->     Associated_Groups VARCHAR(300),
    ->     Description VARCHAR(500)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO MITRE_GROUP
    -> (Group_ID, Group_Name, Associated_Groups, Description)
    -> VALUES
    -> ('G0006', 'APT1', 'Comment Crew, Comment Group',
    ->  'Chinese cyber espionage group'),
    ->
    -> ('G0016', 'APT29', 'Cozy Bear, The Dukes',
    ->  'Cyber espionage group'),
    ->
    -> ('G0022', 'APT3', 'Gothic Panda, UPS Team',
    ->  'Threat group associated with China'),
    ->
    -> ('G0049', 'OilRig', 'Twisted Kitten, Crambus',
    ->  'Threat group targeting organizations'),
    ->
    -> ('G0102', 'Wizard Spider', 'Grim Spider',
    ->  'Cybercrime threat group');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM MITRE_GROUP;
+----------+--------------+---------------------------+-----------------------------------+
| Group_ID | Group_Name   | Associated_Groups         | Description                       |
+----------+--------------+---------------------------+-----------------------------------+
| G0006    | APT1         | Comment Crew, Comment Group | Chinese cyber espionage group     |
| G0016    | APT29        | Cozy Bear, The Dukes      | Cyber espionage group             |
| G0022    | APT3         | Gothic Panda, UPS Team    | Threat group associated with China |
| G0049    | OilRig       | Twisted Kitten, Crambus   | Threat group targeting organizations |
| G0102    | Wizard Spider | Grim Spider              | Cybercrime threat group           |
+----------+--------------+---------------------------+-----------------------------------+
5 rows in set (0.00 sec)

mysql>
```

MITRE_GROUP creation and data

```
mysql> CREATE TABLE MITRE_TECHNIQUE (
    ->     Technique_ID VARCHAR(15) PRIMARY KEY,
    ->     Technique_Name VARCHAR(150) NOT NULL,
    ->     Tactic VARCHAR(100),
    ->     Platform VARCHAR(200)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO MITRE_TECHNIQUE
    -> (Technique_ID, Technique_Name, Tactic, Platform)
    -> VALUES
    -> ('T1566', 'Phishing', 'Initial Access', 'Windows, Linux, macOS'),
    -> ('T1566.002', 'Spearphishing Link', 'Initial Access', 'Windows, Linux, macOS'),
    -> ('T1204.001', 'Malicious Link', 'Execution', 'Windows, Linux, macOS'),
    -> ('T1078', 'Valid Accounts', 'Defense Evasion', 'Windows, Linux, macOS'),
    -> ('T1059.001', 'PowerShell', 'Execution', 'Windows'),
    -> ('T1003', 'OS Credential Dumping', 'Credential Access', 'Windows'),
    -> ('T1021.001', 'Remote Services: RDP', 'Lateral Movement', 'Windows'),
    -> ('T1087', 'Account Discovery', 'Discovery', 'Windows, Linux, macOS'),
    -> ('T1016.001', 'Internet Connection Discovery', 'Discovery', 'Windows, Linux, macOS'),
    -> ('T1027.013', 'Encrypted/Encoded File', 'Defense Evasion', 'Windows, Linux, macOS'),
    -> ('T1486', 'Data Encrypted for Impact', 'Impact', 'Windows, Linux, macOS'),
    -> ('T1053.005', 'Scheduled Task/Job: Scheduled Task', 'Persistence', 'Windows'),
    -> ('T1047', 'Windows Management Instrumentation', 'Execution', 'Windows');
Query OK, 13 rows affected (0.01 sec)
Records: 13  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM MITRE_TECHNIQUE;
+--------------+-----------------------------------+------------------+-----------------------+
| Technique_ID | Technique_Name                    | Tactic           | Platform              |
+--------------+-----------------------------------+------------------+-----------------------+
| T1003        | OS Credential Dumping             | Credential Access | Windows               |
| T1016.001    | Internet Connection Discovery     | Discovery        | Windows, Linux, macOS |
| T1021.001    | Remote Services: RDP              | Lateral Movement | Windows               |
| T1027.013    | Encrypted/Encoded File            | Defense Evasion  | Windows, Linux, macOS |
| T1047        | Windows Management Instrumentation | Execution       | Windows               |
| T1053.005    | Scheduled Task/Job: Scheduled Task | Persistence     | Windows               |
| T1059.001    | PowerShell                        | Execution        | Windows               |
| T1078        | Valid Accounts                    | Defense Evasion  | Windows, Linux, macOS |
| T1087        | Account Discovery                 | Discovery        | Windows, Linux, macOS |
| T1204.001    | Malicious Link                    | Execution        | Windows, Linux, macOS |
| T1486        | Data Encrypted for Impact         | Impact           | Windows, Linux, macOS |
| T1566        | Phishing                          | Initial Access   | Windows, Linux, macOS |
| T1566.002    | Spearphishing Link                | Initial Access   | Windows, Linux, macOS |
+--------------+-----------------------------------+------------------+-----------------------+
13 rows in set (0.00 sec)
```

MITRE_TECHNIQUE creation and data

```
mysql> CREATE TABLE MITRE_SOFTWARE (
    ->     Software_ID VARCHAR(15) PRIMARY KEY,
    ->     Software_Name VARCHAR(150) NOT NULL,
    ->     Software_Type VARCHAR(100),
    ->     Platform VARCHAR(200)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO MITRE_SOFTWARE
    -> (Software_ID, Software_Name, Software_Type, Platform)
    -> VALUES
    -> ('S0012', 'PoisonIvy', 'Remote Access Trojan', 'Windows'),
    -> ('S0034', 'NETEAGLE', 'Remote Access Trojan', 'Windows'),
    -> ('S0154', 'Cobalt Strike', 'Penetration Testing Tool', 'Windows, Linux'),
    -> ('S0260', 'InvisiMole', 'Remote Access Trojan', 'Windows'),
    -> ('S0363', 'Empire', 'Post-Exploitation Framework', 'Windows, Linux, macOS'),
    -> ('S0373', 'Impacket', 'Network Tool', 'Windows, Linux'),
    -> ('S0447', 'Pupy', 'Remote Access Trojan', 'Windows, Linux, macOS');
Query OK, 7 rows affected (0.01 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM MITRE_SOFTWARE;
+-------------+---------------+-----------------------------+-----------------------+
| Software_ID | Software_Name | Software_Type               | Platform              |
+-------------+---------------+-----------------------------+-----------------------+
| S0012       | PoisonIvy     | Remote Access Trojan        | Windows               |
| S0034       | NETEAGLE      | Remote Access Trojan        | Windows               |
| S0154       | Cobalt Strike | Penetration Testing Tool    | Windows, Linux        |
| S0260       | InvisiMole    | Remote Access Trojan        | Windows               |
| S0363       | Empire        | Post-Exploitation Framework | Windows, Linux, macOS |
| S0373       | Impacket      | Network Tool                | Windows, Linux        |
| S0447       | Pupy          | Remote Access Trojan        | Windows, Linux, macOS |
+-------------+---------------+-----------------------------+-----------------------+
7 rows in set (0.00 sec)
```

MITRE_SOFTWARE creation and data

```
mysql> CREATE TABLE SECURITY_INCIDENT (
    ->     Incident_ID VARCHAR(10) PRIMARY KEY,
    ->     Incident_Type VARCHAR(100),
    ->     Severity VARCHAR(20),
    ->     Incident_Date DATE,
    ->     Incident_Status VARCHAR(50),
    ->     Description VARCHAR(500)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO SECURITY_INCIDENT
    -> (Incident_ID, Incident_Type, Severity, Incident_Date, Incident_Status, Description)
    -> VALUES
    -> ('INC001', 'Phishing Attack', 'High', '2026-01-15', 'Closed', 'Phishing email compromised an employee account'),
    -> ('INC002', 'Ransomware Attack', 'Critical', '2026-02-10', 'Under Investigation', 'Ransomware encrypted important organizational files'),
    -> ('INC003', 'Credential Theft', 'High', '2026-03-05', 'Closed', 'User credentials were stolen through a malicious website'),
    -> ('INC004', 'Unauthorized Access', 'Medium', '2026-04-12', 'Resolved', 'Unauthorized login detected on an internal system'),
    -> ('INC005', 'Data Exfiltration', 'Critical', '2026-05-20', 'Under Investigation', 'Sensitive organizational data was transferred outside the network'),
    -> ('INC006', 'Malware Infection', 'High', '2026-06-18', 'Resolved', 'Malware detected on an organizational workstation');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM SECURITY_INCIDENT;
+-------------+--------------------+----------+---------------+--------------------+-----------------------------------------------------------------+
| Incident_ID | Incident_Type      | Severity | Incident_Date | Incident_Status    | Description                                                     |
+-------------+--------------------+----------+---------------+--------------------+-----------------------------------------------------------------+
| INC001      | Phishing Attack    | High     | 2026-01-15    | Closed             | Phishing email compromised an employee account                  |
| INC002      | Ransomware Attack  | Critical | 2026-02-10    | Under Investigation | Ransomware encrypted important organizational files             |
| INC003      | Credential Theft   | High     | 2026-03-05    | Closed             | User credentials were stolen through a malicious website        |
| INC004      | Unauthorized Access | Medium   | 2026-04-12    | Resolved           | Unauthorized login detected on an internal system               |
| INC005      | Data Exfiltration  | Critical | 2026-05-20    | Under Investigation | Sensitive organizational data was transferred outside the network |
| INC006      | Malware Infection  | High     | 2026-06-18    | Resolved           | Malware detected on an organizational workstation               |
+-------------+--------------------+----------+---------------+--------------------+-----------------------------------------------------------------+
6 rows in set (0.00 sec)
```

SECURITY_INCIDENT creation and data

```
mysql> CREATE TABLE ACTOR (
    ->     Actor_ID VARCHAR(10) PRIMARY KEY,
    ->     Actor_Name VARCHAR(100) NOT NULL,
    ->     Actor_Type VARCHAR(100),
    ->     Group_ID VARCHAR(10),
    ->     Description VARCHAR(500),
    ->     FOREIGN KEY (Group_ID) REFERENCES MITRE_GROUP(Group_ID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO ACTOR
    -> (Actor_ID, Actor_Name, Actor_Type, Group_ID, Description)
    -> VALUES
    -> ('AC001', 'APT1', 'Nation State', 'G0006', 'Threat actor associated with targeted cyber espionage'),
    -> ('AC002', 'APT29', 'Nation State', 'G0016', 'Threat actor known for credential theft and phishing'),
    -> ('AC003', 'APT3', 'Nation State', 'G0022', 'Threat actor involved in network intrusion activities'),
    -> ('AC004', 'OilRig', 'Nation State', 'G0049', 'Threat group targeting organizations through phishing'),
    -> ('AC005', 'Wizard Spider', 'Cybercriminal Group', 'G0102', 'Cybercriminal group associated with ransomware'),
    -> ('AC006', 'Unknown Attacker', 'Unknown', NULL, 'Unidentified attacker involved in the incident');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM ACTOR;
+----------+------------------+---------------------+----------+-----------------------------------------------------+
| Actor_ID | Actor_Name       | Actor_Type          | Group_ID | Description                                         |
+----------+------------------+---------------------+----------+-----------------------------------------------------+
| AC001    | APT1             | Nation State        | G0006    | Threat actor associated with targeted cyber espionage |
| AC002    | APT29            | Nation State        | G0016    | Threat actor known for credential theft and phishing  |
| AC003    | APT3             | Nation State        | G0022    | Threat actor involved in network intrusion activities |
| AC004    | OilRig           | Nation State        | G0049    | Threat group targeting organizations through phishing |
| AC005    | Wizard Spider    | Cybercriminal Group | G0102    | Cybercriminal group associated with ransomware      |
| AC006    | Unknown Attacker | Unknown             | NULL     | Unidentified attacker involved in the incident      |
+----------+------------------+---------------------+----------+-----------------------------------------------------+
6 rows in set (0.00 sec)

mysql>
```

ACTOR creation and data

```
mysql> CREATE TABLE ATTACK_EVENT (
    ->     Event_ID VARCHAR(10) PRIMARY KEY,
    ->     Incident_ID VARCHAR(10),
    ->     Actor_ID VARCHAR(10),
    ->     Asset_ID VARCHAR(10),
    ->     Technique_ID VARCHAR(15),
    ->     Software_ID VARCHAR(15),
    ->     Event_Order INT,
    ->     Action_Category VARCHAR(100),
    ->     Action_Variety VARCHAR(100),
    ->     Event_Description VARCHAR(500),
    ->     FOREIGN KEY (Incident_ID) REFERENCES SECURITY_INCIDENT(Incident_ID),
    ->     FOREIGN KEY (Actor_ID) REFERENCES ACTOR(Actor_ID),
    ->     FOREIGN KEY (Asset_ID) REFERENCES ASSET(Asset_ID),
    ->     FOREIGN KEY (Technique_ID) REFERENCES MITRE_TECHNIQUE(Technique_ID),
    ->     FOREIGN KEY (Software_ID) REFERENCES MITRE_SOFTWARE(Software_ID)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

ATTACK_EVENT table creation

```
mysql> INSERT INTO ATTACK_EVENT
    -> (Event_ID, Incident_ID, Actor_ID, Asset_ID, Technique_ID, Software_ID,
    ->  Event_Order, Action_Category, Action_Variety, Event_Description)
    -> VALUES
    -> ('EV001', 'INC001', 'AC002', 'AS05', 'T1566', 'S0363',
    ->  1, 'Initial Access', 'Phishing', 'Attacker sent a phishing email to an employee'),
    ->
    -> ('EV002', 'INC001', 'AC002', 'AS05', 'T1566.002', 'S0363',
    ->  2, 'Initial Access', 'Spearphishing Link', 'Employee received a malicious phishing link'),
    ->
    -> ('EV003', 'INC001', 'AC002', 'AS05', 'T1204.001', 'S0363',
    ->  3, 'Execution', 'Malicious Link', 'Employee clicked the malicious link'),
    ->
    -> ('EV004', 'INC002', 'AC005', 'AS01', 'T1486', 'S0154',
    ->  1, 'Impact', 'Data Encryption', 'Ransomware encrypted files on the server'),
    ->
    -> ('EV005', 'INC002', 'AC005', 'AS04', 'T1486', 'S0154',
    ->  2, 'Impact', 'Data Encryption', 'Database files were encrypted'),
    ->
    -> ('EV006', 'INC003', 'AC004', 'AS09', 'T1078', 'S0363',
    ->  1, 'Defense Evasion', 'Valid Accounts', 'Attacker used stolen credentials'),
    ->
    -> ('EV007', 'INC003', 'AC004', 'AS09', 'T1003', 'S0363',
    ->  2, 'Credential Access', 'Credential Dumping', 'Credentials were dumped from the system'),
    ->
    -> ('EV008', 'INC004', 'AC001', 'AS03', 'T1021.001', 'S0373',
    ->  1, 'Lateral Movement', 'Remote Services', 'Attacker accessed a network device remotely'),
    ->
    -> ('EV009', 'INC004', 'AC001', 'AS03', 'T1087', 'S0373',
    ->  2, 'Discovery', 'Account Discovery', 'Attacker discovered user accounts'),
    ->
    -> ('EV010', 'INC005', 'AC003', 'AS06', 'T1016.001', 'S0373',
    ->  1, 'Discovery', 'Network Discovery', 'Attacker collected network information'),
    ->
    -> ('EV011', 'INC005', 'AC003', 'AS06', 'T1027.013', 'S0447',
    ->  2, 'Defense Evasion', 'Encoded Files', 'Data was encoded before transfer'),
    ->
    -> ('EV012', 'INC005', 'AC003', 'AS04', 'T1047', 'S0373',
    ->  3, 'Execution', 'WMI', 'Windows Management Instrumentation was used'),
    ->
    -> ('EV013', 'INC006', 'AC006', 'AS10', 'T1059.001', 'S0363',
    ->  1, 'Execution', 'PowerShell', 'PowerShell was used to execute malicious commands'),
    ->
    -> ('EV014', 'INC006', 'AC006', 'AS10', 'T1053.005', 'S0363',
    ->  2, 'Persistence', 'Scheduled Task', 'A scheduled task was created for persistence');
Query OK, 14 rows affected (0.01 sec)
Records: 14  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM ATTACK_EVENT;
+----------+-------------+----------+----------+--------------+-------------+-------------+-----------------+-----------------+-----------------------------------------------+
| Event_ID | Incident_ID | Actor_ID | Asset_ID | Technique_ID | Software_ID | Event_Order | Action_Category | Action_Variety  | Event_Description                             |
+----------+-------------+----------+----------+--------------+-------------+-------------+-----------------+-----------------+-----------------------------------------------+
| EV001    | INC001      | AC002    | AS05     | T1566        | S0363       |           1 | Initial Access  | Phishing        | Attacker sent a phishing email to an employee |
| EV002    | INC001      | AC002    | AS05     | T1566.002    | S0363       |           2 | Initial Access  | Spearphishing Link | Employee received a malicious phishing link |
| EV003    | INC001      | AC002    | AS05     | T1204.001    | S0363       |           3 | Execution       | Malicious Link  | Employee clicked the malicious link           |
| EV004    | INC002      | AC005    | AS01     | T1486        | S0154       |           1 | Impact          | Data Encryption | Ransomware encrypted files on the server      |
| EV005    | INC002      | AC005    | AS04     | T1486        | S0154       |           2 | Impact          | Data Encryption | Database files were encrypted                 |
| EV006    | INC003      | AC004    | AS09     | T1078        | S0363       |           1 | Defense Evasion | Valid Accounts  | Attacker used stolen credentials              |
| EV007    | INC003      | AC004    | AS09     | T1003        | S0363       |           2 | Credential Access | Credential Dumping | Credentials were dumped from the system |
| EV008    | INC004      | AC001    | AS03     | T1021.001    | S0373       |           1 | Lateral Movement | Remote Services | Attacker accessed a network device remotely   |
| EV009    | INC004      | AC001    | AS03     | T1087        | S0373       |           2 | Discovery       | Account Discovery | Attacker discovered user accounts           |
| EV010    | INC005      | AC003    | AS06     | T1016.001    | S0373       |           1 | Discovery       | Network Discovery | Attacker collected network information       |
| EV011    | INC005      | AC003    | AS06     | T1027.013    | S0447       |           2 | Defense Evasion | Encoded Files   | Data was encoded before transfer              |
| EV012    | INC005      | AC003    | AS04     | T1047        | S0373       |           3 | Execution       | WMI             | Windows Management Instrumentation was used   |
| EV013    | INC006      | AC006    | AS10     | T1059.001    | S0363       |           1 | Execution       | PowerShell      | PowerShell was used to execute malicious commands |
| EV014    | INC006      | AC006    | AS10     | T1053.005    | S0363       |           2 | Persistence     | Scheduled Task  | A scheduled task was created for persistence  |
+----------+-------------+----------+----------+--------------+-------------+-------------+-----------------+-----------------+-----------------------------------------------+
14 rows in set (0.00 sec)
```

ATTACK_EVENT insertion and SELECT output (14 events)

```
mysql> CREATE TABLE EVIDENCE_SOURCE (
    ->     Evidence_ID VARCHAR(10) PRIMARY KEY,
    ->     Incident_ID VARCHAR(10),
    ->     Evidence_Type VARCHAR(100),
    ->     Evidence_Location VARCHAR(200),
    ->     Collected_By VARCHAR(100),
    ->     Collection_Date DATE,
    ->     Description VARCHAR(500),
    ->     FOREIGN KEY (Incident_ID) REFERENCES SECURITY_INCIDENT(Incident_ID)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO EVIDENCE_SOURCE
    -> (Evidence_ID, Incident_ID, Evidence_Type, Evidence_Location, Collected_By, Collection_Date, Description)
    -> VALUES
    -> ('EVS001', 'INC001', 'Email Log', 'Mail Server', 'Analyst 1', '2026-01-15', 'Phishing email containing malicious link'),
    -> ('EVS002', 'INC002', 'System Log', 'Data Center Server', 'Analyst 2', '2026-02-10', 'Server logs showing ransomware activity'),
    -> ('EVS003', 'INC003', 'Authentication Log', 'Domain Controller', 'Analyst 1', '2026-03-05', 'Logs showing unauthorized credential usage'),
    -> ('EVS004', 'INC004', 'Network Log', 'Main Office Network', 'Analyst 3', '2026-04-12', 'Network traffic showing remote access'),
    -> ('EVS005', 'INC005', 'Firewall Log', 'DMZ Firewall', 'Analyst 2', '2026-05-20', 'Firewall logs showing suspicious data transfer'),
    -> ('EVS006', 'INC005', 'Encoded File', 'Web Server', 'Analyst 2', '2026-05-20', 'Encoded data identified on the web server'),
    -> ('EVS007', 'INC006', 'PowerShell Log', 'Operations Workstation', 'Analyst 3', '2026-06-18', 'PowerShell commands executed on workstation'),
    -> ('EVS008', 'INC006', 'System Event Log', 'Operations Workstation', 'Analyst 3', '2026-06-18', 'Scheduled task creation detected');
Query OK, 8 rows affected (0.03 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM EVIDENCE_SOURCE;
+-------------+-------------+--------------------+------------------------+--------------+-----------------+-----------------------------------------------+
| Evidence_ID | Incident_ID | Evidence_Type      | Evidence_Location      | Collected_By | Collection_Date | Description                                   |
+-------------+-------------+--------------------+------------------------+--------------+-----------------+-----------------------------------------------+
| EVS001      | INC001      | Email Log          | Mail Server            | Analyst 1    | 2026-01-15      | Phishing email containing malicious link      |
| EVS002      | INC002      | System Log         | Data Center Server     | Analyst 2    | 2026-02-10      | Server logs showing ransomware activity       |
| EVS003      | INC003      | Authentication Log | Domain Controller      | Analyst 1    | 2026-03-05      | Logs showing unauthorized credential usage    |
| EVS004      | INC004      | Network Log        | Main Office Network    | Analyst 3    | 2026-04-12      | Network traffic showing remote access         |
| EVS005      | INC005      | Firewall Log       | DMZ Firewall           | Analyst 2    | 2026-05-20      | Firewall logs showing suspicious data transfer |
| EVS006      | INC005      | Encoded File       | Web Server             | Analyst 2    | 2026-05-20      | Encoded data identified on the web server     |
| EVS007      | INC006      | PowerShell Log     | Operations Workstation | Analyst 3    | 2026-06-18      | PowerShell commands executed on workstation   |
| EVS008      | INC006      | System Event Log   | Operations Workstation | Analyst 3    | 2026-06-18      | Scheduled task creation detected              |
+-------------+-------------+--------------------+------------------------+--------------+-----------------+-----------------------------------------------+
8 rows in set (0.00 sec)
```

EVIDENCE_SOURCE creation and data

2.8 Conclusion

The Assets and Vulnerabilities part was implemented in MySQL. The ASSET table holds 10 records, and joins connect assets to attack events and MITRE techniques. GROUP BY, HAVING, nested queries and aggregate functions were used to look at attack activity.

A stored function counts the attack events for an asset, and a procedure shows the ordered attack details for one. The trigger rejects an invalid Asset_Status (ERROR 1644), and the view gives a reusable picture of asset and attack information. The dependencies, normal forms and ER relationships all follow the foreign keys that are really in ATTACK_EVENT.

Member 3: Evidence and Investigation

This part covers the forensic and investigation side of the database. Starting from a recorded incident, an analyst should be able to look at the evidence collected for it, see who is leading the response, and get one combined investigation report. There is also audit information, so changes to evidence can be traced.

3.1 What this part covers

The flow I followed is **Incident → Evidence → Security Analyst → Investigation Report**, and each piece maps to a table or view in the database.

Area	How it is done	Commands
Evidence	EVIDENCE_SOURCE joined to SECURITY_INCIDENT and counted per incident	1, 7, 11, 12
Security analyst	INCIDENT_RESPONSE.Lead_Investigator, shown as Security_Analyst in the view	9
Investigation report	View VW_INVESTIGATION_REPORT and procedure sp_investigation_report	9, 13
Attack details	Incident, attack event, actor, asset and technique queries	2–6, 10, 16
Audit	Trigger on EVIDENCE_SOURCE that writes to an audit table, plus UPDATE and DELETE testing	14, 15

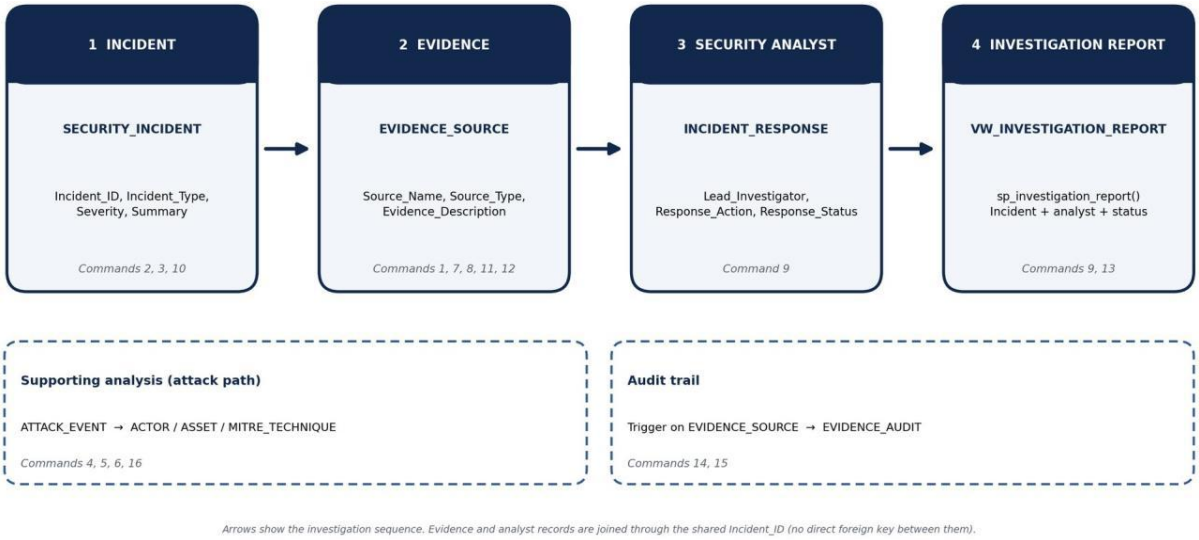


Figure 3.1: Investigation workflow

> The project guidelines mention PL/SQL, but our database (SecurityAttackPathDB) runs on MySQL, which does not support it. The function, procedure and trigger are therefore written as MySQL stored routines.
>
> The schema has no separate Security Analyst table. The analyst is the text column Lead_Investigator in INCIDENT_RESPONSE (for example “SOC Team”), and that is what I used. Nothing was added to the team schema. Evidence and analyst records also have no direct foreign key between them; both link to the incident through Incident_ID.

3.2 Tables in the database

The database mixes VERIS-style incident classification (incident, actor, asset, action) with MITRE ATT&CK reference data (groups, techniques, software). An incident is broken into ordered attack events, and each event

ties together the actor, the asset, the technique and, when known, the software. The row counts are what the team's dbms.sql script inserts.

Table	Rows	Role
SECURITY_INCIDENT	6	The incident under investigation
EVIDENCE_SOURCE	8	Evidence for each incident
INCIDENT_RESPONSE	6	Analyst, response action and status
ATTACK_EVENT	14	The ordered attack steps
ACTOR	6	Who carried out the steps
ASSET	10	What was affected
MITRE_TECHNIQUE	13	How: technique and tactic
MITRE_SOFTWARE	7	Malware or tools used
MITRE_GROUP	5	Threat group an actor belongs to

3.3 SQL commands 1 to 16

These are the 16 commands I ran on SecurityAttackPathDB, in the order I ran them.

Command 1: INNER JOIN

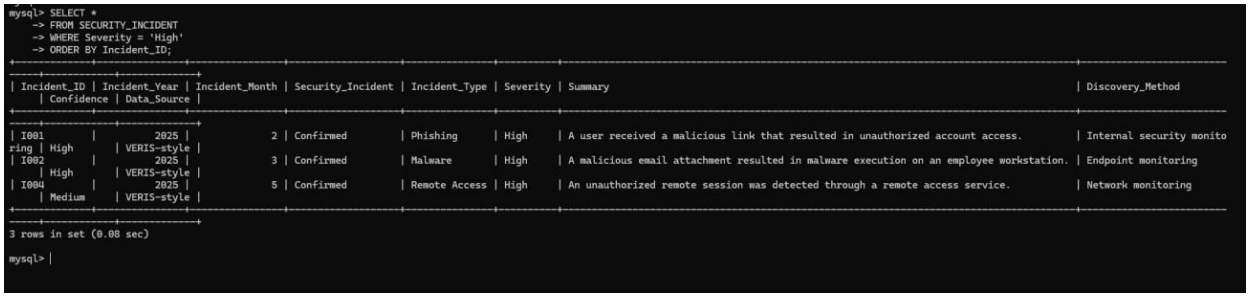
```
SELECT e.Evidence_ID, e.Incident_ID, si.Incident_Type, si.Severity,
       e.Source_Name, e.Source_Type, e.Evidence_Description
FROM EVIDENCE_SOURCE e
INNER JOIN SECURITY_INCIDENT si
        ON si.Incident_ID = e.Incident_ID;
```

```
+-------------+-------------+-----------------+----------+-----------------------------------------+-------------+----
-----------------------------------------+
| Evidence_ID | Incident_ID | Incident_Type   | Severity | Source_Name                             | Source_Type | Evi
dence_Description                        |
+-------------+-------------+-----------------+----------+-----------------------------------------+-------------+----
-----------------------------------------+
| EV001       | I001        | Phishing        | High     | VERIS Community Database                | VERIS       | Inc
ident structure and classification reference |
| EV002       | I001        | Phishing        | High     | MITRE ATT&CK - Phishing                 | MITRE       | Tec
hnique reference for phishing activity   |
| EV003       | I002        | Malware         | High     | MITRE ATT&CK - Emotet                   | MITRE       | Sof
tware and technique reference            |
| EV004       | I003        | Credential Abuse | Critical | VERIS Community Database                | VERIS       | Act
or and asset classification reference    |
| EV005       | I004        | Remote Access   | High     | MITRE ATT&CK - Remote Services          | MITRE       | Rem
ote access technique reference           |
| EV006       | I005        | Ransomware      | Critical | MITRE ATT&CK - Data Encrypted for Impact | MITRE       | Imp
act technique reference                  |
| EV007       | I006        | Supply Chain    | Critical | MITRE ATT&CK - SolarWinds Compromise    | MITRE       | Cam
paign and attack technique reference     |
| EV008       | I006        | Supply Chain    | Critical | MITRE ATT&CK - GoldFinder               | MITRE       | Sof
tware and network discovery reference    |
+-------------+-------------+-----------------+----------+-----------------------------------------+-------------+----
-----------------------------------------+
8 rows in set (0.01 sec)
```

Joins every evidence record to its incident through Incident_ID, so the incident type and severity appear next to each piece of evidence. It returns 8 rows, EV001 to EV008. The table wraps in the terminal, which is why Evidence_Description drops onto a second line.

Command 2: WHERE and ORDER BY

```
SELECT *
FROM SECURITY_INCIDENT
WHERE Severity = 'High'
ORDER BY Incident_ID;
```



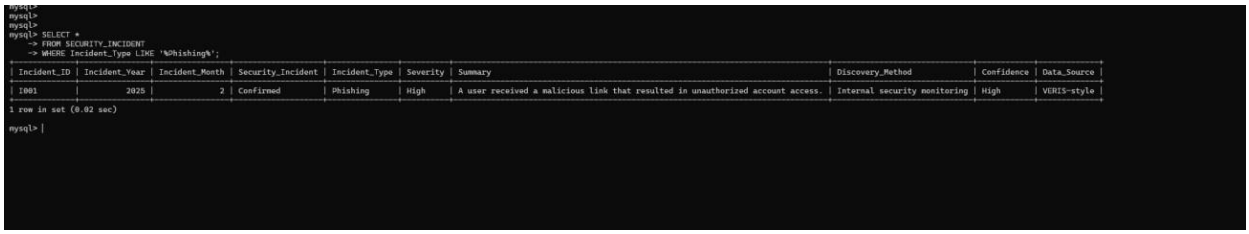
```
mysql> SELECT *
    -> FROM SECURITY_INCIDENT
    -> WHERE Severity = 'High'
    -> ORDER BY Incident_ID;
+-------------+---------------+----------------+-------------------+---------------+----------+-----------------------------------------------------------------------------------+-------------------------+
| Incident_ID | Incident_Year | Incident_Month | Security_Incident | Incident_Type | Severity | Summary                                                                           | Discovery_Method        |
| Confidence | Data_Source |
+-------------+---------------+----------------+-------------------+---------------+----------+-----------------------------------------------------------------------------------+-------------------------+
| I001        |          2025 |              2 | Confirmed         | Phishing      | High     | A user received a malicious link that resulted in unauthorized account access.    | Internal security monito
ring | High       | VERIS-style |
| I002        |          2025 |              3 | Confirmed         | Malware       | High     | A malicious email attachment resulted in malware execution on an employee workstation. | Endpoint monitoring
| High       | VERIS-style |
| I004        |          2025 |              5 | Confirmed         | Remote Access | High     | An unauthorized remote session was detected through a remote access service.      | Network monitoring
| Medium     | VERIS-style |
+-------------+---------------+----------------+-------------------+---------------+----------+-----------------------------------------------------------------------------------+-------------------------+
3 rows in set (0.08 sec)

mysql>
```

Shows only the High-severity incidents, sorted by Incident_ID. Three rows come back: I001 (Phishing), I002 (Malware) and I004 (Remote Access).

Command 3: LIKE

```
SELECT *
FROM SECURITY_INCIDENT
WHERE Incident_Type LIKE '%Phishing%';
```



```
mysql>
mysql>
mysql>
mysql> SELECT *
    -> FROM SECURITY_INCIDENT
    -> WHERE Incident_Type LIKE '%Phishing%';
+-------------+---------------+----------------+-------------------+---------------+----------+-----------------------------------------------------------------------------------+-------------------------+------------+-------------+
| Incident_ID | Incident_Year | Incident_Month | Security_Incident | Incident_Type | Severity | Summary                                                                           | Discovery_Method        | Confidence | Data_Source |
+-------------+---------------+----------------+-------------------+---------------+----------+-----------------------------------------------------------------------------------+-------------------------+------------+-------------+
| I001        |          2025 |              2 | Confirmed         | Phishing      | High     | A user received a malicious link that resulted in unauthorized account access.    | Internal security monitoring | High       | VERIS-style |
+-------------+---------------+----------------+-------------------+---------------+----------+-----------------------------------------------------------------------------------+-------------------------+------------+-------------+
1 row in set (0.02 sec)

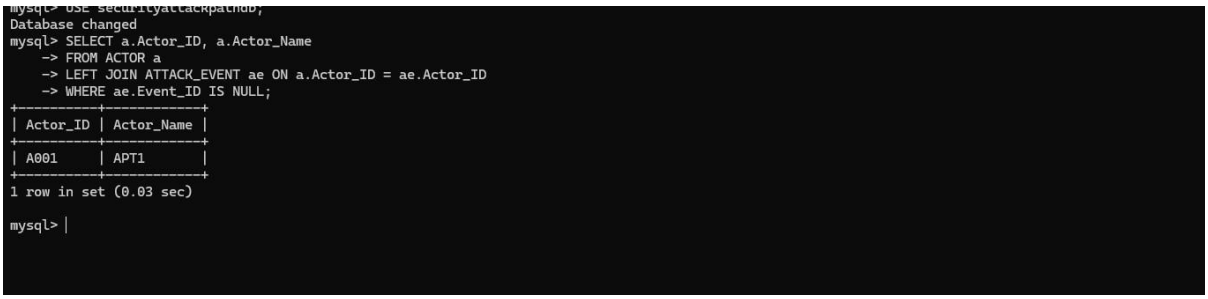
mysql>
```

Searches for incidents whose type contains the word Phishing. The % signs allow any text before or after it. Only I001 matches.

Command 4: LEFT JOIN with IS NULL

```
USE securityattackpathdb;

SELECT a.Actor_ID, a.Actor_Name
FROM ACTOR a
LEFT JOIN ATTACK_EVENT ae
       ON a.Actor_ID = ae.Actor_ID
WHERE ae.Event_ID IS NULL;
```



```
mysql> USE securityattackpathdb;
Database changed
mysql> SELECT a.Actor_ID, a.Actor_Name
    -> FROM ACTOR a
    -> LEFT JOIN ATTACK_EVENT ae ON a.Actor_ID = ae.Actor_ID
    -> WHERE ae.Event_ID IS NULL;
+----------+------------+
| Actor_ID | Actor_Name |
+----------+------------+
| A001     | APT1       |
+----------+------------+
1 row in set (0.03 sec)

mysql>
```

Finds actors that are not part of any attack event. LEFT JOIN keeps every actor, and IS NULL picks out the ones with no matching event (NULL cannot be tested with =). One row comes back, A001 (APT1), which is recorded in the actor table but has no attack event.

Command 5: Multiple joins

```
SELECT si.Incident_ID, ae.Event_ID, ac.Actor_Name,
       ast.Asset_Variety, mt.Technique_Name
FROM ATTACK_EVENT ae
INNER JOIN SECURITY_INCIDENT si ON si.Incident_ID  = ae.Incident_ID
INNER JOIN ACTOR ac             ON ac.Actor_ID     = ae.Actor_ID
INNER JOIN ASSET ast            ON ast.Asset_ID    = ae.Asset_ID
INNER JOIN MITRE_TECHNIQUE mt   ON mt.Technique_ID = ae.Technique_ID
ORDER BY si.Incident_ID, ae.Event_ID;
```

```
mysql>
mysql>
mysql> SELECT si.Incident_ID,
    ->        ae.Event_ID,
    ->        ac.Actor_Name,
    ->        ast.Asset_Variety,
    ->        mt.Technique_Name
    -> FROM ATTACK_EVENT ae
    -> INNER JOIN SECURITY_INCIDENT si ON si.Incident_ID = ae.Incident_ID
    -> INNER JOIN ACTOR ac ON ac.Actor_ID = ae.Actor_ID
    -> INNER JOIN ASSET ast ON ast.Asset_ID = ae.Asset_ID
    -> INNER JOIN MITRE_TECHNIQUE mt ON mt.Technique_ID = ae.Technique_ID
    -> ORDER BY si.Incident_ID, ae.Event_ID;
+-------------+----------+-----------------------+--------------------+-----------------------------------------------------------------------+
| Incident_ID | Event_ID | Actor_Name            | Asset_Variety      | Technique_Name                                                        |
+-------------+----------+-----------------------+--------------------+-----------------------------------------------------------------------+
| I001        | E001     | Wizard Spider         | Employee Laptop    | Phishing: Spearphishing Link                                          |
| I001        | E002     | Wizard Spider         | Employee Laptop    | User Execution: Malicious Link                                        |
| I002        | E003     | OilRig                | Employee Workstation | Phishing                                                            |
| I002        | E004     | OilRig                | Employee Workstation | Command and Scripting Interpreter: PowerShell                       |
| I003        | E005     | APT3                  | Domain Controller  | OS Credential Dumping                                                 |
| I003        | E006     | APT3                  | Domain Controller  | Valid Accounts                                                        |
| I004        | E007     | Unknown External Actor | VPN Gateway       | Valid Accounts                                                        |
| I004        | E008     | Unknown External Actor | Domain Controller | Remote Services: Remote Desktop Protocol                              |
| I005        | E009     | Wizard Spider         | File Server        | Data Encrypted for Impact                                             |
| I005        | E010     | Wizard Spider         | Backup Server      | Data Encrypted for Impact                                             |
| I006        | E011     | APT29                 | Build Server       | Obfuscated Files or Information                                       |
| I006        | E012     | APT29                 | Build Server       | Command and Scripting Interpreter: PowerShell                         |
| I006        | E013     | APT29                 | Database Server    | Account Discovery                                                     |
| I006        | E014     | APT29                 | Network Server     | System Network Configuration Discovery: Internet Connection Discovery |
+-------------+----------+-----------------------+--------------------+-----------------------------------------------------------------------+
14 rows in set (0.03 sec)
```

A five-table join that shows the incident, actor, affected asset and MITRE technique for every attack step. ATTACK_EVENT is the central table because it holds all the foreign keys. It returns 14 rows, one per event. With inner joins an event missing any of the actor, asset or technique would be dropped, but in our data all 14 are complete.

Command 6: DISTINCT

```
SELECT DISTINCT si.Incident_ID, si.Incident_Type,
       ast.Asset_ID, ast.Asset_Variety
FROM SECURITY_INCIDENT si
INNER JOIN ATTACK_EVENT ae ON ae.Incident_ID = si.Incident_ID
INNER JOIN ASSET ast       ON ast.Asset_ID   = ae.Asset_ID
ORDER BY si.Incident_ID, ast.Asset_ID;
```

```
mysql>
mysql>
mysql>
mysql>
mysql> SELECT DISTINCT
    ->        si.Incident_ID,
    ->        si.Incident_Type,
    ->        ast.Asset_ID,
    ->        ast.Asset_Variety
    -> FROM SECURITY_INCIDENT si
    -> INNER JOIN ATTACK_EVENT ae ON ae.Incident_ID = si.Incident_ID
    -> INNER JOIN ASSET ast ON ast.Asset_ID = ae.Asset_ID
    -> ORDER BY si.Incident_ID, ast.Asset_ID;
+-------------+------------------+----------+----------------------+
| Incident_ID | Incident_Type    | Asset_ID | Asset_Variety        |
+-------------+------------------+----------+----------------------+
| I001        | Phishing         | AS01     | Employee Laptop      |
| I002        | Malware          | AS09     | Employee Workstation |
| I003        | Credential Abuse | AS03     | Domain Controller    |
| I004        | Remote Access    | AS03     | Domain Controller    |
| I004        | Remote Access    | AS04     | VPN Gateway          |
| I005        | Ransomware       | AS05     | File Server          |
| I005        | Ransomware       | AS06     | Backup Server        |
| I006        | Supply Chain     | AS07     | Build Server         |
| I006        | Supply Chain     | AS08     | Database Server      |
| I006        | Supply Chain     | AS10     | Network Server       |
+-------------+------------------+----------+----------------------+
10 rows in set (0.01 sec)

mysql>
```

Lists the unique assets hit in each incident. I001 has two attack events on AS01 (Employee Laptop), but DISTINCT shows it only once. The result has 10 rows.

Command 7: GROUP BY with COUNT

```
SELECT Incident_ID, COUNT(*) AS Evidence_Count
FROM EVIDENCE_SOURCE
GROUP BY Incident_ID
ORDER BY Evidence_Count DESC;
```

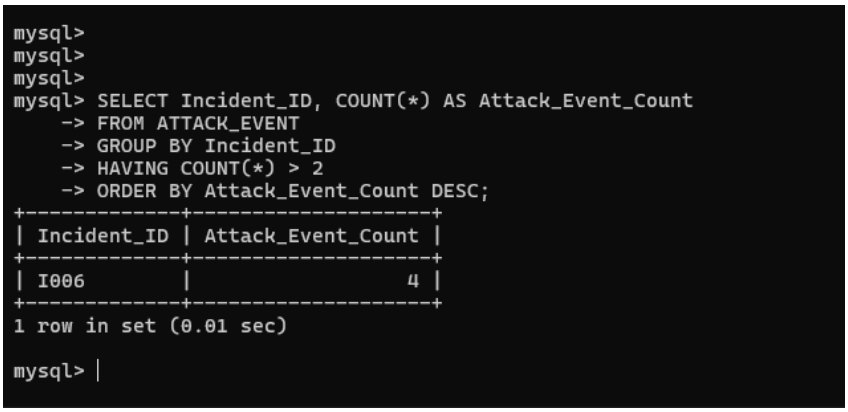
```
mysql>
mysql>
mysql> SELECT Incident_ID, COUNT(*) AS Evidence_Count
    -> FROM EVIDENCE_SOURCE
    -> GROUP BY Incident_ID
    -> ORDER BY Evidence_Count DESC;
+-------------+----------------+
| Incident_ID | Evidence_Count |
+-------------+----------------+
| I001        |              2 |
| I006        |              2 |
| I002        |              1 |
| I003        |              1 |
| I004        |              1 |
| I005        |              1 |
+-------------+----------------+
6 rows in set (0.01 sec)

mysql>
```

Counts the evidence sources for each incident. I001 and I006 have two each, and I002, I003, I004 and I005 have one each.

Command 8: HAVING

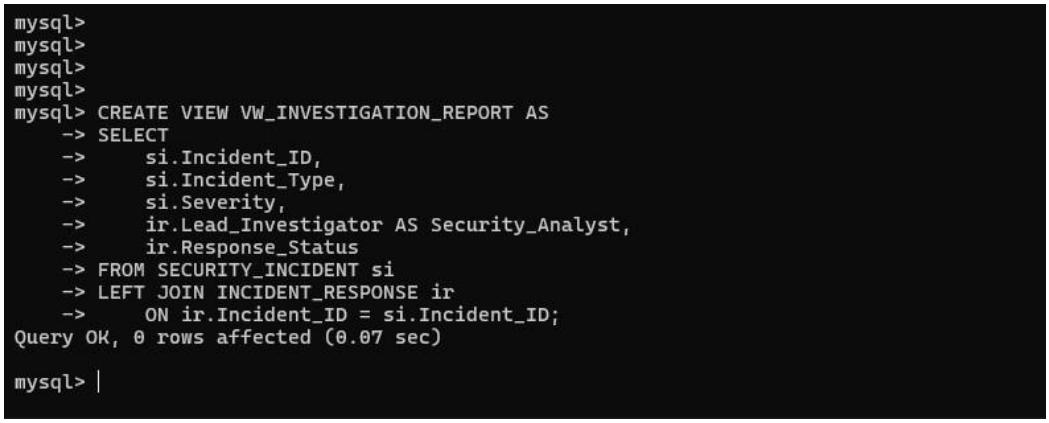
```
SELECT Incident_ID, COUNT(*) AS Attack_Event_Count
FROM ATTACK_EVENT
GROUP BY Incident_ID
HAVING COUNT(*) > 2
ORDER BY Attack_Event_Count DESC;
```



Finds incidents with more than two attack events. Only I006 qualifies, with four. WHERE filters rows before grouping, while HAVING filters the groups after the count is worked out.

Command 9: CREATE VIEW

```
CREATE VIEW VW_INVESTIGATION_REPORT AS
SELECT si.Incident_ID, si.Incident_Type, si.Severity,
       ir.Lead_Investigator AS Security_Analyst,
       ir.Response_Status
FROM SECURITY_INCIDENT si
LEFT JOIN INCIDENT_RESPONSE ir
       ON ir.Incident_ID = si.Incident_ID;
```



Creates VW_INVESTIGATION_REPORT, a saved query that puts each incident next to its response details. Lead_Investigator is shown as Security_Analyst, and the LEFT JOIN keeps incidents that have no response record yet. MySQL only reports “Query OK, 0 rows affected” because a view stores the query, not the data, so it always reflects the current tables.

Command 10: CASE

```
SELECT Incident_ID, Incident_Type, Severity,
       CASE
           WHEN Severity = 'Critical' THEN 'Immediate Attention'
           WHEN Severity = 'High'     THEN 'High Priority'
           ELSE 'Normal Priority'
       END AS Priority_Status
FROM SECURITY_INCIDENT;
```

```
mysql> SELECT
    ->     Incident_ID,
    ->     Incident_Type,
    ->     Severity,
    ->     CASE
    ->         WHEN Severity = 'Critical' THEN 'Immediate Attention'
    ->         WHEN Severity = 'High' THEN 'High Priority'
    ->         ELSE 'Normal Priority'
    ->     END AS Priority_Status
    -> FROM SECURITY_INCIDENT;
+-------------+------------------+----------+---------------------+
| Incident_ID | Incident_Type    | Severity | Priority_Status     |
+-------------+------------------+----------+---------------------+
| I001        | Phishing         | High     | High Priority       |
| I002        | Malware          | High     | High Priority       |
| I003        | Credential Abuse | Critical | Immediate Attention |
| I004        | Remote Access    | High     | High Priority       |
| I005        | Ransomware       | Critical | Immediate Attention |
| I006        | Supply Chain     | Critical | Immediate Attention |
+-------------+------------------+----------+---------------------+
6 rows in set (0.00 sec)

mysql>
```

Adds a Priority_Status column based on severity. The Critical incidents (I003, I005, I006) become Immediate Attention and the High ones (I001, I002, I004) become High Priority.

Command 11: Subquery

```
SELECT Incident_ID, Incident_Type, Severity, Summary
FROM SECURITY_INCIDENT
WHERE Incident_ID IN (
    SELECT Incident_ID
    FROM EVIDENCE_SOURCE
    GROUP BY Incident_ID
    HAVING COUNT(*) > 1
);
```

```
mysql>
mysql>
mysql> SELECT Incident_ID, Incident_Type, Severity, Summary
    -> FROM SECURITY_INCIDENT
    -> WHERE Incident_ID IN (
    ->  SELECT Incident_ID
    ->  FROM EVIDENCE_SOURCE
    ->  GROUP BY Incident_ID
    ->  HAVING COUNT(*) > 1
    ->  );
+-------------+--------------+----------+-----------------------------------------------------------------------------------------------------+
| Incident_ID | Incident_Type | Severity | Summary                                                                                             |
+-------------+--------------+----------+-----------------------------------------------------------------------------------------------------+
| I006        | Supply Chain  | Critical | A software supply-chain compromise resulted in unauthorized access to affected environments. |
+-------------+--------------+----------+-----------------------------------------------------------------------------------------------------+
1 row in set (0.04 sec)

mysql>
```

Finds incidents with more than one evidence source. The inner query groups EVIDENCE_SOURCE by incident and keeps the groups with a count above one, and the outer query then fetches those incidents' details. The output shows I006.

Command 12: Stored function

```
DELIMITER $$

CREATE FUNCTION fn_evidence_count(p_incident_id VARCHAR(10))
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO v_count
    FROM EVIDENCE_SOURCE
    WHERE Incident_ID = p_incident_id;

    RETURN v_count;
END$$

DELIMITER ;
```

```
mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION fn_evidence_count(p_incident_id VARCHAR(10))
    -> RETURNS INT
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_count INT DEFAULT 0;
    ->
    ->     SELECT COUNT(*) INTO v_count
    ->     FROM EVIDENCE_SOURCE
    ->     WHERE Incident_ID = p_incident_id;
    ->
    ->     RETURN v_count;
    -> END$$
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

fn_evidence_count takes an Incident_ID and returns how many evidence sources it has. DELIMITER $$ is there so MySQL does not end the command at the first semicolon inside the function body.

Command 13: Stored procedure

```
DELIMITER $$

CREATE PROCEDURE sp_investigation_report(IN p_incident_id VARCHAR(10))
BEGIN
    SELECT Incident_ID, Incident_Type, Severity, Summary
    FROM SECURITY_INCIDENT
    WHERE Incident_ID = p_incident_id;
END$$

DELIMITER ;

CALL sp_investigation_report('I006');
```

```
mysql> CREATE PROCEDURE sp_investigation_report(IN p_incident_id VARCHAR(10))
    -> BEGIN
    ->     SELECT Incident_ID, Incident_Type, Severity, Summary
    ->     FROM SECURITY_INCIDENT
    ->     WHERE Incident_ID = p_incident_id;
    -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL sp_investigation_report('I006');
+-------------+---------------+----------+-----------------------------------------------------------------------------------------------------+
| Incident_ID | Incident_Type | Severity | Summary                                                                                             |
+-------------+---------------+----------+-----------------------------------------------------------------------------------------------------+
| I006        | Supply Chain  | Critical | A software supply-chain compromise resulted in unauthorized access to affected environments. |
+-------------+---------------+----------+-----------------------------------------------------------------------------------------------------+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.02 sec)
```

sp_investigation_report takes an Incident_ID and displays that incident's details. Called with I006 it returns the Supply Chain incident, marked Critical, together with its summary. A procedure is run with CALL and can return a result set, unlike a function.

Command 14: Trigger

```
DELIMITER $$

CREATE TRIGGER trg_evidence_after_insert
AFTER INSERT ON EVIDENCE_SOURCE
FOR EACH ROW
BEGIN
    INSERT INTO EVIDENCE_AUDIT
        (Evidence_ID, Action_Type, New_Incident_ID,
         New_Source_Name, New_Source_Type,
         New_Evidence_Description, Changed_By)
    VALUES
        (NEW.Evidence_ID, 'INSERT', NEW.Incident_ID,
         NEW.Source_Name, NEW.Source_Type,
         NEW.Evidence_Description, CURRENT_USER());
END$$

DELIMITER ;
```



This gives evidence an audit trail. Whenever a row is inserted into EVIDENCE_SOURCE, the trigger copies it into EVIDENCE_AUDIT along with the MySQL account that made the change (CURRENT_USER()). NEW refers to the row that was just inserted.

Command 15: UPDATE and DELETE

```
UPDATE EVIDENCE_SOURCE
SET Source_Name = 'Updated Evidence Source'
WHERE Evidence_ID = 'EV001';

DELETE FROM EVIDENCE_SOURCE
WHERE Evidence_ID = 'EV001';
```

```
mysql>
mysql> DELIMITER ;
mysql>
mysql>
mysql>
mysql>
mysql> UPDATE EVIDENCE_SOURCE
    -> SET Source_Name = 'Updated Evidence Source'
    -> WHERE Evidence_ID = 'EV001';
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> DELETE FROM EVIDENCE_SOURCE
    -> WHERE Evidence_ID = 'EV001';
Query OK, 1 row affected (0.01 sec)

mysql>
```

Tests changing and removing an evidence row. Both statements filter on the primary key, so only EV001 is touched. The update shows one row matched and one changed, and the delete reports one row affected.

Command 16: Attack events per MITRE tactic

```
SELECT mt.Tactic,
       COUNT(*) AS Event_Count
FROM ATTACK_EVENT ae
INNER JOIN MITRE_TECHNIQUE mt
        ON mt.Technique_ID = ae.Technique_ID
GROUP BY mt.Tactic
ORDER BY Event_Count DESC;
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> SELECT mt.Tactic,
    ->        COUNT(*) AS Event_Count
    -> FROM ATTACK_EVENT ae
    -> INNER JOIN MITRE_TECHNIQUE mt
    ->     ON mt.Technique_ID = ae.Technique_ID
    -> GROUP BY mt.Tactic
    -> ORDER BY Event_Count DESC;
+----------------------------+-------------+
| Tactic                     | Event_Count |
+----------------------------+-------------+
| Execution                  |           3 |
| Discovery                  |           2 |
| Initial Access / Persistence |           2 |
| Impact                     |           2 |
| Initial Access             |           2 |
| Credential Access          |           1 |
| Lateral Movement           |           1 |
| Defense Evasion            |           1 |
+----------------------------+-------------+
8 rows in set (0.01 sec)

mysql>
```

Counts attack events for each MITRE tactic. Eight rows come back, with Execution highest at 3, and the counts add up to 14, the number of events. “Initial Access / Persistence” is one stored tactic value (used by technique T1078), so it is counted separately from “Initial Access”.

3.4 ER diagram

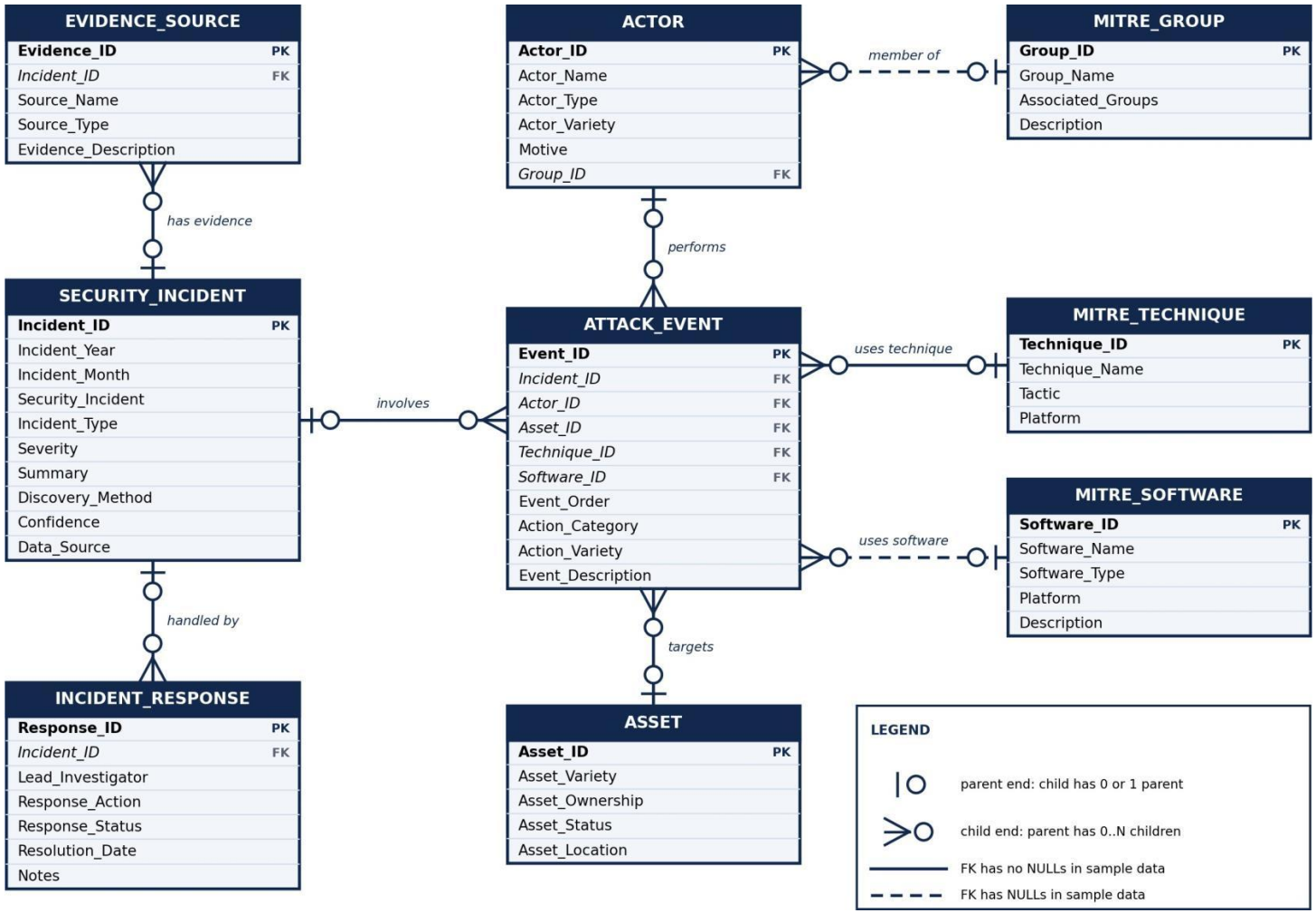


Figure 3.2: ER diagram drawn from the foreign keys declared in dbms.sql (crow's foot notation)

3.5 EER diagram

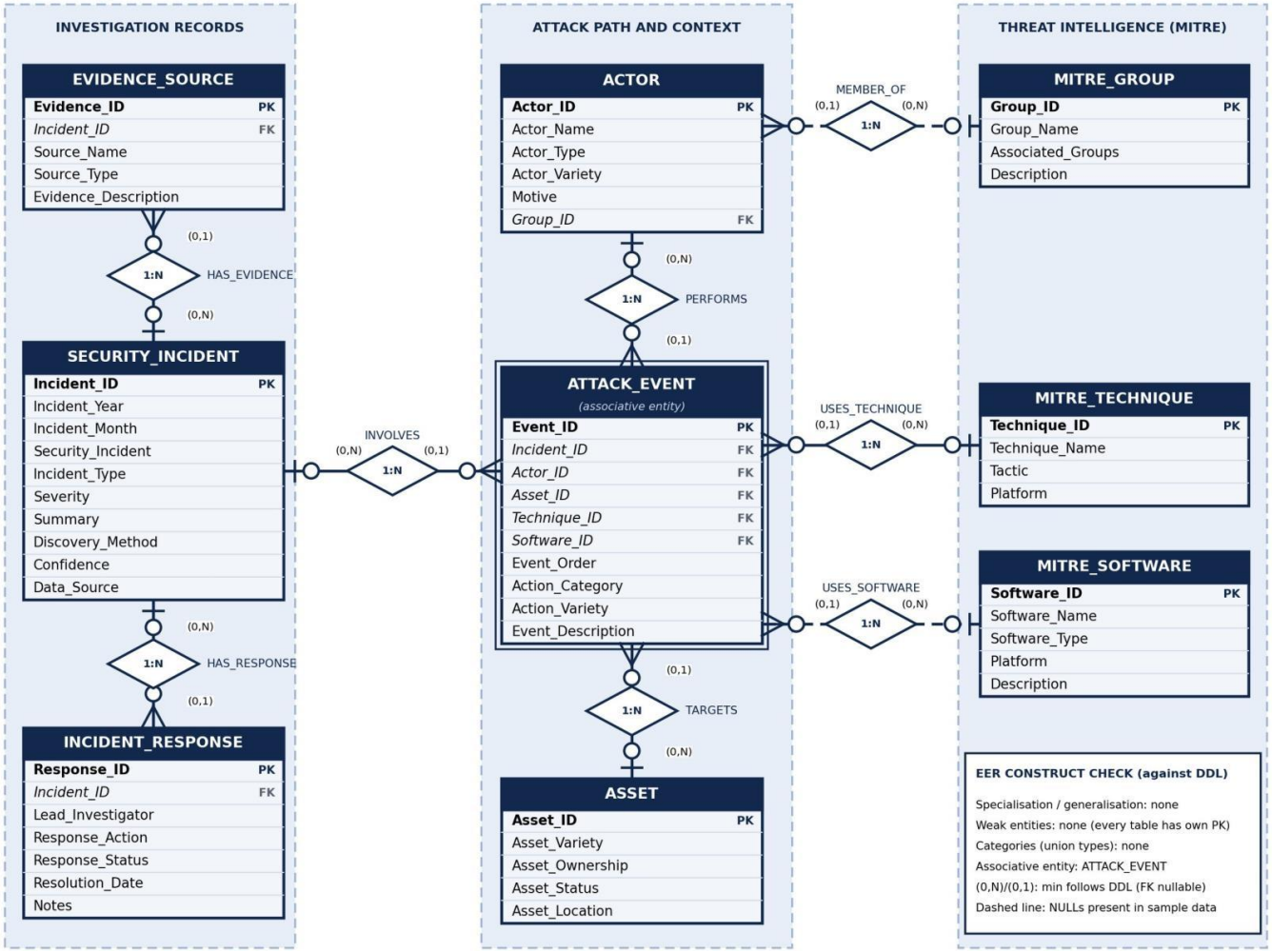


Figure 3.3: EER diagram with participation constraints, grouped by subject area

3.6 Functional dependencies and normalization

The dependencies come from the primary and foreign keys in dbms.sql. No UNIQUE constraints are declared, so the primary key is the only candidate key in each table.

Relation	Functional dependency
SECURITY_INCIDENT	Incident_ID → Incident_Year, Incident_Month, Security_Incident, Incident_Type, Severity, Summary, Discovery_Method, Confidence, Data_Source
EVIDENCE_SOURCE	Evidence_ID → Incident_ID, Source_Name, Source_Type, Evidence_Description
INCIDENT_RESPONSE	Response_ID → Incident_ID, Lead_Investigator, Response_Action, Response_Status, Resolution_Date, Notes
ATTACK_EVENT	Event_ID → Incident_ID, Actor_ID, Asset_ID, Technique_ID, Software_ID, Event_Order, Action_Category, Action_Variety, Event_Description
ACTOR	Actor_ID → Actor_Name, Actor_Type, Actor_Variety, Motive, Group_ID
ASSET	Asset_ID → Asset_Variety, Asset_Ownership, Asset_Status, Asset_Location
MITRE_GROUP	Group_ID → Group_Name, Associated_Groups, Description
MITRE_TECHNIQUE	Technique_ID → Technique_Name, Tactic, Platform
MITRE_SOFTWARE	Software_ID → Software_Name, Software_Type, Platform, Description

Foreign keys such as ATTACK_EVENT.Actor_ID point to another table's primary key, so actor details are stored once in ACTOR rather than repeated in ATTACK_EVENT. The same technique does not fix the action category: events E006 and E007 both use T1078, but E006 is Credential Access and E007 is Initial Access, so Technique_ID → Action_Category is not a dependency here.

Relation	1NF	2NF	3NF / BCNF	Comment
SECURITY_INCIDENT	Yes	Yes	Yes	Single-column key; nothing non-key determines anything else
EVIDENCE_SOURCE	Yes	Yes	Yes	Only Incident_ID is stored, not incident details
INCIDENT_RESPONSE	Yes	Yes	Yes	Lead_Investigator is plain text, not a key
ATTACK_EVENT	Yes	Yes	Yes	Holds only keys and event-specific attributes
ACTOR	Yes	Yes	Yes	Group details live in MITRE_GROUP
ASSET	Yes	Yes	Yes	Everything depends on Asset_ID only
MITRE_GROUP	Partly	Yes	Yes	Associated_Groups holds a list of names
MITRE_TECHNIQUE	Partly	Yes	Yes	Platform (sometimes Tactic) holds several values
MITRE_SOFTWARE	Partly	Yes	Yes	Platform can hold several values

Every table has a single-column primary key, so 2NF follows as soon as a table is in 1NF. In each table the primary key is the only determinant and no non-key column determines another, so there are no transitive dependencies, which gives 3NF and BCNF. The only weak spot is 1NF: a few descriptive columns store comma-separated lists, such as Platform = "Windows, Linux, macOS". They are never used as keys or in joins, so none of the queries in this report are affected, but strictly speaking they are not atomic.

3.7 ER relationships and cardinalities

Every relationship comes from a FOREIGN KEY clause in dbms.sql. The parent holds the primary key and the child holds the foreign key.

Parent (one)	Child (many)	Foreign key	Cardinality	In the sample data
SECURITY_INCIDENT	EVIDENCE_SOURCE	Incident_ID	1 : N	Filled in all 8 rows
SECURITY_INCIDENT	INCIDENT_RESPONSE	Incident_ID	1 : N	Filled in all 6 rows
SECURITY_INCIDENT	ATTACK_EVENT	Incident_ID	1 : N	Filled in all 14 rows
ACTOR	ATTACK_EVENT	Actor_ID	1 : N	Filled in all 14 rows
ASSET	ATTACK_EVENT	Asset_ID	1 : N	Filled in all 14 rows
MITRE_TECHNIQUE	ATTACK_EVENT	Technique_ID	1 : N	Filled in all 14 rows
MITRE_SOFTWARE	ATTACK_EVENT	Software_ID	1 : N	NULL in 7 of 14 rows
MITRE_GROUP	ACTOR	Group_ID	1 : N	NULL for actor A006

- No foreign key is declared NOT NULL, so on paper participation is optional everywhere. The last two relationships are optional in practice too, because the sample data has NULLs there (dashed lines in the diagrams).
- INCIDENT_RESPONSE is 1 : N because the schema allows several responses per incident, even though the sample data has one each.
- ATTACK_EVENT links five parent tables, so it works as an associative entity: through it, incidents and actors, incidents and assets, incidents and techniques, and actors and assets are related many-to-many.
- There is no direct foreign key between EVIDENCE_SOURCE and ATTACK_EVENT, or between EVIDENCE_SOURCE and INCIDENT_RESPONSE. They are connected only through SECURITY_INCIDENT.
- The schema has no specialisation, weak entities or category types, so the EER diagram adds only what the schema supports: participation constraints, ATTACK_EVENT as an associative entity, and grouping by subject area.

3.8 Conclusion

The 16 commands show how, starting from an incident, you can pull out its evidence, the analyst responsible, the attack events and the actors, assets and MITRE techniques involved, using joins, filtering, grouping, a view, a function, a procedure and a trigger.

Two pieces carry the investigation report: the view VW_INVESTIGATION_REPORT puts the incident, analyst and response status together, and the procedure sp_investigation_report displays any chosen incident. The trigger on EVIDENCE_SOURCE adds an audit entry for new evidence. The diagrams, dependencies and normalization show a design built on single-column primary keys joined by foreign keys, with every table in third normal form apart from a few list-valued descriptive columns.

Member 4: Response and Transaction Management

4.1 Member 4 Responsibility

Member 4 handles the response and transaction side of the Security Incident Attack-Path Database. Starting from a recorded security incident, the module lets an analyst assign a response, change the status of the incident's response, resolve the incident, and trace every change through an audit trail. It also shows how related changes are grouped into transactions and how several analysts can work on the same record at the same time without overwriting each other.

Member 4 also contributes to the shared ER/EER diagrams, DDL/DML, SQL queries, views, stored routines, triggers, functional dependencies, normalization and serializability analysis (sections 4.5 to 4.10).

The Member 4 workflow demonstrated in this report is:

Assign Response → Change Status → Resolve Incident → Trigger → Transaction / Concurrency

4.1.1 Assigned responsibilities and where they are implemented

Responsibility	How it is implemented in this report	Commands
Incident Response	INCIDENT_RESPONSE joined with SECURITY_INCIDENT; incidents without a response found with LEFT JOIN	1, 4, 5, 10
Response actions	INCIDENT_RESPONSE.Response_Action and Lead_Investigator; a response is assigned with INSERT; procedure sp_update_response_status	3, 12, 14
Incident status	INCIDENT_RESPONSE.Response_Status (SECURITY_INCIDENT has no status column); filtered, counted and classified	2, 6, 9, 14
Resolution	Resolution_Date is filled when the status becomes resolved	12, 14
Audit-related information	AFTER UPDATE trigger on INCIDENT_RESPONSE that writes to the audit table RESPONSE_AUDIT	13, 14
Views	VW_RESPONSE_STATUS	8
GROUP BY / HAVING	Responses per status; analysts leading more than one response	6, 7
Joins	INNER JOIN, LEFT JOIN and multi-table joins	1, 4, 5, 8
Functions	Stored function fn_open_response_count (MySQL)	11
Procedures	Stored procedure sp_update_response_status (MySQL)	12
Trigger	AFTER UPDATE trigger trg_response_after_update	13

Transactions	START TRANSACTION, SAVEPOINT, ROLLBACK and COMMIT	15
Concurrency	Row locking with SELECT ... FOR UPDATE in two sessions	16
Serializability	Schedules, conflicts and precedence graphs	Section 4.10

NOTE The project guidelines list "PL/SQL functions". The team database (`SecurityAttackPathDB`) runs on **MySQL**, which does not support PL/SQL, so the function, procedure and trigger are written as MySQL stored routines.

The schema has **no separate incident status column**. The status of an incident's response is recorded in the text column `Response_Status` of INCIDENT_RESPONSE, with the date in `Resolution_Date`, and those columns are used as the incident status in this module. The only object added is the log table RESPONSE_AUDIT (Command 13); no table or column of the team schema has been changed.

4.2 Overview of Response & Transaction Management

The database records security incidents using a hybrid of two frameworks: VERIS-style incident classification (incident, actor, asset, action) and MITRE ATT&CK reference data (groups, techniques, software). An incident is broken down into ordered attack events; each event links the incident to the actor who performed it, the asset it affected, the MITRE technique used and, when known, the software involved. **2.1 Tables in the database (from dbms.sql)**

Table	Primary key	Foreign keys	Rows	Role in Member 4 module
SECURITY_INCIDENT	Incident_ID	—	6	Core: the incident being responded to
EVIDENCE_SOURCE	Evidence_ID	Incident_ID	8	Supporting: evidence behind the incident
INCIDENT_RESPONSE	Response_ID	Incident_ID	6	Core: analyst, action, status, resolution
ATTACK_EVENT	Event_ID	Incident_ID, Actor_ID, Asset_ID, Technique_ID, Software_ID	14	Supporting: attack sequence to contain
ACTOR	Actor_ID	Group_ID	6	Supporting: who acted
ASSET	Asset_ID	—	10	Supporting: what was affected and must be restored
MITRE_TECHNIQUE	Technique_ID	—	13	Supporting: how (tactic, technique)
MITRE_SOFTWARE	Software_ID	—	7	Supporting: malware or tools
MITRE_GROUP	Group_ID	—	5	Supporting: threat group of an actor

Row counts are the number of rows inserted by the team's dbms.sql script. The log table RESPONSE_AUDIT is created in Command 13 and is not part of dbms.sql.

4.2.2 What happens when an incident is responded to

- **Incident.** The response starts from a SECURITY_INCIDENT record: type, severity, summary and how it was discovered.
- **Assign response.** A new record in INCIDENT_RESPONSE names the lead investigator and the response action and starts with an open status.
- **Change status.** Response_Status is updated while the work continues; the procedure sp_update_response_status performs the change.
- **Resolve incident.** The status is set to resolved and Resolution_Date is filled with the date of resolution.
- **Audit.** The trigger trg_response_after_update records the old and new status and analyst in RESPONSE_AUDIT, so every change can be traced.
- **Transaction and concurrency.** Related changes are grouped in one transaction, and row locks stop two analysts from overwriting each other's update (Commands 15 and 16).

4.3 SQL Implementation and Results — Commands 1–16

This section presents the 16 SQL commands prepared for MySQL on the SecurityAttackPathDB database, in the order in which they are demonstrated. Each command is followed by a placeholder for its MySQL output screenshot, an explanation, the result expected from the sample data, and the SQL concepts it demonstrates.

Boxes marked REVIEW NOTE flag points that need attention before submission and can be deleted once they are resolved.

> **REVIEW NOTE** These commands are **proposed** and have not been run for this report, so no output is shown. Run each one in MySQL and paste the real screenshot into its placeholder.
>
> The commands use the column names of dbms.sql. Status values such as "Open", "In Progress" and "Resolved" are assumed; if the Response_Status values in dbms.sql differ, change them in Commands 2, 9, 11, 12 and 14.

No.	Command	Main concept	Tables used
1	INNER JOIN	Join responses to incidents	INCIDENT_RESPONSE, SECURITY_INCIDENT
2	WHERE + ORDER BY	Responses not yet resolved	INCIDENT_RESPONSE
3	LIKE	Responses led by a team	INCIDENT_RESPONSE
4	LEFT JOIN + IS NULL	Incidents with no response	SECURITY_INCIDENT, INCIDENT_RESPONSE
5	MULTIPLE JOIN	Response with attack path	INCIDENT_RESPONSE, SECURITY_INCIDENT, ATTACK_EVENT, ACTOR, ASSET
6	GROUP BY + COUNT	Responses per status	INCIDENT_RESPONSE
7	HAVING	Analysts leading more than one response	INCIDENT_RESPONSE

8	CREATE VIEW	Response status view	SECURITY_INCIDENT, INCIDENT_RESPONSE
9	CASE	Response priority classification	SECURITY_INCIDENT, INCIDENT_RESPONSE
10	SUBQUERY	Responses for incidents with many events	INCIDENT_RESPONSE, ATTACK_EVENT
11	STORED FUNCTION	Open response count function	INCIDENT_RESPONSE
12	STORED PROCEDURE	Update status and resolve procedure	INCIDENT_RESPONSE
13	TRIGGER	Audit on response update	INCIDENT_RESPONSE, RESPONSE_AUDIT
14	INSERT + UPDATE	Assign, change status, resolve	INCIDENT_RESPONSE, RESPONSE_AUDIT
15	TRANSACTION	COMMIT, ROLLBACK and SAVEPOINT	INCIDENT_RESPONSE
16	CONCURRENCY	Row locking in two sessions	INCIDENT_RESPONSE

Command 1: INNER JOIN

SQL COMMAND (PROPOSED)

```
SELECT ir.Response_ID,
ir.Incident_ID,
si.Incident_Type,
si.Severity,
ir.Lead_Investigator,
ir.Response_Action,
ir.Response_Status FROM
INCIDENT_RESPONSE ir
INNER JOIN SECURITY_INCIDENT si
        ON si.Incident_ID = ir.Incident_ID;
```

MYSQL OUTPUT

```
+-------------+-------------+---------------------+----------+-------------------------+-----------------------------------------------------------------------+-----------
----------+
| Response_ID | Incident_ID | Incident_Type       | Severity | Lead_Investigator       | Response_Action                                                       | Response_S
tatus     |
+-------------+-------------+---------------------+----------+-------------------------+-----------------------------------------------------------------------+-----------
----------+
| R001        | I001        | Cyber Attack        | High     | SOC Team                | Blocked malicious email and reset affected user credentials.          | Resolved
      |
| R002        | I002        | Unauthorized Access | Critical | Incident Response Team  | Disabled compromised accounts and reset authentication credentials.   | Resolved
      |
| R003        | I003        | Intrusion           | Critical | Network Security Team   | Blocked remote access and isolated affected network device.           | Contained
      |
| R004        | I004        | Malware             | High     | SOC Team                | Terminated PowerShell processes and isolated affected endpoint.       | Resolved
      |
| R005        | I005        | Network Attack      | Medium   | Security Operations Team| Blocked suspicious network activity and reviewed DNS records.         | Resolved
      |
| R006        | I006        | Malware             | Critical | Digital Forensics Team  | Collected forensic evidence and isolated compromised systems.         | Under Inve
stigation |
+-------------+-------------+---------------------+----------+-------------------------+-----------------------------------------------------------------------+-----------
----------+
6 rows in set (0.00 sec)
```

Explanation This command combines data from the INCIDENT_RESPONSE and SECURITY_INCIDENT tables using their common `Incident_ID`. It displays each response together with the type and severity of the incident it belongs to.

Expected result 6 rows, one for every response record, each matched to its incident (INCIDENT_RESPONSE holds 6 rows and every Incident_ID is filled).

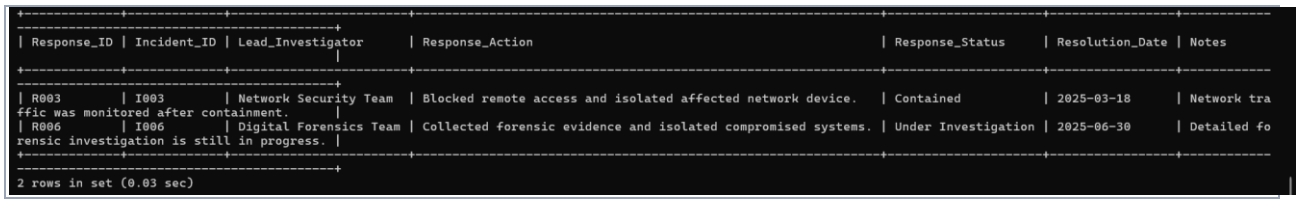
SQL concepts INNER JOIN, table aliases (`ir`, `si`), ON condition, foreign key `Incident_ID`.

Command 2: WHERE + ORDER BY

SQL COMMAND (PROPOSED)

```
SELECT *
FROM INCIDENT_RESPONSE
WHERE Response_Status <> 'Resolved'
ORDER BY Incident_ID;
```

MYSQL OUTPUT



```
+-------------+-------------+---------------------+-----------------------------------------------------------------+-------------------+-----------------+-------------
-------------------------------------------+
| Response_ID | Incident_ID | Lead_Investigator      | Response_Action                                                 | Response_Status     | Resolution_Date | Notes
                                           |
+-------------+-------------+---------------------+-----------------------------------------------------------------+-------------------+-----------------+-------------
-------------------------------------------+
| R003        | I003        | Network Security Team  | Blocked remote access and isolated affected network device.     | Contained           | 2025-03-18      | Network tra
ffic was monitored after containment.      |
| R006        | I006        | Digital Forensics Team | Collected forensic evidence and isolated compromised systems. | Under Investigation | 2025-06-30      | Detailed fo
rensic investigation is still in progress. |
+-------------+-------------+---------------------+-----------------------------------------------------------------+-------------------+-----------------+-------------
-------------------------------------------+
2 rows in set (0.03 sec)
```

Explanation This command displays only the responses whose status is not “Resolved”, which are the incidents still being worked on, and arranges them in ascending order of `Incident_ID`.

Expected result Only unresolved responses, sorted by Incident_ID. The number of rows depends on the Response_Status values stored in dbms.sql.

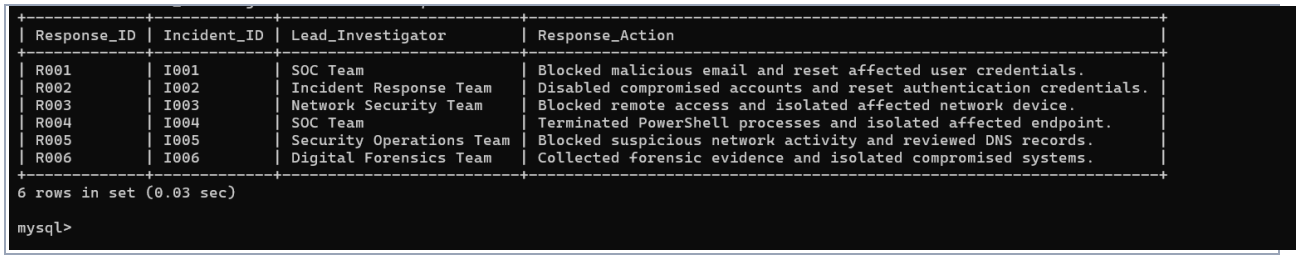
SQL concepts SELECT *, WHERE filter with the `<>` operator on a text value, ORDER BY (ascending is the default).

Command 3: LIKE

SQL COMMAND (PROPOSED)

```
SELECT Response_ID, Incident_ID, Lead_Investigator, Response_Action
FROM INCIDENT_RESPONSE
WHERE Lead_Investigator LIKE '%Team%';
```

MYSQL OUTPUT



```
+-------------+-------------+-------------------------+-----------------------------------------------------------------+
| Response_ID | Incident_ID | Lead_Investigator       | Response_Action                                                 |
+-------------+-------------+-------------------------+-----------------------------------------------------------------+
| R001        | I001        | SOC Team                | Blocked malicious email and reset affected user credentials.    |
| R002        | I002        | Incident Response Team  | Disabled compromised accounts and reset authentication credentials. |
| R003        | I003        | Network Security Team   | Blocked remote access and isolated affected network device.     |
| R004        | I004        | SOC Team                | Terminated PowerShell processes and isolated affected endpoint. |
| R005        | I005        | Security Operations Team | Blocked suspicious network activity and reviewed DNS records.  |
| R006        | I006        | Digital Forensics Team  | Collected forensic evidence and isolated compromised systems.   |
+-------------+-------------+-------------------------+-----------------------------------------------------------------+
6 rows in set (0.03 sec)

mysql>
```

Explanation This command uses LIKE to find responses whose `Lead_Investigator` contains the word “Team”, such as “SOC Team”. The `%` wildcard allows any characters before or after the word.

Expected result The responses led by a team rather than by a named person, including the “SOC Team” responses.

SQL concepts LIKE, `%` wildcard, pattern matching.

Command 4: LEFT JOIN + IS NULL

SQL COMMAND (PROPOSED)

```
SELECT si.Incident_ID, si.Incident_Type, si.Severity
FROM SECURITY_INCIDENT si
LEFT JOIN INCIDENT_RESPONSE ir
       ON si.Incident_ID = ir.Incident_ID
WHERE ir.Response_ID IS NULL;
```

MYSQL OUTPUT

```
+-------------+---------------+----------+
| Incident_ID | Incident_Type | Severity |
+-------------+---------------+----------+
| I007        | Malware       | Critical |
+-------------+---------------+----------+
1 row in set (0.00 sec)
```

Explanation This command finds **incidents that have no response yet**. `LEFT JOIN` keeps every incident, and `IS NULL` selects those for which no matching response record was found. It shows which incidents still need a response to be assigned.

Expected result Empty set (0 rows). In the sample data every one of the 6 incidents has a response record.

SQL concepts LEFT JOIN, IS NULL (finding rows with no match). `IS NULL` is used because a NULL value cannot be tested with `=`.

Command 5: MULTIPLE JOIN

SQL COMMAND (PROPOSED)

```
SELECT si.Incident_ID,        ir.Response_ID,
ir.Lead_Investigator AS Security_Analyst,
ir.Response_Status,        ae.Event_ID,
ac.Actor_Name,        ast.Asset_Variety FROM
INCIDENT_RESPONSE ir
INNER JOIN SECURITY_INCIDENT si ON si.Incident_ID = ir.Incident_ID
INNER JOIN ATTACK_EVENT ae      ON ae.Incident_ID = si.Incident_ID
INNER JOIN ACTOR ac             ON ac.Actor_ID = ae.Actor_ID
INNER JOIN ASSET ast            ON ast.Asset_ID = ae.Asset_ID
ORDER BY si.Incident_ID, ae.Event_ID;
```

MYSQL OUTPUT

Incident_ID	Response_ID	Security_Analyst	Response_Status	Event_ID	Actor_Name	Asset_Variety
I001	R001	SOC Team	Resolved	E001	APT1	Endpoint
I001	R001	SOC Team	Resolved	E002	APT1	Endpoint
I002	R002	Incident Response Team	Resolved	E003	APT29	Domain Controller
I002	R002	Incident Response Team	Resolved	E004	APT29	Domain Controller
I003	R003	Network Security Team	Contained	E005	APT3	Network Device
I003	R003	Network Security Team	Contained	E006	APT3	Server
I004	R004	SOC Team	Resolved	E007	OilRig	Employee Laptop
I004	R004	SOC Team	Resolved	E008	OilRig	Employee Laptop
I005	R005	Security Operations Team	Resolved	E009	Wizard Spider	Web Server
I005	R005	Security Operations Team	Resolved	E010	Wizard Spider	Web Server
I006	R006	Digital Forensics Team	Under Investigation	E011	Unknown Attacker	Server
I006	R006	Digital Forensics Team	Under Investigation	E012	Unknown Attacker	Server
I006	R006	Digital Forensics Team	Under Investigation	E013	Unknown Attacker	Domain Controller
I006	R006	Digital Forensics Team	Under Investigation	E014	Unknown Attacker	Domain Controller

14 rows in set (0.00 sec)

Explanation This command combines data from **five related tables** to show, for every attack event, the response that handles its incident, the analyst and status, the actor behind the event and the asset it affected. It tells the responder what has to be contained and restored.

Expected result 14 rows, one for every attack event. Each incident has one response, so each event appears once; all 14 events have an actor and an asset.

SQL concepts Multi-table INNER JOIN, aliases, column alias (`Security_Analyst`), ORDER BY on two columns.

Command 6: GROUP BY + COUNT

SQL COMMAND (PROPOSED)

```
SELECT Response_Status, COUNT(*) AS Response_Count
FROM INCIDENT_RESPONSE
GROUP BY Response_Status
ORDER BY Response_Count DESC;
```

MYSQL OUTPUT

Response_Status	Response_Count
Resolved	4
Contained	1
Under Investigation	1

3 rows in set (0.03 sec)

Explanation This command counts the **number of responses in each status**. `GROUP BY` groups the records by `Response_Status`, and `COUNT(*)` calculates the total in each group.

Expected result One row per distinct status; the Response_Count values add up to 6, the number of response records.

SQL concepts GROUP BY, COUNT(*) aggregate function, column alias, ORDER BY ... DESC.

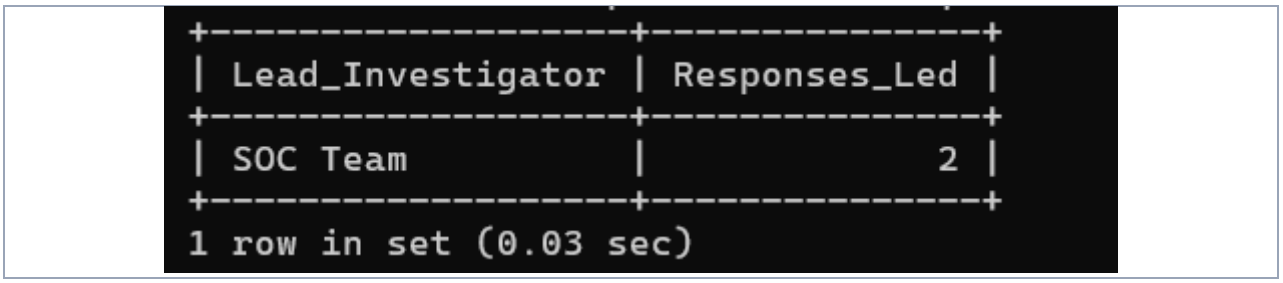
Command 7: HAVING

SQL COMMAND (PROPOSED)

```
SELECT Lead_Investigator, COUNT(*) AS Responses_Led
FROM INCIDENT_RESPONSE
GROUP BY Lead_Investigator
HAVING COUNT(*) > 1
```

```
ORDER BY Responses_Led DESC;
```

MYSQL OUTPUT



```
+-------------------+---------------+
| Lead_Investigator | Responses_Led |
+-------------------+---------------+
| SOC Team          |             2 |
+-------------------+---------------+
1 row in set (0.03 sec)
```

Explanation This command finds analysts who lead **more than one response**, which shows where the workload is concentrated. `HAVING` is used to filter the groups created by `GROUP BY`.

Expected result Only analysts or teams that appear in two or more response records. The result depends on the Lead_Investigator values in dbms.sql; an empty set means every response has a different lead.

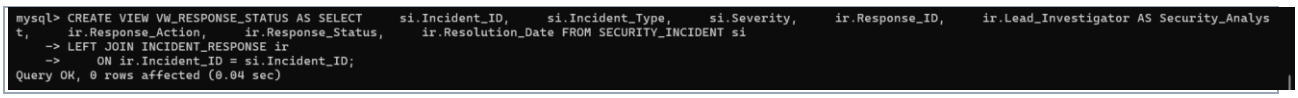
SQL concepts GROUP BY with HAVING. `WHERE` filters individual rows before grouping, while `HAVING` filters groups after the count is calculated.

Command 8: CREATE VIEW

SQL COMMAND (PROPOSED)

```
CREATE VIEW VW_RESPONSE_STATUS AS SELECT     si.Incident_ID,     si.Incident_Type,
si.Severity,     ir.Response_ID,     ir.Lead_Investigator AS Security_Analyst,
ir.Response_Action,     ir.Response_Status,     ir.Resolution_Date FROM
SECURITY_INCIDENT si
LEFT JOIN INCIDENT_RESPONSE ir
    ON ir.Incident_ID = si.Incident_ID;
```

MYSQL OUTPUT



```
mysql> CREATE VIEW VW_RESPONSE_STATUS AS SELECT     si.Incident_ID,     si.Incident_Type,     si.Severity,     ir.Response_ID,     ir.Lead_Investigator AS Security_Analys
t,     ir.Response_Action,     ir.Response_Status,     ir.Resolution_Date FROM SECURITY_INCIDENT si
    -> LEFT JOIN INCIDENT_RESPONSE ir
    ->     ON ir.Incident_ID = si.Incident_ID;
Query OK, 0 rows affected (0.04 sec)
```

Explanation This command creates a **view** named `VW_RESPONSE_STATUS` that combines incident and response information, including the status and the resolution date. `LEFT JOIN` keeps incidents that have no response record yet. A view acts like a virtual table that can be queried later.

Expected result "Query OK, 0 rows affected". Creating a view returns no rows; it only stores the query. Running `SELECT * FROM VW_RESPONSE_STATUS;` afterwards returns one row per incident.

SQL concepts CREATE VIEW, LEFT JOIN, column alias. A view stores the query, not the data, so it always shows current table contents.

Command 9: CASE

SQL COMMAND (PROPOSED)

```
SELECT
si.Incident_ID,
si.Severity,
ir.Response_Status,
CASE
        WHEN ir.Resolution_Date IS NOT NULL THEN 'Closed'
        WHEN si.Severity = 'Critical'       THEN 'Escalate Now'
        ELSE 'Monitor'
    END AS Response_Priority
FROM SECURITY_INCIDENT si
INNER JOIN INCIDENT_RESPONSE ir
        ON ir.Incident_ID = si.Incident_ID;
```

MYSQL OUTPUT

```
+-------------+----------+---------------------+-------------------+
| Incident_ID | Severity | Response_Status     | Response_Priority |
+-------------+----------+---------------------+-------------------+
| I001        | High     | Resolved            | Closed            |
| I002        | Critical | Resolved            | Closed            |
| I003        | Critical | Contained           | Closed            |
| I004        | High     | Resolved            | Closed            |
| I005        | Medium   | Resolved            | Closed            |
| I006        | Critical | Under Investigation | Closed            |
+-------------+----------+---------------------+-------------------+
6 rows in set (0.00 sec)
```

Explanation This command uses CASE to classify each response into a **priority level**: closed if a resolution date exists, otherwise “Escalate Now” for critical incidents and “Monitor” for the rest. It creates a new column called `Response_Priority`.

Expected result 6 rows. Resolved incidents show “Closed”; unresolved critical incidents (I003, I005, I006 have Critical severity) show “Escalate Now”; other unresolved incidents show “Monitor”.

SQL concepts CASE WHEN ... THEN ... ELSE ... END, IS NOT NULL, derived (calculated) column, alias.

Command 10: SUBQUERY

SQL COMMAND (PROPOSED)

```
SELECT Response_ID, Incident_ID, Lead_Investigator, Response_Status
FROM INCIDENT_RESPONSE
WHERE Incident_ID IN (
        SELECT Incident_ID
        FROM ATTACK_EVENT
        GROUP BY Incident_ID
        HAVING COUNT(*) > 2
      );
```

MYSQL OUTPUT

```
+-------------+-------------+-----------------------+--------------------+
| Response_ID | Incident_ID | Lead_Investigator     | Response_Status    |
+-------------+-------------+-----------------------+--------------------+
| R006        | I006        | Digital Forensics Team | Under Investigation |
+-------------+-------------+-----------------------+--------------------+
1 row in set (0.01 sec)
```

Explanation This command uses a **subquery** to find the responses for incidents that have more than two attack events, which are the largest incidents. The inner query identifies those incidents, and the outer query displays their responses.

Expected result 1 row: the response for incident I006, the only incident with more than two attack events (4 events).

SQL concepts Subquery with IN, GROUP BY and HAVING inside the subquery.

Command 11: STORED FUNCTION

SQL COMMAND (PROPOSED)

```
DELIMITER $$

CREATE FUNCTION fn_open_response_count(p_incident_id VARCHAR(10))
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO v_count
    FROM INCIDENT_RESPONSE
    WHERE Incident_ID = p_incident_id
      AND Response_Status <> 'Resolved';

    RETURN v_count;
END$$

DELIMITER ;

SELECT fn_open_response_count('I006') AS Open_Responses;
```

MYSQL OUTPUT

```
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
mysql> SELECT fn_open_response_count('I006') AS Open_Responses;
+----------------+
| Open_Responses |
+----------------+
|              1 |
+----------------+
1 row in set (0.01 sec)

mysql>
```

Explanation This command creates a **stored function** that takes an `Incident_ID` and returns the number of responses for that incident that are not yet resolved. The last line calls the function for I006. `DELIMITER $$` lets MySQL treat the semicolons inside the function body as part of the function instead of ending the command.

Expected result "Query OK, 0 rows affected" for the function; the call returns one row with one integer (0 if the response of I006 is resolved, 1 if it is still open).

SQL concepts CREATE FUNCTION, input parameter, RETURNS INT, READS SQL DATA, DECLARE, SELECT ... INTO, RETURN, DELIMITER.

Command 12: STORED PROCEDURE

SQL COMMAND (PROPOSED)

```
DELIMITER $$

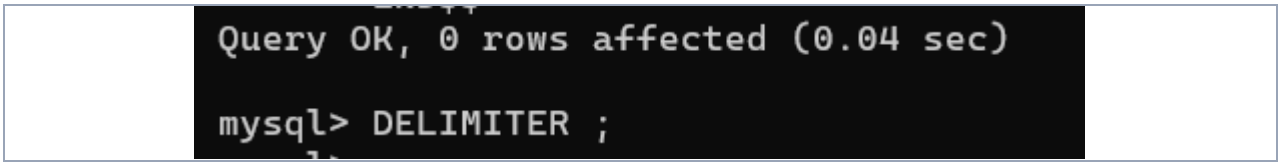
CREATE PROCEDURE sp_update_response_status(
    IN p_response_id VARCHAR(10),
```

```
    IN p_new_status  VARCHAR(50))
BEGIN
    IF p_new_status = 'Resolved' THEN
        UPDATE INCIDENT_RESPONSE
        SET Response_Status = p_new_status,
            Resolution_Date = CURDATE()
        WHERE Response_ID = p_response_id;
    ELSE
        UPDATE INCIDENT_RESPONSE
        SET Response_Status = p_new_status
        WHERE Response_ID = p_response_id;
    END IF;

    SELECT Response_ID, Incident_ID, Lead_Investigator,
           Response_Status, Resolution_Date
    FROM INCIDENT_RESPONSE
    WHERE Response_ID = p_response_id;
END$$

DELIMITER ;
```

MYSQL OUTPUT



Explanation This command creates a **stored procedure** that changes the status of a response. If the new status is “Resolved”, it also stores today’s date in `Resolution_Date`; otherwise it changes only the status. It then displays the updated record. The procedure is executed with `CALL` in Command 14.

Expected result “Query OK, 0 rows affected”: the procedure was created.

SQL concepts CREATE PROCEDURE, IN parameters, IF ... THEN ... ELSE, CURDATE(), CALL. Unlike a function, a procedure is run with CALL and can return a result set.

Command 13: TRIGGER

SQL COMMAND (PROPOSED)

```
CREATE TABLE IF NOT EXISTS RESPONSE_AUDIT (
    Audit_ID         INT AUTO_INCREMENT PRIMARY KEY,
    Response_ID      VARCHAR(10),
    Incident_ID      VARCHAR(10),
    Action_Type      VARCHAR(10) NOT NULL,
    Old_Status       VARCHAR(50),
    New_Status       VARCHAR(50),
    Old_Investigator VARCHAR(100),
    New_Investigator VARCHAR(100),
    Changed_By       VARCHAR(100),
    Changed_At       DATETIME DEFAULT CURRENT_TIMESTAMP
);

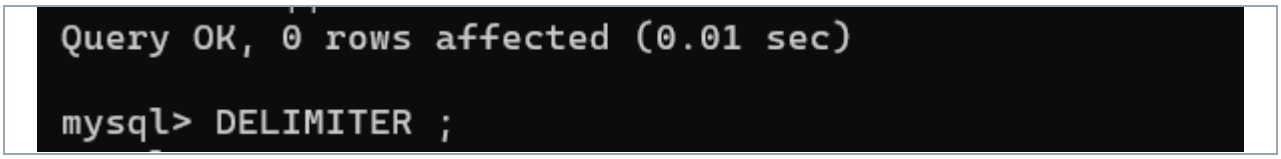
DELIMITER $$

CREATE TRIGGER trg_response_after_update
AFTER UPDATE ON INCIDENT_RESPONSE
```

```
FOR EACH ROW
BEGIN
    IF NOT (OLD.Response_Status <=> NEW.Response_Status)
       OR NOT (OLD.Lead_Investigator <=> NEW.Lead_Investigator) THEN
        INSERT INTO RESPONSE_AUDIT
            (Response_ID, Incident_ID, Action_Type,
             Old_Status, New_Status,
             Old_Investigator, New_Investigator, Changed_By)
        VALUES
            (NEW.Response_ID, NEW.Incident_ID, 'UPDATE',
             OLD.Response_Status, NEW.Response_Status,
             OLD.Lead_Investigator, NEW.Lead_Investigator,
             CURRENT_USER());
    END IF;
END$$

DELIMITER ;
```

MYSQL OUTPUT



Explanation This command first creates the log table `RESPONSE_AUDIT`, then creates a **trigger** that automatically records a row in it whenever the status or the lead investigator of a response changes. `OLD` and `NEW` hold the values before and after the update, `<=>` is the null-safe comparison, and `CURRENT_USER()` stores the MySQL account that made the change.

Expected result "Query OK, 0 rows affected" for the table and for the trigger.

SQL concepts CREATE TABLE, CREATE TRIGGER, AFTER UPDATE, FOR EACH ROW, OLD.column and NEW.column, IF inside a trigger, CURRENT_USER().

> **REVIEW NOTE** RESPONSE_AUDIT is a new log table that is not in dbms.sql. It has no foreign key, so audit rows stay even if a response is later deleted. If the team prefers not to add it, this command and Command 14 can be dropped without affecting the other commands.

Command 14: INSERT + UPDATE (RESPONSE LIFECYCLE)

SQL COMMAND (PROPOSED)

```
-- 1. Assign a new response to incident I004
INSERT INTO INCIDENT_RESPONSE
    (Response_ID, Incident_ID, Lead_Investigator, Response_Action,
     Response_Status, Resolution_Date, Notes)
VALUES
    ('R999', 'I004', 'SOC Team', 'Test - isolate affected host',
     'Open', NULL, 'Member 4 demo record');

-- 2. Change the status
CALL sp_update_response_status('R999', 'In Progress');

-- 3. Resolve the incident
CALL sp_update_response_status('R999', 'Resolved');

-- 4. Check the audit trail written by the trigger
SELECT Audit_ID, Response_ID, Old_Status, New_Status, Changed_By
```

```
FROM RESPONSE_AUDIT
ORDER BY Audit_ID;
```

MYSQL OUTPUT

```
mysql> -- 1. Assign a new response to incident I004
mysql> INSERT INTO INCIDENT_RESPONSE
    ->     (Response_ID, Incident_ID, Lead_Investigator, Response_Action,
    ->      Response_Status, Resolution_Date, Notes)
    -> VALUES
    ->     ('R999', 'I004', 'SOC Team', 'Test - isolate affected host',
    ->      'Open', NULL, 'Member 4 demo record');
Query OK, 1 row affected (0.03 sec)

mysql> -- 2. Change the status
mysql> CALL sp_update_response_status('R999', 'In Progress');
+-------------+-------------+-------------------+-----------------+-----------------+
| Response_ID | Incident_ID | Lead_Investigator | Response_Status | Resolution_Date |
+-------------+-------------+-------------------+-----------------+-----------------+
| R999        | I004        | SOC Team          | In Progress     | NULL            |
+-------------+-------------+-------------------+-----------------+-----------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> -- 3. Resolve the incident
mysql> CALL sp_update_response_status('R999', 'Resolved');
+-------------+-------------+-------------------+-----------------+-----------------+
| Response_ID | Incident_ID | Lead_Investigator | Response_Status | Resolution_Date |
+-------------+-------------+-------------------+-----------------+-----------------+
| R999        | I004        | SOC Team          | Resolved        | 2026-10-17      |
+-------------+-------------+-------------------+-----------------+-----------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> -- 4. Check the audit trail written by the trigger
mysql> SELECT Audit_ID, Response_ID, Old_Status, New_Status, Changed_By
    -> FROM RESPONSE_AUDIT
    -> ORDER BY Audit_ID;
+----------+-------------+-------------+-------------+----------------+
| Audit_ID | Response_ID | Old_Status  | New_Status  | Changed_By     |
+----------+-------------+-------------+-------------+----------------+
|        1 | R999        | Open        | In Progress | root@localhost |
|        2 | R999        | In Progress | Resolved    | root@localhost |
+----------+-------------+-------------+-------------+----------------+
2 rows in set (0.00 sec)
```

Explanation This command walks through the **whole response lifecycle** on a test record, R999. The INSERT assigns the response, the first CALL changes its status, the second CALL resolves it and sets the resolution date, and the final SELECT shows the rows that the trigger wrote automatically. A test record is used so that the real data of the team is not changed.

Expected result INSERT: 1 row affected. Each CALL displays the updated record. The audit query returns 2 rows for R999: Open to In Progress, and In Progress to Resolved, each with the changing account. No audit row is written by the INSERT because the trigger is an AFTER UPDATE trigger.

SQL concepts INSERT with a column list, CALL, UPDATE through a procedure, trigger execution, ORDER BY. INCIDENT_RESPONSE allows several responses per incident, so I004 can receive an extra response.

Command 15: TRANSACTION (COMMIT, ROLLBACK, SAVEPOINT)

SQL COMMAND (PROPOSED)

```
-- Part A: ROLLBACK undoes the whole transaction
START TRANSACTION;
UPDATE INCIDENT_RESPONSE
SET Response_Status = 'Escalated'
WHERE Response_ID = 'R999';
SELECT Response_ID, Response_Status
FROM INCIDENT_RESPONSE WHERE Response_ID = 'R999';
ROLLBACK;
SELECT Response_ID, Response_Status
FROM INCIDENT_RESPONSE WHERE Response_ID = 'R999';

-- Part B: SAVEPOINT keeps one change and undoes the next
START TRANSACTION;
UPDATE INCIDENT_RESPONSE
SET Notes = 'Reviewed by SOC lead'
WHERE Response_ID = 'R999';
SAVEPOINT after_notes;
UPDATE INCIDENT_RESPONSE
SET Response_Status = 'Escalated'
WHERE Response_ID = 'R999';
ROLLBACK TO SAVEPOINT after_notes;
COMMIT;
SELECT Response_ID, Response_Status, Notes
FROM INCIDENT_RESPONSE WHERE Response_ID = 'R999';
```

MYSQL OUTPUT

MySQL 8.0 Command Line Cli

```
mysql> -- Part A: ROLLBACK undoes the whole transaction
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE INCIDENT_RESPONSE
    -> SET Response_Status = 'Escalated'
    -> WHERE Response_ID = 'R999';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> SELECT Response_ID, Response_Status
    -> FROM INCIDENT_RESPONSE WHERE Response_ID = 'R999';
+-------------+-----------------+
| Response_ID | Response_Status |
+-------------+-----------------+
| R999        | Escalated       |
+-------------+-----------------+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT Response_ID, Response_Status
    -> FROM INCIDENT_RESPONSE WHERE Response_ID = 'R999';
+-------------+-----------------+
| Response_ID | Response_Status |
+-------------+-----------------+
| R999        | Escalated       |
+-------------+-----------------+
1 row in set (0.00 sec)

mysql> -- Part B: SAVEPOINT keeps one change and undoes the next
mysql> START TRANSACTION;
```

```
MySQL 8.0 Command Line Cli
mysql> -- Part B: SAVEPOINT keeps one change and undoes the next
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE INCIDENT_RESPONSE
    -> SET Notes = 'Reviewed by SOC lead'
    -> WHERE Response_ID = 'R999';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> SAVEPOINT after_notes;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE INCIDENT_RESPONSE
    -> SET Response_Status = 'Escalated'
    -> WHERE Response_ID = 'R999';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> ROLLBACK TO SAVEPOINT after_notes;
Query OK, 0 rows affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT Response_ID, Response_Status, Notes
    -> FROM INCIDENT_RESPONSE WHERE Response_ID = 'R999';
+-------------+-----------------+----------------------+
| Response_ID | Response_Status | Notes                |
+-------------+-----------------+----------------------+
| R999        | Escalated       | Reviewed by SOC lead |
+-------------+-----------------+----------------------+
1 row in set (0.00 sec)
```

Explanation This command shows a **transaction**. `START TRANSACTION` switches off automatic saving, so the changes are saved only by `COMMIT`. In Part A the status is changed to “Escalated”, seen inside the transaction, and then `ROLLBACK` restores “Resolved”. The trigger’s audit row is part of the same transaction, so it is also undone.
In Part B the note is kept, the later status change is undone with `ROLLBACK TO SAVEPOINT`, and `COMMIT` saves the note.

Expected result Part A: the first SELECT shows “Escalated”, the second shows the earlier status “Resolved”. Part B: the final SELECT shows status “Resolved” with Notes = “Reviewed by SOC lead”.

SQL concepts START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT, ROLLBACK TO SAVEPOINT, atomicity. Transactions need the InnoDB storage engine, which is the MySQL default.

Command 16: CONCURRENCY (ROW LOCKING IN TWO SESSIONS)

SQL COMMAND (PROPOSED)

```
-- SESSION 1
START TRANSACTION;
SELECT Response_ID, Lead_Investigator, Response_Status
FROM INCIDENT_RESPONSE
WHERE Response_ID = 'R999'
```

```
FOR UPDATE;
UPDATE INCIDENT_RESPONSE
SET Lead_Investigator = 'Senior SOC Analyst'
WHERE Response_ID = 'R999';

-- SESSION 2 (second terminal, while Session 1 is still open)
START TRANSACTION;
UPDATE INCIDENT_RESPONSE
SET Response_Status = 'Escalated'
WHERE Response_ID = 'R999';      -- waits for Session 1

-- SESSION 1
COMMIT;                          -- Session 2 can now continue

-- SESSION 2
COMMIT;
SELECT @@transaction_isolation;
```

MYSQL OUTPUT

```
mysql> UPDATE INCIDENT_RESPONSE
    -> SET Lead_Investigator = 'Senior SOC Analyst'
    -> WHERE Response_ID = 'R999';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> -- SESSION 2 (second terminal, while Session 1 is still open)
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.03 sec)

mysql> UPDATE INCIDENT_RESPONSE
    -> SET Response_Status = 'Escalated'
    -> WHERE Response_ID = 'R999';      -- waits for Session 1
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> -- SESSION 1
mysql> COMMIT;                          -- Session 2 can now continue
Query OK, 0 rows affected (0.01 sec)

mysql> -- SESSION 2
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @@transaction_isolation;
+-------------------------+
| @@transaction_isolation |
+-------------------------+
| REPEATABLE-READ         |
+-------------------------+
1 row in set (0.00 sec)
```

Explanation This command shows **concurrency control** with two MySQL sessions open at the same time. `SELECT ... FOR UPDATE` makes Session 1 lock the row of R999. When Session 2 tries to update the same row,

it has to **wait** instead of overwriting the change. After Session 1 commits and releases the lock, the update of Session 2 runs. Both changes are kept and no update is lost. The last query shows the isolation level in use.

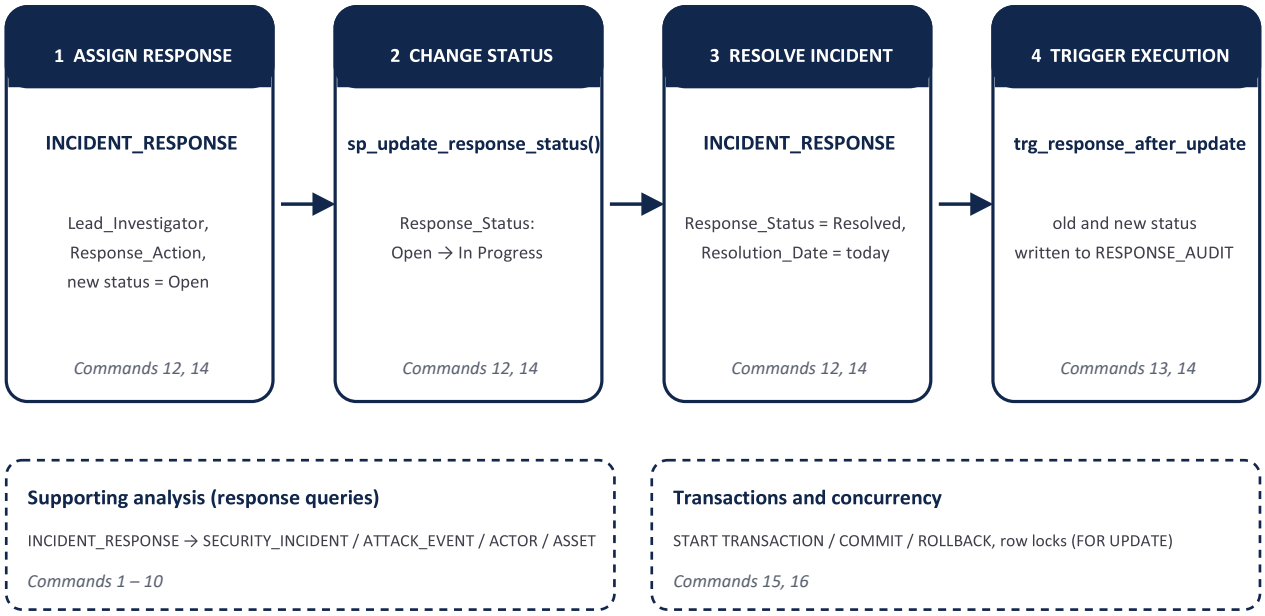
Expected result Session 2's UPDATE does not return until Session 1 runs COMMIT (it fails with a lock-wait timeout after 50 seconds by default). Afterwards both changes are present and the audit table holds a row for each. The isolation level is REPEATABLE READ unless the server was changed.

SQL concepts Row-level locks, SELECT ... FOR UPDATE, lock waiting, isolation level, lost update prevention. A plain SELECT in Session 2 would not wait, because InnoDB reads a consistent snapshot without locking.

REVIEW NOTE Run this command with two separate MySQL terminals. If the isolation query fails on an older MySQL version, use SELECT @@tx_isolation; instead.

4.4 Response Workflow

The diagram shows the Member 4 response module as a four-stage flow. Each stage maps to a table, procedure or trigger in the database and to the commands that demonstrate it.



Arrows show the response sequence. Status and resolution are stored in INCIDENT_RESPONSE; SECURITY_INCIDENT has no status column.

Figure 4.1: Member 4 response workflow

Stage	Database object	What it represents
1. Assign response	INCIDENT_RESPONSE	A new response record names the lead investigator and the response action for an incident.
2. Change status	sp_update_response_status	The status of the response is updated while the work continues (Open, In Progress and so on).
3. Resolve incident	INCIDENT_RESPONSE (Response_Status, Resolution_Date)	The status becomes Resolved and the resolution date is stored.

4. Trigger execution	trg_response_after_update, RESPONSE_AUDIT	Every change of status or lead investigator is logged with the old value, the new value, the account and the time.

4.4.1 How the workflow represents Member 4's module

A response starts from an incident record. The analyst assigns a response (Command 14), changes its status (Commands 12 and 14) and finally resolves it (Commands 12 and 14). Each change fires the audit trigger (Command 13), so the history of the incident's response can be read from RESPONSE_AUDIT.

The incident and its response are linked only through `Incident_ID`: INCIDENT_RESPONSE holds the foreign key, and the view and the queries use this shared key to bring the two tables together. Two supporting parts complete the module: the response queries (Commands 1 to 10) show what each response has to deal with, and the transaction and concurrency examples (Commands 15 and 16) show that the changes stay correct when they are grouped together or made by several analysts at the same time.

4.5 ER Diagram — Security Incident Attack-Path Database

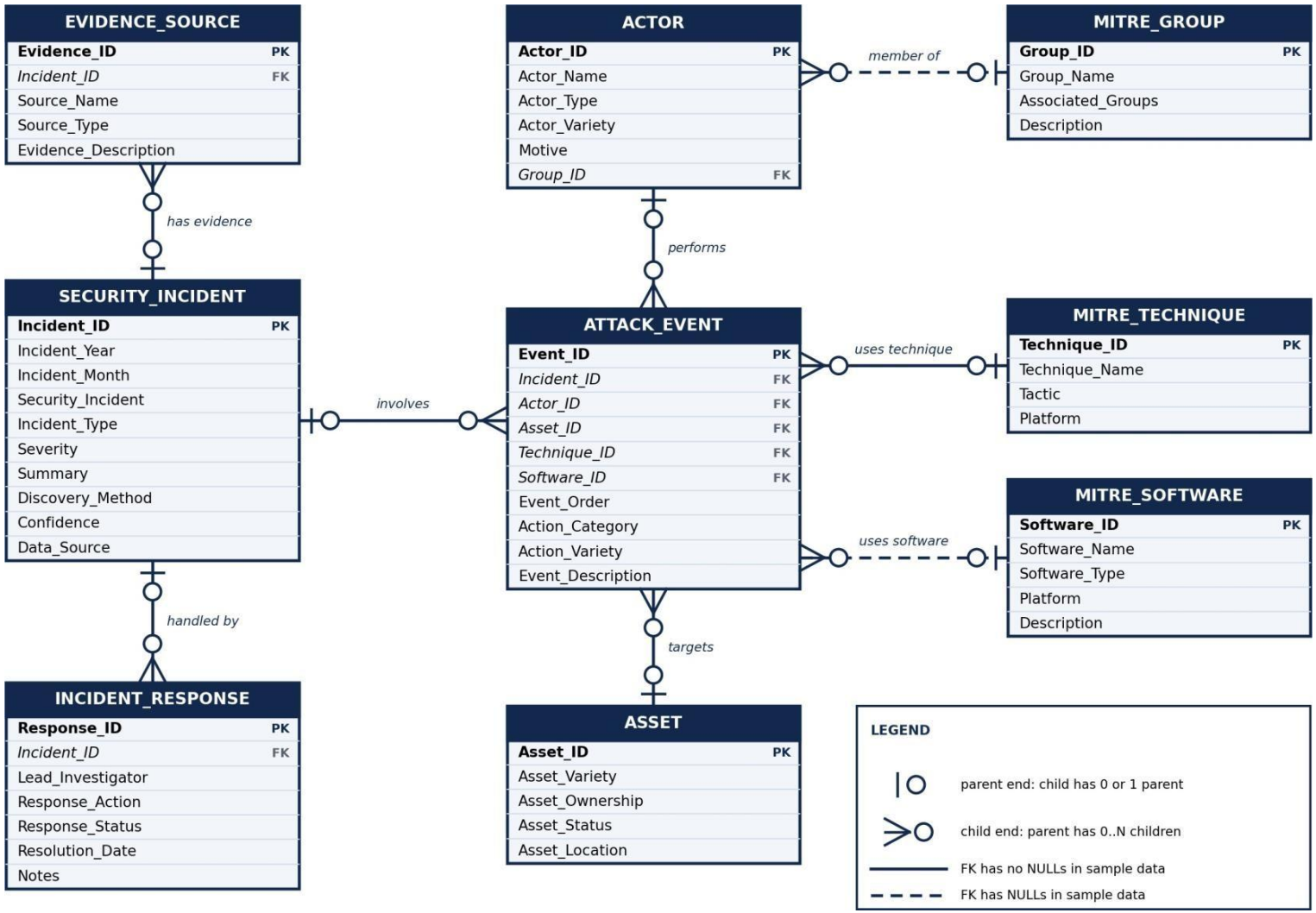


Figure 4.2: ER diagram drawn from the foreign keys declared in dbms.sql (crow's foot notation; PK = primary key, FK = foreign key)

4.6 Enhanced Entity–Relationship (EER) Diagram

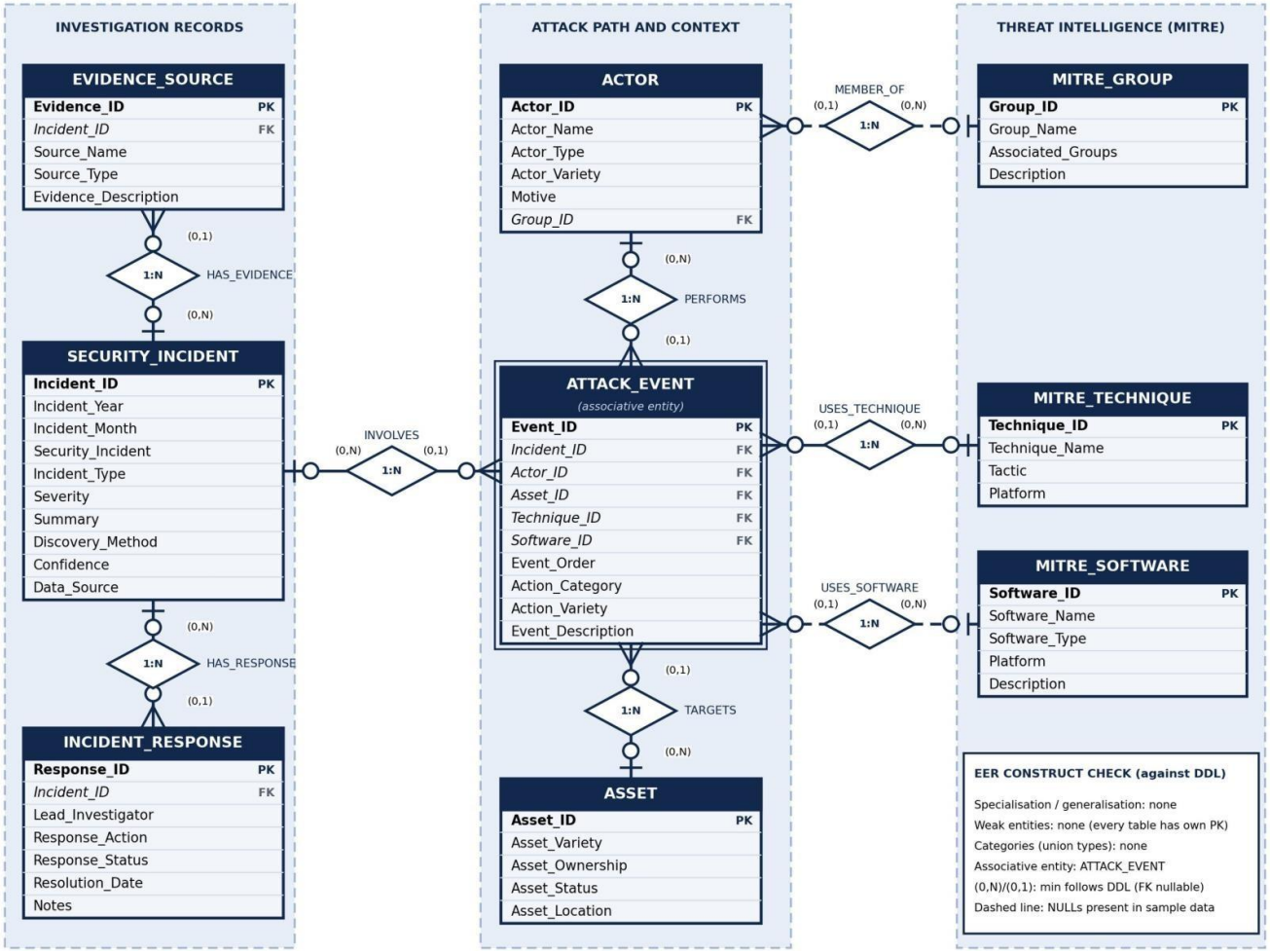


Figure 4.3: EER diagram with participation constraints, grouped by subject area (same entities and relationships as Figure 4.2)

4.7 Functional Dependencies

A functional dependency X → Y means that the value of X determines the value of Y. The dependencies below come from the primary keys and foreign keys declared in dbms.sql. No UNIQUE constraints are declared, so the primary key is the only candidate key identified for each table.

4.7.1 Dependencies in each relation

Relation	Functional dependencies
INCIDENT_RESPONSE	Response_ID → Incident_ID, Lead_Investigator, Response_Action, Response_Status, Resolution_Date, Notes
SECURITY_INCIDENT	Incident_ID → Incident_Year, Incident_Month, Security_Incident, Incident_Type, Severity, Summary, Discovery_Method, Confidence, Data_Source
EVIDENCE_SOURCE	Evidence_ID → Incident_ID, Source_Name, Source_Type, Evidence_Description
ATTACK_EVENT	Event_ID → Incident_ID, Actor_ID, Asset_ID, Technique_ID, Software_ID, Event_Order, Action_Category, Action_Variety, Event_Description
ACTOR	Actor_ID → Actor_Name, Actor_Type, Actor_Variety, Motive, Group_ID
ASSET	Asset_ID → Asset_Variety, Asset_Ownership, Asset_Status, Asset_Location
MITRE_GROUP	Group_ID → Group_Name, Associated_Groups, Description
MITRE_TECHNIQUE	Technique_ID → Technique_Name, Tactic, Platform
MITRE_SOFTWARE	Software_ID → Software_Name, Software_Type, Platform, Description
RESPONSE_AUDIT (added in Command 13)	Audit_ID → Response_ID, Incident_ID, Action_Type, Old_Status, New_Status, Old_Investigator, New_Investigator, Changed_By, Changed_At

4.7.2 Observations

- Foreign keys such as INCIDENT_RESPONSE.Incident_ID refer to the primary key of another table. Details of the incident are stored once in SECURITY_INCIDENT and are not repeated in INCIDENT_RESPONSE.
- Response_Status does not determine Resolution_Date. Several responses can share the status “Resolved” and still have different resolution dates, so only Response_ID determines both.
- Lead_Investigator does not determine Response_Action. The same analyst or team can carry out different actions in different responses, so the schema declares no dependency between the two.
- Incident_ID does not determine Response_ID in this schema. No UNIQUE constraint exists, so one incident can have several responses, as the test in Command 14 shows.

4.8 Normalization

The normal form of each relation is judged from the functional dependencies in Section 4.7 and the declared constraints. This is an analysis of the existing design; the team schema has not been changed.

Relation	1NF	2NF	3NF / BCNF	Comment
INCIDENT_RESPONSE	Yes	Yes	Yes	Lead_Investigator and Response_Status are plain text, not keys to other tables.
SECURITY_INCIDENT	Yes	Yes	Yes	Single-column key; no non-key attribute determines another.
EVIDENCE_SOURCE	Yes	Yes	Yes	Incident details are not repeated; only Incident_ID is stored.
ATTACK_EVENT	Yes	Yes	Yes	Holds only keys and event-specific attributes.
ACTOR	Yes	Yes	Yes	Group details live in MITRE_GROUP.
ASSET	Yes	Yes	Yes	All attributes depend on Asset_ID only.
MITRE_GROUP	Partly	Yes	Yes	Associated_Groups holds a list of names in one column.
MITRE_TECHNIQUE	Partly	Yes	Yes	Platform (and sometimes Tactic) holds several values in one column.
MITRE_SOFTWARE	Partly	Yes	Yes	Platform can hold several values in one column.
RESPONSE_AUDIT (added)	Yes	Yes	Yes	Audit_ID is the only determinant; the old and new values are separate atomic columns.

4.8.1 Reasoning

- **2NF.** Every table has a single-column primary key, so no attribute can depend on only part of a key. Once a table is in 1NF it is therefore in 2NF.
- **3NF and BCNF.** In every table the only determinant is the primary key. No non-key attribute determines another non-key attribute, so there are no transitive dependencies. In INCIDENT_RESPONSE, neither Response_Status nor Lead_Investigator determines any other column.
- **1NF.** Most columns are atomic. A few descriptive text columns store comma-separated lists, for example Platform = “Windows, Linux, macOS”. They are never used as keys or in joins, so they do not affect the queries in this report, but strictly they are not atomic.

4.8.2 Possible improvements (proposed, not implemented)

- Add NOT NULL to foreign-key columns that are always filled (for example INCIDENT_RESPONSE.Incident_ID), so the database itself enforces that every response belongs to an incident.
- Restrict Response_Status to a fixed set of values, with a CHECK constraint or a small lookup table, so that a mistyped status cannot be stored and the status values used in Commands 2, 9, 11 and 12 are guaranteed.
- Store the analyst in a separate table and keep only its key in INCIDENT_RESPONSE, so that a team or person is named in one place. This changes the team schema, so it would need the whole team’s agreement.
- Move the list-valued columns (Platform, Tactic, Associated_Groups) into separate linking tables if platformor tactic-level queries become necessary.

4.9 ER Relationships and Cardinalities

All relationships below were taken from the FOREIGN KEY clauses in dbms.sql. In each row the parent table holds the primary key and the child table holds the foreign key.

#	Parent (one side)	Child (many side)	Foreign key	Cardinality	Participation of the foreign key
1	SECURITY_INCIDENT	EVIDENCE_SOURCE	Incident_ID	1 : N	Optional in DDL; filled in all 8 rows
2	SECURITY_INCIDENT	INCIDENT_RESPONSE	Incident_ID	1 : N	Optional in DDL; filled in all 6 rows
3	SECURITY_INCIDENT	ATTACK_EVENT	Incident_ID	1 : N	Optional in DDL; filled in all 14 rows
4	ACTOR	ATTACK_EVENT	Actor_ID	1 : N	Optional in DDL; filled in all 14 rows
5	ASSET	ATTACK_EVENT	Asset_ID	1 : N	Optional in DDL; filled in all 14 rows
6	MITRE_TECHNIQUE	ATTACK_EVENT	Technique_ID	1 : N	Optional in DDL; filled in all 14 rows
7	MITRE_SOFTWARE	ATTACK_EVENT	Software_ID	1 : N	Optional; NULL in 7 of 14 rows
8	MITRE_GROUP	ACTOR	Group_ID	1 : N	Optional; NULL for actor A006

4.9.1 Notes on the cardinalities

- Every foreign-key column in dbms.sql allows NULL, because none is declared NOT NULL. Participation is therefore optional according to the DDL. Relationships 7 and 8 are optional in practice too, because the sample data contains NULLs there (shown as dashed lines in the diagrams).
- Relationship 2 is the one used most by this module. It is 1 : N because the schema allows several response records for one incident, although the sample data has one response per incident. This is why Command 14 can assign an extra response to I004, and why Command 4 returns no row for the sample data.
- ATTACK_EVENT links five parent tables, so it acts as an associative entity. Through it, incidents and actors, incidents and assets, incidents and techniques, and actors and assets are related many-to-many.
- There is no direct foreign key between INCIDENT_RESPONSE and EVIDENCE_SOURCE, or between INCIDENT_RESPONSE and ATTACK_EVENT. They are connected only through SECURITY_INCIDENT, which is why Command 5 joins through it.
- RESPONSE_AUDIT is a log table without a foreign key. It repeats Response_ID and Incident_ID as plain values so that audit rows stay available if a response is deleted, and for this reason it is not drawn in the ER and EER diagrams.

4.9.2 Enhanced ER constructs

The schema contains no specialisation or generalisation, no weak entities and no category (union) types: every table has its own primary key and no table inherits from another. The EER diagram therefore adds only

constructs the schema supports: participation constraints in (min, max) form, ATTACK_EVENT shown as an associative entity, and the grouping of tables by subject area.

4.10 Transactions, Concurrency and Serializability

Commands 15 and 16 show transactions and locking in practice. This section explains the ideas behind them and analyses two schedules of the response module for serializability. The schedules are worked out on paper to illustrate the theory; they were not run on the database.

4.10.1 Transactions in the response module

A transaction groups SQL statements into one logical unit of work. Resolving a response is a good example: the update of INCIDENT_RESPONSE and the audit row written by the trigger must both happen or both be undone. The four ACID properties apply as follows.

Property	Meaning	In this module
Atomicity	All statements succeed or none does.	ROLLBACK undoes the status change and the audit row together (Command 15, Part A).
Consistency	A transaction moves the database from one valid state to another.	Primary keys and foreign keys keep Incident_ID valid; a transaction cannot leave a response pointing to a missing incident.
Isolation	Concurrent transactions do not disturb each other.	Row locks and isolation levels keep two analysts from overwriting each other (Command 16).
Durability	Committed changes survive a failure.	After COMMIT, InnoDB keeps the change in its redo log.

4.10.2 Concurrency control in MySQL

Without control, two analysts working on the same response can cause a **lost update**: both read the record, both change it, and the second write replaces the first. InnoDB, the default MySQL storage engine, prevents this with row-level locks and with isolation levels. If two transactions wait for each other's locks, InnoDB detects the deadlock and rolls one of them back with error 1213, and the application then repeats that transaction.

Isolation level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ (InnoDB default)	Prevented	Prevented	Largely prevented by InnoDB
SERIALIZABLE	Prevented	Prevented	Prevented

At SERIALIZABLE level, InnoDB treats every plain SELECT as a locking read, so transactions behave as if they ran one after the other. The default REPEATABLE READ level is enough for Command 16 because the writer uses `FOR UPDATE` to lock the row it intends to change.

4.10.3 Serializability

A schedule is **conflict serializable** if it can be turned into a serial schedule by swapping non-conflicting operations. Two operations conflict when they belong to different transactions, work on the same data item and at least one is a write. The precedence graph has one node per transaction and an edge Ti → Tj when an operation of Ti conflicts with a later operation of Tj. A schedule is conflict serializable exactly when this graph has no cycle.

Take two responses, A and B, of the same incident, as data items, and two transactions. **T1** resolves both responses: R1(A), W1(A), R1(B), W1(B). **T2** reads both for a status report: R2(A), R2(B).

Schedule S2 — interleaved

Step	T1	T2
1	R1(A)	
2	W1(A)	
3		R2(A)
4		R2(B)
5	R1(B)	
6	W1(B)	

Conflicts. W1(A) comes before R2(A), which gives the edge T1 → T2. R2(B) comes before W1(B), which gives the edge T2 → T1. The graph in Figure 4.4 has a cycle, so S2 is **not conflict serializable**. In practice T2 would report A as resolved but B as still open, a picture of the incident that never existed at any single moment.

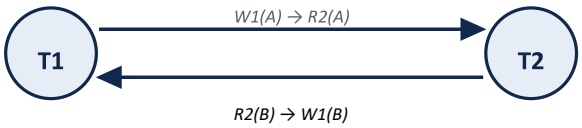


Figure 4.4: Precedence graph of schedule S2 (not serializable)

Schedule S4 — interleaved but serializable

Step	T1	T2
1	R1(A)	
2	W1(A)	
3		R2(A)
4	R1(B)	
5	W1(B)	
6		R2(B)

Conflicts. W1(A) before R2(A) and W1(B) before R2(B) both give the edge T1 → T2, and there is no edge back. The graph in Figure 4.5 has no cycle, so S4 is **conflict serializable** and is equivalent to the serial schedule T1 followed by T2 (the serial schedule S3 is R1(A), W1(A), R1(B), W1(B), R2(A), R2(B)). T2 sees both responses as resolved.



Figure 4.5: Precedence graph of schedule S4 (serializable)

MySQL avoids a schedule like S2 by isolation. Under REPEATABLE READ, T2 reads one consistent snapshot of both rows. Under SERIALIZABLE, T2's reads wait for T1's locks. In both cases the result is equal to a serial order. A lost update, the other classic problem, appears as the schedule R1(A), R2(A), W1(A), W2(A): it has the edges T1 → T2 and T2 → T1, so it also contains a cycle and is not serializable. The row lock taken in Command 16 prevents it.

4.11 Conclusion

The Security Incident Attack-Path Database was designed to organize security incident information in a structured and interconnected manner. It links incident details with actors, affected assets, MITRE ATT&CK techniques, software, evidence sources, and incident-response activities through primary and foreign key relationships. This integrated design helps users trace an incident from its initial record through the sequence of attack events and the associated investigation.

The project applies important relational database concepts through nine related tables: MITRE_GROUP, MITRE_TECHNIQUE, MITRE_SOFTWARE, SECURITY_INCIDENT, ACTOR, ASSET, ATTACK_EVENT, EVIDENCE_SOURCE, and INCIDENT_RESPONSE. The ATTACK_EVENT table plays a central role in reconstructing attack paths by recording individual events and their execution order. SQL joins make it possible to combine related information and present the sequence of events in a clear and meaningful form.

The implementation demonstrates core DBMS operations and concepts, including data definition and manipulation, joins, nested queries, aggregate functions, GROUP BY, HAVING, views, stored functions, stored procedures, and triggers. The project also incorporates database design, normalization, functional dependencies, transaction management, concurrency control, and serializability. These concepts help demonstrate the importance of data consistency, integrity, and efficient information retrieval in a relational database system.

The incident-response module further extends the project by recording investigation activities, responsible personnel, response actions, and incident status. The browser-based dashboard provides a user-friendly interface for exploring incident records, visualizing attack sequences, reviewing evidence, and examining response information. The dashboard currently operates as a prototype using illustrative sample data; connecting it directly to the live MySQL database would be a further enhancement.

Overall, the project demonstrates how database management principles can be applied to a cybersecurity investigation scenario. It provides a foundation for organizing incident information, reconstructing attack paths, and supporting investigation workflows. Future enhancements may include live database integration, role-based authentication, improved validation, and the use of verified real-world incident data. The project therefore fulfills its educational objective of applying DBMS concepts to a practical and relevant cybersecurity problem.